

Semantic SQL via Workload-Aware Schema Enrichment

Wenchao Bai¹, Wenfei Fan^{2,3}, Weilong Ren², Zihuan Xu², Sihan Wang², Jiahui Jin¹
¹Southeast University, China ²Shenzhen Institute of Computing Sciences, China ³University of Edinburgh, UK
 {wbai, jjin}@seu.edu.cn, {wenfei, renweilong, xuzihuan, wangsihan}@sics.ac.cn

ABSTRACT

This paper presents LiteDBX, a lightweight database extender for recurring semantic queries over relational and unstructured data. Given a database, associated unstructured data, a semantic-query workload, and a small human-labeling budget, LiteDBX turns recurring semantic computation into workload-specific relational state: it materializes only query-relevant semantic attributes and rewrites semantic predicates into reusable SQL, avoiding repeated online LLM calls. We formalize this workload-aware schema-enrichment problem and show it is NP-hard. We develop SEQR, which jointly selects semantic attributes and their SQL rewrites under limited supervision, with an end-to-end guarantee on macro-averaged F_1 . As data and query intent evolve, LiteDBX certifies reusable state and selectively refreshes affected attributes and rewrites. On real-life datasets, it improves the accuracy over the best-performing label-assisted (resp. zero-shot) method by 71.6% (resp. 16.5%), reducing monetary cost by up to 99.9% and latency by up to 99.2% (resp. 98.9%).

PVLDB Reference Format:

Wenchao Bai¹, Wenfei Fan^{2,3}, Weilong Ren², Zihuan Xu², Sihan Wang², Jiahui Jin¹. Semantic SQL via Workload-Aware Schema Enrichment. PVLDB, 20(1): XXX-XXX, 2027.
 doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/SICS-Fundamental-Research-Center/litedbx>.

1 INTRODUCTION

The need to jointly query relational and unstructured data is increasingly evident [52]. In applications such as clinical decision support, decisions often depend on both structured records (e.g., demographics, lab tests, medications) and unstructured data (e.g., radiology images or physician notes) [46], to enable more accurate diagnosis, treatment planning, and patient prioritization [65]. We call conditions imposed on unstructured data *semantic conditions*, which capture concepts (e.g., clinical findings) not directly represented as structured attributes; queries combining standard SQL predicates with such conditions are *semantic queries*.

Supporting such queries presents a central tradeoff. At one end of the spectrum, *multi-modal databases* (MMDBs) [10, 27, 28, 34, 38, 50, 51, 57] interpret semantic predicates online with large language models (LLMs), either zero-shot [10, 34, 38, 50, 51, 57] or with a few human-provided labels as in ThalamusDB [27, 28], but repeated inference incurs cost, latency, variability, and opaque de-

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
 Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
 doi:XX.XX/XXX.XX

id	time (A_0)	age (A_1)	spo ₂ (A_2)	RR (A_3)	CRP (A_4)	opacity (B_1)	bilateral (B_2)	atelectasis (B_3)	effusion (B_4)	notepna (B_5)	lobarconsol (B_6)	EMG (Y)
t_1	2020-12	76	89	33	140	0.78	1	0	1	—	—	Yes
t_2	2020-12	82	88	34	160	0.82	1	0	1	—	—	Yes
t_3	2020-12	69	95	20	18	0.12	0	0	0	—	—	No
t_4	2021-01	70	90	32	120	0.70	1	0	1	—	—	Yes
t_5	2021-01	67	93	26	55	0.36	1	0	1	—	—	No
t_6	2021-01	74	92	28	65	0.48	1	0	0	—	—	No
t_7	2021-01	9	95	22	10	0.12	0	1	0	—	—	No
t_8	2021-01	41	91	29	70	—	—	—	—	—	—	—
t_r	2021-02	78	90	31	110	0.72	1	0	1	—	—	Yes
t_u	2022-05	66	93	26	80	0.40	1	0	1	—	—	Yes
t_p	2023-03	71	94	24	40	0.28	0	—	0	1	1	Yes

Table 1: Enriched patient data: structured attributes A_0 – A_4 , CXR- and note-derived attributes B_1 – B_6 , and emergency label Y . Data/query shifts introduce B_5 – B_6 while replacing B_3 .

cisions. At the other extreme, *feature stores* [16, 40, 48, 64, 67] and *materialized views* [3, 8, 36, 53] precompute reusable features or cross-domain transformations, but generic materialization can introduce redundancy and poor workload alignment. This tradeoff is especially problematic for *recurring production analytics*, such as financial risk analysis [23] and clinical decision support [24], where related semantic queries are repeatedly issued over new records, time windows, or cohorts. Recurring workloads therefore require amortizing semantic interpretation: repeated online inference is costly, while materializing all semantic features is wasteful; only workload-relevant, reusable semantic state should be retained.

LiteDBX. To address this tradeoff, we develop LiteDBX (*Lightweight DB eXtender*), a lightweight extension to relational DBMSs for routine semantic analytics over relational and unstructured data. LiteDBX targets the middle ground between online semantic execution and generic pre-materialization: recurring or slowly evolving workloads whose semantic intent is reusable, but whose relational representation is workload-dependent. Its key idea is to turn recurring semantic computation into workload-specific relational state by jointly determining what semantic attributes to materialize and how to express semantic predicates over them in SQL.

Given a relational database $\mathcal{D}$ of schema $\mathcal{R}$, unstructured data $\mathcal{U}$, a semantic-query workload Q^S , and a human-labeling budget β_{lab} , LiteDBX enriches $\mathcal{R}$ with a small set of query-aware, human-interpretable attributes extracted from $\mathcal{U}$, yielding $\mathcal{R}^E$ and its instance $\mathcal{D}^E$, and rewrites semantic queries into standard SQL Q^T on the enriched data. Within β_{lab} , it expands the annotated tuples into a compact supervision basis and amortizes this one-time labeling cost across aligned queries and repeated executions. When data and workloads evolve, LiteDBX incrementally maintains the enriched state and selectively refreshes affected attributes and rewrites.

Thus, LiteDBX compiles recurring semantic interpretation into workload-specific relational state (explicit attributes and reusable SQL) for predictable, low-cost, and auditable execution.

Example 1: Consider the hospital dataset in Table 1, containing structured patient records and chest X-rays (CXR). Clinicians may ask for patients over 65 with low SpO₂ and severe bilateral lung opacity. The first two conditions are expressible in SQL, whereas

the imaging condition must be inferred from CXR; similar vitals can mask markedly different pulmonary severity [13, 42, 65].

An MMDB can evaluate the imaging condition by invoking an LLM at query time, but recurring queries repeatedly pay this inference cost (e.g., \$1.62 and 218.4s for Palimpzest on 200 rows [34]). A feature store can instead pre-extract a broad set of imaging features, many of which may never be used by this workload, while a materialized-view approach precomputes a fixed transformation that must be redesigned when the semantic criteria change [3].

LiteDBX instead materializes only workload-relevant attributes, e.g., lung-opacity extent (B_1) and bilateral involvement (B_2) in Table 1. It then rewrites the imaging condition into predicates over B_1 and B_2 , so the query executes as standard SQL. These attributes are reused across related queries, while workload changes trigger selective refresh of affected attributes and rewrites. $\square$

Positioning. Online MMDBs suit highly ad-hoc queries with little semantic reuse, while generic pre-materialization suits workloads sharing a fixed feature space. LiteDBX instead targets recurring or slowly evolving workloads with reusable semantic intent but workload-dependent representations, jointly deciding what semantic state to materialize and how to query it in SQL.

Challenges. LiteDBX raises two key challenges. First, *what state should be materialized?* Attribute selection and semantic-to-SQL rewriting are coupled under limited supervision: predictive attributes may not admit faithful bounded rewrites. Second, *when can that state be reused?* As data and query intent evolve, LiteDBX must determine whether the materialized attributes and rewrites remain valid and refresh only the affected state.

Contributions & Organization. We address the challenges.

(1) *Problem formulation and LiteDBX system* (Sections 3-4). We formalize *workload-aware schema enrichment for semantic queries*: given $(\mathcal{R}, \mathcal{D}, \mathcal{U}, Q^S)$ and enrichment, labeling and rewrite budgets, construct an enriched configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ that enables semantic predicates to execute as reusable SQL. We show that the problem is NP-hard. Nonetheless we present LiteDBX that supports the lifecycle of such enriched configurations, from offline construction to online SQL execution and incremental maintenance.

(2) *Joint enrichment and rewriting* (Section 5). To construct this state under limited supervision, we develop SEQR (*Schema Enrichment and Query Rewriting*), which jointly selects query-relevant semantic attributes and their bounded SQL rewrites, rather than doing them independently. We establish an end-to-end lower bound on the macro-averaged F_1 of the resulting rewritten workload.

(3) *Incremental maintenance* (Section 6). To maintain the enriched state as data and query intent evolve, LiteDBX combines certified reuse under data evolution with semantic alignment and rewrite-consistency checking under workload evolution. When reuse is invalid, it localizes the affected state and selectively refreshes values, attributes, and rewrites before falling back to full SEQR.

(4) *Experimental evaluation* (Section 7). Using real-life datasets, we evaluate compiled execution and maintenance under evolution. We find that (a) LiteDBX improves F_1 over the strongest label-assisted method by 71.6% (resp. 76.1%) in static (resp. dynamic) settings; (b) with a small labeling budget, it improves F_1 over the strongest

zero-shot method by 16.5% (resp. 21.2%); (c) against the strongest label-assisted (resp. zero-shot) method, it reduces monetary cost by up to 99.9% (resp. 99.9%) and average latency by 98.7% (resp. 98.4%).

We will discuss related work in Section 2 and summarize the novelty of the work in Section 8. We defer detailed proofs to [4].

2 BACKGROUND AND RELATED WORK

This section reviews semantic queries and prior work.

2.1 Semantic Queries

Datasets. We consider both relational and unstructured data.

Relations. Consider a database schema $\mathcal{R} = (R_1, \dots, R_{|\mathcal{R}|})$ in which R_i is a relation schema $R_i(A_1, \dots, A_{|R_i|})$, where each A_j is an attribute. A dataset $\mathcal{D}$ of $\mathcal{R}$ is $(D_1, \dots, D_{|\mathcal{R}|})$, where D_i is a relation of R_i . Assume w.l.o.g. that each tuple $t \in D_i$ represents an entity [14].

Unstructured data. We consider a collection $\mathcal{U}$ of documents (e.g., clinical notes), images (e.g., radiology scans), audio (e.g., dictations), and video (e.g., surveillance clips), represented as text, pixel arrays, waveforms and frame sequences, respectively. Each $u \in \mathcal{U}$ is opaque at the storage level and accessed via feature extraction.

We assume a tuple-object mapping associates each tuple $t \in D_i$ with zero or more objects $u \in \mathcal{U}$ that provide its semantic context (e.g., clinical notes or product reviews), following standard practice in prior multimodal querying systems [27, 28, 34, 38, 50, 51].

Semantic query. Such queries extend SQL with user-specified *natural-language predicates* evaluated over $\mathcal{D}$ and associated unstructured data $\mathcal{U}$ [27, 28, 38, 50, 51]. Such predicates capture high-level semantic intent (e.g., conditions implied by text, images or videos) not expressible by standard attribute comparisons alone.

We represent a *semantic query* as a quadruple $q^S = (\Pi, \mathcal{F}, \Sigma, P^S)$, where: (1) Π is the set of projected attributes in the SELECT clause; (2) $\mathcal{F}$ is the set of joinable relation atoms $R_i \in \mathcal{R}$ in the FROM clause; (3) Σ is the set of user-defined SQL predicates that restrict the analytical scope over $\mathcal{D}$; and (4) P^S is the set of semantic (typically natural-language) predicates referring to both $\mathcal{D}$ and $\mathcal{U}$.

Denote by $Q^S = \{q_i^S\}$ the set of user-specified semantic queries, and by $\Sigma_i(\mathcal{D})$ the set of tuples in $\mathcal{D}$ satisfying Σ in q_i^S . A semantic query q_i^S returns tuples in $\mathcal{D}$ satisfying both Σ_i and P_i^S , e.g., the analytical task in Example 1 can be represented by q_e^S below:

```
SELECT * from Patient AS P JOIN CXR AS X ON P.id = X.id
WHERE P.time BETWEEN '2020-12' AND '2021-01' AND P.A1 ≥ 65
AND Sem(X, "the patient appears critically ill based on
clinical risk indicators and the X-ray")
```

where the two predicates (provided by users or experts [45]) form constraints over the relational attributes of Patient in Table 1, and Sem is a semantic operator applied to both Patient and image data.

2.2 Related Work

We categorize the related work as follows.

Semantic Query Processing. This line of work extends DBMSs with native support for querying multi-modal data.

Online semantic querying. LOTUS [50, 51], Palimpzest [38] and CAESURA [57] execute label-free semantic operators or plans at query time; Symphony [10], FlockMTL [19] and BigQuery [22] invoke models or perform semantic discovery during processing.

ThalamusDB [27, 28] combines zero-shot models with a few human labels for label-assisted AQP with deterministic bounds, while iPDB [32] optimizes in-database ML/LLM predicates. Text-centric systems, e.g., OpineDB [35], NeuralDB [55, 56], WandaDB [25], ZenDB [37], ELEET [58], SUQL [41] and CoddLLM [68], similarly extend SQL with NLP or neural operators over text. SemBench [34] benchmarks these systems along cost-latency-quality tradeoffs.

Proxy and retrieval systems. BARGAIN [66] reduces expensive oracle LLM calls through a proxy-oracle cascade with statistical quality guarantees. AI-query approximation systems [12] use embeddings and lightweight proxy models to approximate AIIF- and AIRANK-style operators. LanceDB [49] supports vector, full-text and hybrid retrieval with structured filtering over multimodal data.

Pre-materialization. This line of work precomputes representations from unstructured data, without reusable SQL rewrites.

Feature stores. Prior systems extract features from unstructured data and materialize them as tables for downstream analytics, as in RALF [64], PA-FEAT [67], FeathrPO [40], Hopsworks [16] and [48].

Materialized views. Prior systems leverage LLMs to automatically extract structured, queryable tables from document collections, as in EVAPORATE [3], TWIX [36], DocETL [53] and Doctopus [8].

This work differs from prior systems in the following. (1) *Execution model*: Zero-shot systems invoke models online [22, 32, 38, 50, 51, 57], while ThalamusDB uses limited labels for predicate calibration [27, 28]; in contrast, LiteDBX learns reusable attributes and SQL rewrites from a few labels, trading supervision for higher accuracy than zero-shot systems. (2) *Execution engine*: Unlike semantic operators [19, 50, 51] or proxy/cascade execution [12, 66], LiteDBX runs standard SQL directly on a relational engine. (3) *Interpretability*: Unlike model-mediated execution [10, 38] or vector retrieval [49], LiteDBX exposes inspectable attributes and predicates. (4) *Cost*: LiteDBX amortizes one-time annotation and enrichment across aligned queries and executions, avoiding repeated online inference. (5) *Population*: Unlike predictive features [40, 64, 67], collection-wide views [3, 36], or drift-model pools [43], LiteDBX materializes only workload-relevant attributes, and retains propagated labels and reusable rewrites. (6) *Guarantees*: Existing systems provide operator-, model-, or oracle-relative guarantees [12, 27, 28, 50, 51, 66]; LiteDBX instead lower-bounds end-to-end query F_1 .

Drift-aware adaptation. MATCHMAKER [43] handles covariate or concept drift by selecting predictive models, whereas LiteDBX maintains coupled relational state: semantic attributes, their values, and the SQL predicates that realize workload semantics.

Schema Enrichment. This line of work enhances relational schemas using external structured data sources to support downstream tasks, including (1) *Data discovery*, which identifies joinable or unionable tables via embeddings or semantic similarity (e.g., PEXESO, AutoTUS, Starmie, Josie, Santos and Cocoa [18, 20, 21, 26, 31, 47, 70]); and (2) *ML model performance*, which improves accuracy or efficiency by enriching schemas with external data (e.g., ARDA, AutoFeature, Leva, RECON and Turl [11, 17, 39, 62, 69]).

This work differs in two respects. (1) *Enrichment source*: Unlike methods that join external structured data [11, 39, 69], LiteDBX extracts semantic attributes from unstructured data. (2) *Workload*:

Unlike enrichment for data discovery, such as join or union detection [18, 21], LiteDBX targets semantic-query workloads.

3 PROBLEM AND COMPLEXITY

This section formulates the schema enrichment problem for answering semantic queries, and proves its intractability.

Setting. Given a relational dataset $\mathcal{D}$ of schema $\mathcal{R}$, associated unstructured data $\mathcal{U}$, and a semantic workload Q^S , LiteDBX constructs an enriched configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ under labeling budget β_{lab} . We focus on the static construction problem; maintenance under data or workload evolution is deferred to Section 6.

We call $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ an *enriched configuration*: it is the workload-specific relational state consisting of the enriched schema and data together with the reusable SQL rewrites.

Answers and supervision. For a semantic query $q_i^S \in Q^S$, let $\Sigma_i(\mathcal{D})$ denote its relational scope, i.e., tuples that satisfy the SQL predicates Σ_i of q_i^S , and $G_i \subseteq \Sigma_i(\mathcal{D})$ denote its *ground-truth answer set*, i.e., tuples that also satisfy the semantic predicate P_i^S w.r.t. $\mathcal{U}$.

For instance, the ground truth G_e of q_e^S in Section 2.1 contains the Patient tuples in Table 1 that satisfy the relational scope Σ (i.e., age over 65 and admission between December 2020 and January 2021) and are identified as critically ill by the semantic predicate P_e^S (“clinical risk indicators and X-ray”).

Define $Y_i(t) \in \{0, 1\}$ for $t \in \Sigma_i(\mathcal{D})$, where $Y_i(t) = 1$ iff $t \in G_i$. Then we define *semantic selectivity* of q_i^S as $\pi_i = \frac{|G_i|}{|\Sigma_i(\mathcal{D})|}$.

Under labeling budget β_{lab} , LiteDBX obtains an *observed set* $O_i \subseteq \Sigma_i(\mathcal{D})$ with $|O_i| \leq \beta_{\text{lab}}$ for user annotation, and algorithmically expands these labels into supervision for enrichment and rewriting.

For each q_i^S , let $E_i \subseteq \mathcal{D}$ be its execution result and $\mathcal{E} = (E_1, \dots, E_{|Q^S|})$. We measure *workload accuracy* by macro-averaged F_1 :

$$\text{acc}(Q^S, \mathcal{E}) = \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} F_1(E_i, G_i),$$

where $F_1(E_i, G_i) = \frac{2 \text{precision}_i \text{recall}_i}{\text{precision}_i + \text{recall}_i}$, with $\text{precision}_i = \frac{|E_i \cap G_i|}{|E_i|}$ and $\text{recall}_i = \frac{|E_i \cap G_i|}{|G_i|}$. For instance, if G_i contains 10 tuples and E_i returns 8, of which 6 are correct, then $\text{precision}_i = 6/8$ and $\text{recall}_i = 6/10$.

Schema enrichment. For a relation schema $R = (\bar{A})$ in $\mathcal{R}$, where $\bar{A} = (A_1, \dots, A_n)$ are base attributes, LiteDBX outputs an *enriched schema* $R^E = (\bar{A}, \bar{B})$, where $\bar{B} = (B_1, \dots, B_l)$ contains workload-relevant semantic attributes extracted from $\mathcal{U}$. Each B_j has an explicit, interpretable semantic definition and typed domain.

For example, attributes B_1 – B_4 in Table 1 are extracted from X-ray images for semantic queries such as q_e^S in Example 1.

Let $\text{arity}(R) = |R|$ and $\text{arity}(\mathcal{R}) = \sum_{R \in \mathcal{R}} \text{arity}(R)$. Schema enrichment is subject to budget β_{SE} : $\text{arity}(\mathcal{R}^E) - \text{arity}(\mathcal{R}) \leq \beta_{\text{SE}}$.

Query rewrite. Given a semantic query $q_i^S = (\Pi, \mathcal{F}, \Sigma_i, P_i^S)$ and enriched data $\mathcal{D}^E$ of schema $\mathcal{R}^E$, LiteDBX outputs $q_i^T = (\Pi, \mathcal{F}, \Sigma_i, P_i^T)$ by preserving $\Pi, \mathcal{F}$ and Σ_i and rewriting only the semantic predicate P_i^S over base and enriched attributes. We restrict P_i^T to a β_{rew} -UCQ (bounded union of conjunctive queries) [1, 61]:

$$P_i^T = \bigvee_{j=1}^{m_i} \bar{p}_j, \quad m_i \leq \beta_{\text{rew}},$$

where each $\bar{p}_j$ is a conjunction of comparisons $A \oplus c$, with $A \in \mathcal{R}^E$, constant c , and $\oplus \in \{=, \neq, <, >, \geq\}$. We use UCQ rather than

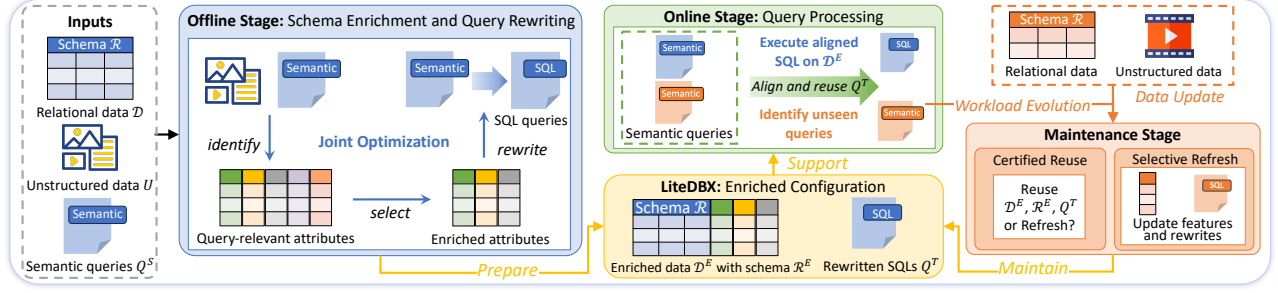


Figure 1: LiteDBX workload lifecycle

general relational algebra because the rewrite replaces a Boolean selection predicate: conjunction captures joint attribute conditions, while union captures alternative semantic cases; projection, joins, aggregation, and other relational operators are already represented by the outer query. The budget β_{rew} bounds the number of alternative decision regions, controlling rewrite complexity and overfitting under limited supervision while retaining multiple semantic cases. Denote by $Q^T = \{q_i^T\}$ the rewritten workload.

For instance, q_e^S in Example 1 can be rewritten as:

```
SELECT * FROM ExtendedPatient WHERE age ≥ 65 AND time
BETWEEN '2020-12' AND '2021-01' AND Sp02 ≤ 90 AND (1)
opacity ≥ 0.70 AND bilateral = 1 AND effusion = 1;
```

Here Π , $\mathcal{F}$ and Σ are inherited from q_e^S , while P_e^S (i.e., Sem) is compiled into predicates over base attributes A_0 – A_2 and enriched attributes B_1 , B_2 and B_4 in Table 1 (see Example 5 for more).

Budgets. LiteDBX uses three budgets controlling distinct resources: β_{lab} bounds the user-labeled tuples $|O_i|$ per query, β_{SE} bounds the number of materialized semantic attributes, and β_{rew} bounds the number of conjunctive clauses used to represent each rewritten semantic predicate P_i^T . The first limits human supervision, the second controls the size of the workload-specific relational state, and the third regularizes semantic approximation under limited supervision rather than relational execution cost.

Problem. We are now ready to state *the schema enrichment problem*. The optimization target is the executable enriched configuration, not the intermediate predictive model: learning is used only to construct and evaluate candidate relational representations.

- *Input:* $Q^S, \mathcal{R}, \mathcal{D}, \mathcal{U}$, and budgets β_{SE}, β_{lab} , and β_{rew} .
- *Output:* Enriched schema $\mathcal{R}^E$ of $\mathcal{R}$, extended dataset $\mathcal{D}^E$ of $\mathcal{D}$ w.r.t. $\mathcal{R}^E$, rewritten queries Q^T of Q^S , and query result $\mathcal{E}$ of Q^T .
- *Objective:* Find $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ that maximizes $\text{acc}(Q^S, \mathcal{E})$, s.t. the schema enrichment budget $\text{arity}(\mathcal{R}^E) - \text{arity}(\mathcal{R}) \leq \beta_{SE}$, the labeling budget $|O_i| \leq \beta_{lab}$, and the rewrite bound β_{rew} on P_i^T .

Scope. LiteDBX targets semantic predicates that admit compact, explicit relational representations, and identifies a compact set of workload-relevant attributes from $\mathcal{U}$. Hence, LiteDBX does not assume that every semantic predicate should be relationalized; it targets predicates representable under the available attribute and rewrite budgets. Concepts requiring inherently high-dimensional latent representations fall outside the current scope; hybrid vector-relational execution is complementary. Following standard learning assumptions [59, 60], we assume a finite candidate attribute space [5]. A query may admit multiple SQL rewrites [15], with P_i^T restricted to at most β_{rew} conjunctive clauses.

Table 2 summarizes the key notations used in this paper.

Notation	Description
$\mathcal{R}, \mathcal{R}^E, \mathcal{D}, \mathcal{D}^E$	Enriched schema $\mathcal{R}^E$ of $\mathcal{R}$, and dataset $\mathcal{D}^E$ w.r.t. $\mathcal{R}^E$ of $\mathcal{D}$ w.r.t. $\mathcal{R}$.
$\mathcal{U}$	Unstructured data associated with tuples in $\mathcal{D}$.
$q_i^S \in Q^S; \pi_i$	Semantic queries over $\mathcal{D}$ and $\mathcal{U}$; semantic selectivity of q_i^S .
$q_i^T \in Q^T$	Rewritten union-of-conjunctive-query form of each q_i^S .
G_i	Ground truth answer set of query q_i^S .
$E_i \in \mathcal{E}$	The set of tuples in $\mathcal{D}$ returned when executing $q_i^T \in Q^T$.
$O_i, C_i \mid O_i$	user labels, pseudo labels
$\Sigma_i(\mathcal{D}^E) \setminus C_i$	propagated labels
$\beta_{lab}, \beta_{SE}, \beta_{rew}$	budgets: user-labeling, schema-enrichment, rewrite-complexity

Table 2: Notations

Intractability. The decision version of the problem is to decide, given $Q^S, \mathcal{R}, \mathcal{D}, \mathcal{U}, \beta_{lab}, \beta_{SE}, \beta_{rew}$ and a bound ω as input, whether there exist extensions $\mathcal{R}^E$ of $\mathcal{R}$, $\mathcal{D}^E$ of $\mathcal{D}$, and Q^T of Q^S such that $\text{acc}(Q^S, \mathcal{E}) \geq \omega$ and $\text{arity}(\mathcal{R}^E) - \text{arity}(\mathcal{R}) \leq \beta_{SE}$.

Theorem 1: *The schema enrichment problem is NP-hard.* $\square$

Proof sketch: We prove NP-hardness by reduction from the NP-complete set cover problem [29] in a relaxed oracle setting where the ground-truth answer sets G_i are given. Given a set cover instance, we construct (1) $\mathcal{D}$ w.r.t. $\mathcal{R}$ encoding the elements; (2) $\mathcal{U}$ with Boolean bag-of-words features [44], each corresponding to a subset; (3) a query set $Q^S = \{q_1^S\}$; and (4) set $\beta_{lab} = 1$, β_{SE} to the set cover budget, $\beta_{rew} = 1$, and $\omega = 1$. We show that schema enrichment is feasible if and only if the set cover instance has a valid cover. $\square$

4 LITEDBX: A DBMS EXTENDER

This section presents the semantic workload lifecycle in LiteDBX.

LiteDBX workload lifecycle. As illustrated in Figure 1, LiteDBX realizes the problem formulation in Section 3 through three stages: offline preparation, online execution, and incremental maintenance. Recall that $\mathcal{R}$ is the relational schema, $\mathcal{D}$ is its dataset, $\mathcal{U}$ is the associated unstructured data, and Q^S is the semantic-query workload. LiteDBX operates above an unmodified relational DBMS and maintains an enriched configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$, with semantic processing performed offline or during maintenance, and routine queries executed as native SQL over enriched data.

(1) *Offline preparation* (Section 5). Given $(\mathcal{R}, \mathcal{D}, \mathcal{U}, Q^S)$ and budgets β_{lab}, β_{SE} and β_{rew} , LiteDBX invokes SEQR (*Schema Enrichment and Query Rewriting*) to construct the enriched configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$. It selects workload-relevant semantic attributes from $\mathcal{U}$, materializes their values in $\mathcal{D}^E$, and rewrites semantic predicates in workload Q^S into reusable SQL queries in Q^T .

(2) *Online execution.* Given an incoming semantic query, LiteDBX checks whether its semantic predicate can reuse the current enriched configuration, either directly or through semantic alignment. Reusable queries execute as native SQL over the currently published

$\mathcal{D}^E$, with no online semantic inference on the routine path. Queries unsupported by the current state are passed to maintenance.

(3) *Incremental maintenance* (Section 6). When $\mathcal{D}$, $\mathcal{U}$ or Q^S evolve, LiteDBX determines whether the current configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ remains reusable. It reuses valid state when possible, selectively refreshes affected state otherwise, and reruns SEQR only when bounded refresh is insufficient. Refresh is performed off the query path while queries use the last published configuration and its corresponding data version; a refreshed configuration is published atomically only after all dependent state is materialized.

Example 2: Continuing Example 1, we illustrate the three stages of LiteDBX in clinical analytics. See Examples 3–5 (offline and online stages) and Examples 6–7 (maintenance stage) for more details.

(1) *Offline*. Given semantic queries over patient records and CXR images, LiteDBX uses an LLM to derive query-relevant CXR attributes, selects a compact and high-utility subset under the schema budget (e.g., $\{B_1, B_2, B_3, B_4\}$ in Table 1), and compiles the semantic predicates into executable SQL (e.g., the one in Section 3).

(2) *Online*. Routine queries, such as identifying high-risk elderly patients, reuse the materialized attributes and rewritten SQL, with only scope changes such as updated time windows.

(3) *Maintenance*. When data or query intent changes, LiteDBX reuses valid state and selectively refreshes the rest under the same budget. For example, a shift to pneumonia screening may replace B_3 with B_5 and B_6 and update the affected SQL rewrites. $\square$

5 SEMANTIC SCHEMA ENRICHMENT

This section presents the *schema enrichment and query rewriting* (SEQR) process (offline stage) in LiteDBX. It identifies the key challenges (Section 5.1), presents the SEQR framework (Section 5.2) and establishes its theoretical guarantees (Section 5.3).

5.1 Design Challenge

Given relational data $\mathcal{D}$ of schema $\mathcal{R}$, unstructured data $\mathcal{U}$, and a historical semantic workload Q^S , SEQR (1) extracts query-relevant attributes from $\mathcal{U}$, (2) constructs an enriched schema $\mathcal{R}^E$ and its instance $\mathcal{D}^E$, and (3) rewrites Q^S into executable SQL Q^T for routine execution. It outputs the enriched configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$.

Challenges. A natural approach is to extract semantic attributes with an LLM, obtain a small labeled sample, and derive SQL rewrites. This raises three challenges: (C1) *workload-aware enrichment*: semantic extraction should focus on workload-relevant tuples and concepts, rather than collection-wide; (C2) *limited supervision*: only a small user-labeled sample is available to evaluate candidate semantic representations; and (C3) *joint enrichment and rewriting*: attributes predictive of a semantic predicate may still yield inaccurate or overly complex SQL rewrites under the rewrite budget.

Why joint optimization? A highly discriminative attribute may induce fragmented decision regions requiring many SQL disjunctions, whereas favoring simple rewrites may lose predictive distinctions. Thus, enrichment and rewriting must be optimized jointly.

In Example 1, an LLM may extract a large-domain CXR severity attribute (e.g., an opacity-extent score in $\{0, \dots, 8\}$). Its interactions with SpO₂ and CRP may induce many disjoint positive regions and

yield many narrow SQL disjunctions, making a faithful bounded rewrite difficult. This motivates the joint optimization in SEQR.

5.2 The SEQR Framework

SEQR constructs an enriched configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ in three phases: (1) *query-aware preparation*, which restricts semantic extraction to workload-relevant tuples and derives candidate attributes from $\mathcal{U}$ (addressing challenge C1); (2) *coreset construction*, which expands the small user-labeled set into a pseudo-labeled supervision basis (C2); and (3) *joint enrichment and rewriting*, which selects attributes together with their bounded SQL rewrites (C3).

(1) **Query-aware preparation.** For each $q_i^S = (\Pi, \mathcal{F}, \Sigma_i, P_i^S) \in Q^S$, SEQR first restricts processing to its workload-relevant scope $\Sigma_i(\mathcal{D})$. It samples a *user-annotated set* $O_i \subseteq \Sigma_i(\mathcal{D})$ with $|O_i| \leq \beta_{\text{lab}}$, and uses (q_i^S, O_i) to derive a query-aware candidate attribute pool $\mathcal{R}_{\mathcal{U}}$ from $\mathcal{U}$ by using an LLM. Users provide ground-truth labels $Y_i(t)$ only for $t \in O_i$; no additional user labels are requested in subsequent phases. These labels refine the candidate attributes and estimate semantic selectivity $\hat{\pi}_i = \frac{1}{|O_i|} \sum_{t \in O_i} Y_i(t)$. Implementation details, including LLM prompts, are deferred to [4].

Example 3: Continuing Example 2, consider the static workload $Q^S = \{q_e^S, q_p^S\}$ during Dec. 2020–Feb. 2021, for elderly (age ≥ 65) and pediatric (age ≤ 12) patients, with $\beta_{SE} = 5$ and $\beta_{\text{lab}} = 10$.

For q_e^S , SEQR first applies Σ to obtain the workload-relevant scope $\Sigma_e(\mathcal{D})$ (e.g., t_1-t_6); tuples outside the scope, such as t_8 , are ignored. Under $\beta_{\text{lab}} = 10$, SEQR samples at most 10 tuples from $\Sigma_e(\mathcal{D})$ for annotation, forming $O_e \subseteq \Sigma_e(\mathcal{D})$ (e.g., t_1-t_3 in Table 1). Conditioned on (q_e^S, O_e) , it derives a query-aware candidate pool $\mathcal{R}_{\mathcal{U}} = \{B_1, B_2, B_3, B_4, B_7, B_8, B_9\}$ from the associated CXRs. Here B_1-B_4 appear in Table 1, while B_7 is the opacity-extent attribute from Section 5.1. Clinician annotations on O_e refine the candidate attributes and yield the selectivity estimate $\hat{\pi}_e$ (e.g., 0.3). $\square$

Thus, semantic extraction is restricted to query-relevant tuples while human supervision remains bounded by β_{lab} .

(2) **Coreset for limited supervision.** Since O_i is too small to reliably evaluate candidate enrichments and rewrites, SEQR expands it into a *pseudo-labeled coreset* C_i using informative tuples from $\Sigma_i(\mathcal{D}) \setminus O_i$. Only O_i is user-labeled; $C_i \setminus O_i$ is pseudo-labeled. SEQR combines prediction confidence for label reliability with structural confidence based on proximity to labeled positives and negatives.

For each semantic query q_i^S , SEQR (a) populates candidate attributes in $\mathcal{R}_{\mathcal{U}}$ for tuples in $\Sigma_i(\mathcal{D}) \setminus O_i$ using local LLMs, with O_i as in-context examples; (b) trains a tree-based model on O_i to obtain predicted labels with confidence $f_p(t) \in [0, 1]$ and feature-importance scores for $\mathcal{R}_{\mathcal{U}}$; and (c) computes structural confidence

$$f_s(t) = \frac{d^-(t)}{d^+(t) + d^-(t)},$$

where $d^+(t)$ (resp. $d^-(t)$) is the feature-importance-weighted average distances from t to labeled positive (resp. negative) tuples in O_i .

SEQR combines the two signals using the estimated selectivity:

$$s(t) = \hat{\pi}_i f_p(t) + (1 - \hat{\pi}_i) f_s(t).$$

It ranks tuples in $\Sigma_i(\mathcal{D}) \setminus O_i$ by $s(t)$ and augments O_i with high-score pseudo positives and low-score pseudo negatives, so that the positive fraction in C_i matches $\hat{\pi}_i$. The added tuples receive pseudo labels rather than user annotations. Details are provided in [4].

The resulting C_i provides a compact, query-aware supervision basis for evaluating both label propagation and rewrite fidelity.

Example 4: Continuing Example 3, SEQR expands the user-labeled set O_e into a coreset C_e for q_e^S . (a) For each $t \in \Sigma_e(\mathcal{D}) \setminus O_e$, a local model (e.g., Llama3-8B) populates candidate CXR attributes using O_e as in-context examples. (b) A tree-based classifier (e.g., XGBoost [9]) trained on O_e yields feature importance and prediction confidence $f_p(t)$. (c) SEQR computes structural confidence $f_s(t)$ from feature-importance-weighted distances to labeled positives and negatives (e.g., t_1 and t_3). (d) It combines the signals as $s(t) = 0.3f_p(t) + 0.7f_s(t)$ and selects high- and low-score tuples according to the estimated selectivity, e.g., adding t_4 as a pseudo positive and t_5, t_6 as pseudo negatives, with no user annotation. $\square$

(3) Joint schema enrichment and rewriting. Given the candidate pool $\mathcal{R}_U$ and coresets $\{C_i\}$, SEQR aggregates query-specific importance scores into a global attribute ranking. For each top- k set $\mathcal{R}_k$, $k \in [1, \beta_{SE}]$, it constructs the corresponding enriched schema and β_{rew} -bounded rewrites. To avoid combinatorial subset enumeration, SEQR searches these nested top- k prefixes as a practical approximation to the NP-hard subset-selection problem. It then selects the configuration that best balances propagation reliability and rewrite fidelity under the objective below. Details are deferred to [4].

Schema enrichment and rewriting. For each candidate $\mathcal{R}_k$, SEQR (a) extends the base schema $\mathcal{R}$ to $\mathcal{R} \cup \mathcal{R}_k$ and populates attributes in $\mathcal{R}_k$ via local LLMs for tuples in each scope $\Sigma_i(\mathcal{D})$; (b) trains a tree-based model on each C_i and propagates labels to $\Sigma_i(\mathcal{D}^E) \setminus C_i$; and (c) constructs a β_{rew} -bounded UCQ from high-support positive decision paths, replacing P_i^S as defined in Section 3.

Joint evaluation. For each candidate $\mathcal{R}_k$ and query q_i^S , SEQR evaluates (a) *propagation error*, via the subjective score $\widehat{\mathcal{L}}_{\text{subj}}^{(i)}$, and (b) *rewrite error*, via the objective score $\widehat{\mathcal{L}}_{\text{obj}}^{(i)}$.

The subjective score is based on the leave-one-out (LOO) error

$$\widehat{\mathcal{L}}_{\text{LOO}}^{(i)} = \frac{1}{|O_i|} \sum_{t \in O_i} \ell_{\text{prop}}(Y_i(t), \widehat{Y}_i^{\text{LOO}}(t)),$$

augmented with a stability-aware finite-sample penalty under class imbalance (see [4]). Here $Y_i(t)$ is the user-provided ground-truth label, $\widehat{Y}_i^{\text{LOO}}(t)$ is the propagated label obtained with t excluded from O_i , and ℓ_{prop} is a class-imbalance-aware cost-sensitive 0-1 error.

The objective score measures the mismatch between the rewritten query result under $\mathcal{R} \cup \mathcal{R}_k$ and the propagated labels over $\Sigma_i(\mathcal{D}^E)$, augmented with a penalty for estimation under the β_{rew} -bounded rewrite, discouraging overfitting to propagated labels.

SEQR combines the two as $\mathcal{L}_{\text{static}}^{(i)} = \widehat{\mathcal{L}}_{\text{subj}}^{(i)} + \widehat{\mathcal{L}}_{\text{obj}}^{(i)}$ and selects $\mathcal{R}_k$ that minimizes $\frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} \mathcal{L}_{\text{static}}^{(i)}$, with its induced rewrites.

Example 5: Continuing with Example 4, let $\beta_{rew} = 1$. Using C_e , SEQR evaluates top- k attribute sets together with their induced rewrites. Although B_7 is predictive, its fragmented decision regions cannot be faithfully captured under $\beta_{rew} = 1$, so including it does not improve the joint objective. SEQR ultimately selects $\mathcal{R}^* = \{B_1, B_2, B_3, B_4\}$; adding B_8 – B_9 yields no further improvement. The resulting model produces rewritten SQL (1) in Section 3.

Online query execution. Once $\mathcal{R}^E = \mathcal{R} \cup \mathcal{R}^*$ and the rewrite are

Input: $\mathcal{R}, \mathcal{D}, \mathcal{U}, Q^S$, and budgets $\beta_{SE}, \beta_{lab}, \beta_{rew}$.

Output: The enriched schema $\mathcal{R}^E$, dataset $\mathcal{D}^E$, and rewritten workload Q^T .

1. $\{\Sigma_i(\mathcal{D})\}_{i=1}^{|Q^S|} := \text{ScopeFilter}(Q^S, \mathcal{D})$;
2. $(\mathcal{R}_U, \{O_i, \widehat{\pi}_i\}_{i=1}^{|Q^S|}) := \text{QueryAwarePrepare}(Q^S, \{\Sigma_i(\mathcal{D})\}, \mathcal{U}, \beta_{lab})$;
3. $\{C_i\}_{i=1}^{|Q^S|} := \text{BuildCoresets}(\{\Sigma_i(\mathcal{D}), O_i, \widehat{\pi}_i\}, \mathcal{R}_U)$;
4. $\text{FS} := \text{RankAttributes}(\{C_i\}, \mathcal{R}_U, \beta_{SE})$;
5. **for** $k \in [1, \beta_{SE}]$ with $\mathcal{R}_k := \text{FS}[k]$:
6. $(\mathcal{R}^E, \mathcal{D}^E) := \text{EnrichSchema}(\mathcal{R}, \mathcal{R}_k, \mathcal{D}, \{\Sigma_i(\mathcal{D})\})$;
7. **for** each $q_i^S \in Q^S$:
8. $(\widehat{Y}_i, \widehat{\mathcal{L}}_{\text{subj}}^{(i)}) := \text{LabelPropagation}(C_i, \Sigma_i(\mathcal{D}^E), \mathcal{R}^E)$;
9. $(q_i^T, \widehat{\mathcal{L}}_{\text{obj}}^{(i)}) := \text{QueryRewrite}(\Sigma_i(\mathcal{D}^E), \mathcal{R}^E, \widehat{Y}_i, \beta_{rew})$;
10. $\mathcal{L}_{\text{static}} := \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} (\widehat{\mathcal{L}}_{\text{subj}}^{(i)} + \widehat{\mathcal{L}}_{\text{obj}}^{(i)})$;
11. $(\mathcal{R}_{\text{opt}}^E, \mathcal{D}_{\text{opt}}^E, Q_{\text{opt}}^T, \mathcal{L}_{\text{opt}}) := \text{KeepBest}(\mathcal{R}^E, \mathcal{D}^E, \{q_i^T\}_{i=1}^{|Q^S|}, \mathcal{L}_{\text{static}})$;
12. **return** $(\mathcal{R}_{\text{opt}}^E, \mathcal{D}_{\text{opt}}^E, Q_{\text{opt}}^T)$;

Figure 2: Algorithm SEQR

materialized, aligned queries reuse the materialized schema and rewrite with updated scope predicates, e.g., for patients admitted in Jan.–Feb. 2021, returning t_r in Table 1 without rerunning SEQR. $\square$

Provenance. For auditability, LiteDBX records lightweight lineage for each materialized attribute, including its source object, semantic definition, extractor/model version, and motivating semantic predicate. A query result can thus be traced from its rewritten SQL predicate to the enriched attribute and the source object in $\mathcal{U}$.

SEQR addresses challenge C3 by evaluating candidate attributes through both propagation reliability and rewrite fidelity, rather than predictive utility alone. We establish an accuracy guarantee for the resulting SQL rewrites in Theorem 3 to be presented shortly.

Algorithm. Algorithm 2 summarizes SEQR. It takes as input $\mathcal{R}, \mathcal{D}, \mathcal{U}, Q^S$, and budgets β_{SE}, β_{lab} and β_{rew} , and returns $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$.

The model assigns propagated labels to $\Sigma_i(\mathcal{D}^E) \setminus C_i$, while labels in C_i remain fixed. For each query q_i^S , SEQR first derives the scoped tuple set $\Sigma_i(\mathcal{D})$, obtains user-labeled O_i , estimates $\widehat{\pi}_i$, and constructs the query-aware candidate attribute pool $\mathcal{R}_U$ from $\mathcal{U}$ (lines 1–2). It then builds C_i and aggregates query-specific feature-importance scores into a global ranking FS over $\mathcal{R}_U$ (lines 3–4).

SEQR next jointly evaluates candidate attribute sets $\mathcal{R}_k$, the top- k ranked attributes (lines 5–11). For each $\mathcal{R}_k$, it (a) constructs the corresponding enriched schema $\mathcal{R}^E$ and data $\mathcal{D}^E$ (line 6); (b) propagates labels from C_i to scoped tuples and computes the subjective score for each q_i^S (line 8); (c) rewrites each semantic predicate into a β_{rew} -bounded UCQ and computes the objective score (line 9); and (d) aggregates the per-query errors and retains the best configuration (lines 10–11). Finally, it returns $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$.

5.3 End-to-End Query Guarantee

We first establish an LOO identity for propagation and then an end-to-end accuracy guarantee for the selected rewrites.

Lemma 2: Let $n_i = |O_i| \geq 2$, where O_i is drawn i.i.d. from $\Sigma_i(\mathcal{D})$, and let $\mathcal{L}_{\text{prop}, n_i-1}^{(i)}$ denote the expected propagation error of a deterministic propagation algorithm trained on $n_i - 1$ i.i.d. labeled tuples and evaluated on an independent tuple under a fixed bounded loss. Then,

$$\mathbb{E}[\widehat{\mathcal{L}}_{\text{LOO}}^{(i)}] = \mathcal{L}_{\text{prop}, n_i-1}^{(i)},$$

where the expectation $\mathbb{E}$ is over O_i and the held-out tuple. $\square$

Proof sketch: The claim follows from the exchangeability of *i.i.d.* samples: each LOO evaluation is distributionally equivalent to testing the same algorithm, trained on $n_i - 1$ independent draws, on an independent tuple from $\Sigma_i(\mathcal{D})$ [7, 54] (see [4] for details). $\square$

Given a confidence parameter $\delta \in (0, 1)$, LiteDBX certifies the final rewrite q_i^T produced by SEQR by splitting the labeled set O_i into an initial set T_i and a certification set V_i . It (1) runs SEQR on T_i to obtain an auxiliary rewrite $\tilde{q}_i^T$; (2) samples V_i uniformly without replacement from $\Sigma_i(\mathcal{D}) \setminus T_i$; (3) combines an exact hypergeometric upper bound derived from V_i with the observed errors on T_i to obtain a full-scope 0–1 error bound U_i with confidence $1 - \delta/|Q^S|$; and (4) executes $\tilde{q}_i^T$ and q_i^T over $\Sigma_i(\mathcal{D})$ to compute their exact disagreement rate $d_i = \Pr_{t \in \Sigma_i(\mathcal{D})}[\tilde{q}_i^T(t) \neq q_i^T(t)]$. Together, U_i and d_i upper-bound the full-scope 0–1 error of q_i^T .

Theorem 3: *Given a semantic query set Q^S , enriched data $\mathcal{D}^E$ under schema $\mathcal{R}^E$, the rewritten SQL workload Q^T produced by SEQR, and a confidence parameter $\delta \in (0, 1)$, the certification procedure provides, for each q_i^S , an auxiliary rewrite $\tilde{q}_i^T$, a full-scope 0–1 error bound U_i for $\tilde{q}_i^T$ with confidence $1 - \delta/|Q^S|$, and the exact fraction d_i of tuples in $\Sigma_i(\mathcal{D})$ on which $\tilde{q}_i^T$ and the final rewrite q_i^T differ. Define $B_i = \min\{1, U_i + d_i\}$. The results $\mathcal{E}$ of executing Q^T over $\mathcal{D}^E$ satisfy, with probability at least $1 - \delta$,*

$$\text{acc}(Q^S, \mathcal{E}) \geq \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} \Phi(\pi_i, B_i),$$

$$\Phi(\pi_i, B_i) = \begin{cases} \frac{2(\pi_i - B_i)}{2\pi_i - B_i}, & 0 \leq B_i < \pi_i, \\ 0, & B_i \geq \pi_i, \end{cases}$$

where $\pi_i = \frac{|G_i|}{|\Sigma_i(\mathcal{D})|}$ is the true selectivity of q_i^S , and $\Phi(\pi_i, B_i)$ lower-bounds the per-query F_1 of q_i^T . $\square$

Proof sketch: We prove by (1) applying an exact hypergeometric bound after the auxiliary rewrite is fixed to bound its 0–1 error by U_i ; (2) using the pointwise triangle inequality to add the exact disagreement d_i and bound the final rewrite error by B_i ; (3) converting this error bound to an F_1 bound using query selectivity; and (4) averaging over Q^S and applying a union bound. $\square$

Intuitively, Theorem 3 combines the certified auxiliary-error bound U_i with the exact rewrite disagreement d_i to bound the error of q_i^T . When $B_i < \pi_i$, smaller U_i and d_i tighten the per-query F_1 bound, which applies to answers returned by reusable SQL. Moreover, certification only uses labels in O_i , preserving the label budget.

Complexity. Query-aware preparation takes $O(|\mathcal{D}||Q^S| + \beta_{\text{lab}}|Q^S| (c_{\text{LLM}} + c_{\text{human}}))$ time. The first term covers scope filtering; the second covers candidate generation, annotation, and selectivity estimation for at most β_{lab} tuples per query, where c_{LLM} and c_{human} are the per-tuple inference and annotation costs, respectively.

Coreset construction takes $O(|Q^S|(c_{\text{proxy}} + c_{\text{train}} + \sigma\beta_{\text{lab}} + \sigma \log \sigma))$ time, where σ is the average query-scope size, c_{proxy} is the cost of populating candidate features over one query scope, and c_{train} is the tree-training cost. The term $\sigma\beta_{\text{lab}}$ (resp. $\sigma \log \sigma$) is for labeled-unlabeled distance computation (resp. confidence-score ranking).

Joint enrichment and rewriting takes $O(|\text{FS}||Q^S|(c_{\text{train}}\beta_{\text{lab}} + c_{\text{rank}}))$ time, where $|\text{FS}|$ is the number of candidate attribute sets and c_{rank} is the cost of ranking decision paths by support.

6 INCREMENTAL MAINTENANCE

This section presents the incremental maintenance of LiteDBX.

The enriched configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ produced by SEQR is workload-specific relational state intended for reuse. As $\mathcal{D}$, $\mathcal{U}$, or Q^S evolve, the key challenge is to determine whether this state remains reliable and, if not, which artifacts require refresh. LiteDBX follows a unified *reuse-or-refresh* strategy: it certifies reuse under data evolution (Section 6.1), checks reuse under workload evolution (Section 6.2), and selectively refreshes only affected values, attributes, or rewrites when reuse fails (Section 6.3).

Unlike conventional model maintenance, which primarily updates predictive models, LiteDBX maintains coupled relational state $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$. Depending on the change, it may refresh only materialized values, only rewrites, both attributes and rewrites, or the entire configuration. LiteDBX certifies reuse when the current state remains reliable and otherwise refreshes the smallest affected state, rerunning SEQR only when bounded refresh is insufficient.

6.1 Certified Reuse under Data Evolution

When $\mathcal{D}$ or $\mathcal{U}$ changes (e.g., by insertion and deletion) while the semantic workload remains fixed, LiteDBX tests whether the existing $(\mathcal{R}^E, Q^T)$ can be safely reused on the updated data. Rather than rerunning SEQR, it incrementally maintains the affected state and tests whether the current rewrites remain reusable.

Specifically, LiteDBX (a) updates the enriched dataset to $\mathcal{D}_{\text{new}}^E$ by populating required attributes for newly relevant tuples; (b) updates the coresets with informative new tuples and re-propagates labels without additional user annotation, retaining the supervision state for selective refresh; and (c) computes an *incremental update term* $\Delta^{(i)}$ from newly introduced tuples and changes in the outputs of the current rewrite on previously covered tuples. Given the outputs $q_i^T(t; \mathcal{D}^E)$ and $q_i^T(t; \mathcal{D}_{\text{new}}^E)$ of the same rewrite on the two data versions, let $\Omega_i = \Sigma_i(\mathcal{D}^E)$ and $\Omega_i^{\text{new}} = \Sigma_i(\mathcal{D}_{\text{new}}^E)$. The update term is

$$\Delta^{(i)} = \frac{|\Omega_i^{\text{new}} \setminus \Omega_i| + \sum_{t \in \Omega_i \cap \Omega_i^{\text{new}}} \mathbb{1}[q_i^T(t; \mathcal{D}^E) \neq q_i^T(t; \mathcal{D}_{\text{new}}^E)]}{|\Omega_i^{\text{new}}|}.$$

The numerator assigns worst-case error to each newly introduced tuple, and counts shared tuples whose rewrite output changes; the denominator normalizes both contributions by the updated scope. LiteDBX combines the update term $\Delta^{(i)}$ with the static certificate B_i ; it reuses the configuration when the updated bound remains within tolerance and triggers selective refresh otherwise.

Example 6: Continuing with Example 2, we illustrate data evolution using the enriched configuration from Example 5.

Suppose that by May 2022 the hospital receives more elderly patients aged 65–68 (e.g., t_a in Table 1), shifting the patient distribution underlying the high-risk query in Example 1. LiteDBX populates the required enriched attributes (e.g., B_1 , B_2 , and B_4) for t_a , updates the coreset, and re-propagates labels over the expanded query scope. It then computes update term Δ from the newly covered patients and changes in the current rewrite outputs on previously covered patients. If the updated error bound remains within tolerance, LiteDBX reuses the current schema and rewrite; otherwise, it selectively refreshes the affected state (see Example 7 for refresh). $\square$

Certified reuse guarantee. The following result bounds the post-update error when the current configuration is reused.

Theorem 4: Given the current configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ and updated data $\mathcal{D}_{\text{new}}^E$, denote the new certificate for each $q_i^S \in Q^S$ by

$$B_i^{\text{new}} = \min \left\{ 1, \frac{|\Sigma_i(\mathcal{D}^E)|}{|\Sigma_i(\mathcal{D}_{\text{new}}^E)|} B_i + \Delta^{(i)} \right\},$$

where B_i is the pre-update full-scope 0–1 error certificate for q_i^T from Theorem 3. For any $\delta \in (0, 1)$, the results $\mathcal{E}_{\text{new}}$ of executing Q^T on $\mathcal{D}_{\text{new}}^E$ satisfy, with probability at least $1 - \delta$,

$$\text{acc}(Q^S, \mathcal{E}_{\text{new}}) \geq \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} \Phi(\pi_i^{\text{new}}, B_i^{\text{new}}).$$

Here Φ is the F_1 lower-bound function from Theorem 3, and π_i^{new} is the true selectivity of q_i^S on $\Sigma_i(\mathcal{D}_{\text{new}}^E)$. $\square$

Proof sketch: We prove by (1) conditioning on the pre-update certificate event; (2) bounding post-update error by B_i^{new} with unit error on new tuples and exact disagreement on shared tuples; and (3) converting to F_1 and averaging as in Theorem 3 (details in [4]). $\square$

Theorem 4 transfers a certified pre-update error through data evolution. Newly introduced tuples contribute their scope fraction, while shared tuples contribute only when the current rewrite changes their outputs. Thus, the bound B_i^{new} directly certifies the updated rewrite and supports the reuse decision.

6.2 Reuse under Workload Evolution

For a new or revised semantic query, LiteDBX distinguishes three cases: (1) *direct reuse*, when its semantic predicate matches an existing predicate and rewrite; (2) *aligned reuse*, when its predicate units are consistently supported by existing attributes and rewrite fragments; and (3) *refresh*, when one or more predicate units are unsupported or inconsistent with the current configuration.

Coverage-and-consistency test. If direct reuse does not apply, let $P_i^S = \{u_1, \dots, u_m\}$ denote the predicate units of the incoming query q_i^S . A unit u_j is *covered* if the current enriched schema contains a supporting attribute and rewrite fragment, and *consistent* if that fragment implements the condition required by u_j . Formally,

$$q_i^S \text{ is reusable} \iff \forall u_j \in P_i^S, \text{covered}(u_j) \wedge \text{consistent}(u_j).$$

LiteDBX checks this condition in two steps. (1) *Semantic alignment*: a local LLM decomposes the semantic predicate into units and maps each unit to supporting attributes and rewrite fragments; unmatched units are marked uncovered. (2) *Rewrite consistency checking*: for each aligned unit, LiteDBX verifies that the associated SQL condition implements the required semantics; missing, changed, or conflicting conditions are marked inconsistent. If all units pass the test, LiteDBX performs aligned reuse; otherwise, the failed units form the refresh set for Section 6.3.

Semantic drift. A change in the meaning of an existing semantic predicate is treated as workload evolution. Updated criteria, user feedback, or validation evidence yield a revised predicate specification, which is subjected to the same coverage-and-consistency test. Hence, only units whose current relational realization no longer matches the revised semantics require refresh.

6.3 Selective Refresh and Publication

When either reuse test fails, LiteDBX first localizes the affected state. For data evolution, the affected set comprises queries with failed certificates and their changed scopes; for workload evolu-

Datasets	$ \mathcal{D} $	$ \mathcal{R} $	$\mathcal{U}_{\text{text}}$	$\mathcal{U}_{\text{image}}$	$ Q^S $
Movie [34]	1,375,738	2	1,375,738	0	2
Wildlife [34]	8,718	2	0	8,718	1
E-Commerce [34]	44,446	2	44,446	44,446	3
MMQA [34]	5,000	7	5,000	1,000	5
Medical [34]	11,112	6	1,200	10,012	4

Table 3: The tested datasets

tion, it comprises uncovered or inconsistent predicate units. Using enrichment lineage, LiteDBX maps these changes to dependent attributes, materialized values, rewrites, and query scopes.

Selective refresh. LiteDBX refreshes the localized state from least to most costly: (1) *values only*, when existing attributes and rewrites remain valid; (2) *rewrites*, when existing attributes still support the revised semantics; (3) *attributes and rewrites*, when additional semantic attributes are required, re-evaluating enrichment under β_{SE} and materializing them only over affected scopes; and (4) SEQR, only if no bounded refresh can preserve the required accuracy.

Example 7: Continuing Example 2, we illustrate workload evolution using the enriched configuration from Example 6.

Dynamic queries. Suppose the workload expands to pneumonia screening using both CXR images and clinical notes. A new query q_p^S asks whether a patient has pneumonia based on clinical risk factors, X-ray, and clinical notes. Existing attributes B_1 , B_2 , and B_4 remain useful, while uncovered predicate units require $B_5 = \text{notepna}$ (from clinical notes) and $B_6 = \text{lobarconsol}$ (for lobar consolidation in CXRs). LiteDBX thus refreshes only the affected enrichment and rewrite under the fixed schema budget, yielding $\mathcal{R}^E = \mathcal{R} \cup \{B_1, B_2, B_4, B_5, B_6\}$ and a corresponding SQL rewrite (e.g., for t_p in Table 1).

Semantic drift. Suppose updated clinical criteria revise the screening predicate to include pulmonary-edema or heart-failure cues. The corresponding units fail the coverage-and-consistency test, so LiteDBX refreshes their affected attributes and rewrite, falling back to full SEQR only if bounded refresh is insufficient. $\square$

Consistent publication. Queries continue using the last published configuration and its corresponding data snapshot while refresh is performed off the query path. The refreshed state is atomically published as one version only after all dependent artifacts are materialized, so concurrent queries observe a consistent old or new version, never stale or partial enrichment.

Unified reuse-or-refresh. Thus, data and workload evolution follow the same principle: validate reuse, localize invalid state, and refresh the smallest affected state before falling back to full SEQR.

7 EXPERIMENTAL STUDY

Using real-world datasets, we evaluated the effectiveness, monetary cost, runtime efficiency and maintenance behavior of LiteDBX.

Experimental setting. We start with our settings.

Datasets. We used five SemBench datasets [34], preserving the links between their base tables and associated unstructured modalities. Table 3 summarizes their statistics: (1) Movie contains movie records and user reviews; (2) Wildlife contains camera-trap images; (3) E-Commerce contains product images and descriptions; (4) MMQA contains textual contexts, some linked to images; and (5) Medical contains symptom descriptions and medical images.

Semantic workloads. We used the SemBench semantic queries under three workload regimes: (1) the static regime uses the original

Method	Metric	Movie			Wildlife			E-Comm.			MMQA			Medical		
		max	min	avg	max	min	avg	max	min	avg	max	min	avg	max	min	avg
LiteDBX	F_1	1.00	0.96	0.98	1.00	1.00	1.00	1.00	0.87	0.96	1.00	0.92	0.98	0.89	0.57	0.68
	\$	0.0003	0.0002	0.0003	0.0005	0.0005	0.0005	0.0006	0.0002	0.0004	0.0004	0.0002	0.0003	0.0005	0.0003	0.0004
	Time	6.51	3.59	5.05	9.20	9.20	9.20	8.60	4.59	6.53	7.90	0.68	5.75	8.31	4.50	6.29
LOTUS	F_1	0.88	0.80	0.84	0.89	0.89	0.89	1.00	0.64	0.77	0.76	0.20	0.48	0.79	0.00	0.37
	\$	0.8520	0.0490	0.4505	0.1100	0.1100	0.1100	1.2900	0.9800	1.1400	0.1000	0.0750	0.0844	0.5800	0.0520	0.2448
	Time	992.96	67.00	529.98	406.98	406.98	406.98	1996.78	881.00	1618.73	138.41	77.07	112.03	683.00	77.50	339.89
Palimpzest	F_1	0.88	0.80	0.84	1.00	1.00	1.00	1.00	0.50	0.67	1.00	0.76	0.92	0.88	0.26	0.52
	\$	0.8600	0.0380	0.4490	0.0660	0.0660	0.0660	1.5200	0.8300	1.2067	0.0430	0.0320	0.0366	0.7000	0.1023	0.3376
	Time	939.53	89.68	514.61	146.10	146.10	146.10	890.00	418.23	673.15	54.14	25.11	43.17	689.12	115.85	313.15
ThalamusDB	F_1	0.78	0.46	0.62	0.33	0.33	0.33	0.73	0.22	0.48	1.00	0.07	0.41	0.32	0.19	0.26
	\$	0.6800	0.0780	0.3790	0.1000	0.1000	0.1000	1.3700	0.2880	0.8290	0.2400	0.1000	0.1660	0.8200	0.0360	0.3097
	Time	2262.36	263.27	1262.82	251.90	251.90	251.90	1115.02	224.67	669.85	1332.12	370.85	901.27	2350.88	292.17	984.95
BigQuery	F_1	0.85	0.60	0.73	1.00	1.00	1.00	1.00	0.30	0.63	1.00	0.60	0.81	0.85	0.19	0.56
	\$	0.1037	0.0062	0.0550	0.1093	0.1093	0.1093	1.5122	0.2142	0.8272	0.0152	0.0095	0.0118	2.7591	0.0465	0.8639
	Time	121.24	8.48	64.86	168.97	168.97	168.97	1808.68	221.53	768.80	18.90	2.46	12.04	1103.59	59.01	436.22
BARGAIN	F_1	0.88	0.64	0.76	X	X	X	0.88	0.88	0.88	0.76	0.04	0.29	0.17	0.17	0.17
	\$	1.1400	0.0750	0.6075	X	X	X	0.1874	0.1874	0.1874	0.0046	0.0015	0.0023	0.0230	0.0230	0.0230
	Time	1209.25	114.49	661.87	X	X	X	29.69	29.69	29.69	18.26	5.03	11.27	18.60	18.60	18.60
iPDB	F_1	0.88	0.63	0.76	X	X	X	0.39	0.39	0.39	0.76	0.10	0.36	0.18	0.18	0.18
	\$	1.3750	0.0800	0.7275	X	X	X	0.1800	0.1800	0.1800	0.0073	0.0023	0.0043	0.0023	0.0023	0.0023
	Time	810.39	55.73	433.06	X	X	X	84.75	84.75	84.75	10.30	8.80	9.52	11.59	11.59	11.59
LOTUS+Cache	F_1	0.86	0.67	0.77	1.00	1.00	1.00	0.92	0.64	0.76	0.86	0.22	0.53	0.79	0.00	0.37
	\$	0.0003	0.0002	0.0003	0.0005	0.0005	0.0005	0.0006	0.0002	0.0004	0.0004	0.0002	0.0003	0.0005	0.0003	0.0004
	Time	6.78	3.88	5.33	9.22	9.22	9.22	8.85	5.70	7.35	7.92	4.99	6.85	9.31	4.79	6.93

Table 4: Static results across queries. Per-query results are in [4]. Best results are bolded, second-best are underlined, and **X** denotes unsupported.

queries and scale factors; (2) data evolution initializes each dataset with 60% of its tuples and adds 20% at each update, yielding 33% and 66% growth relative to the initial dataset; and (3) query evolution tests paraphrases, new combinations of known predicate units, unseen units, and revisions to existing semantic criteria. The first two test reuse of the current enriched configuration, whereas the latter two test when selective refresh is required. For each update, we use the same query to compare reuse without refresh, selective refresh, a full SEQR rerun, and online execution; see [4] for more details.

Metrics and supervision. Following SemBench, we report the F_1 (Section 3), runtime, and LLM monetary cost based on token usage. The avg column reports query-macro F_1 within each dataset, while cross-dataset results equally weight the five workloads. Under evolution, we additionally report pre- and post-refresh F_1 , certificate slack and violations, safe-reuse (*i.e.*, reuse with an observed error rate at most η on the updated data) and required-refresh rates, and unsafe-reuse and unnecessary-refresh rates at tolerance $\eta \in [0, 1]$.

We separately account for user labels and annotation time, paid LLM labels and oracle calls, offline setup and selective refresh, and recurring execution. Benchmark ground truth is revealed only for tuples in O_i and is otherwise used solely for evaluation; pseudo and propagated labels are outputs of SEQR and are excluded from the user-label budget. Human effort and LLM cost are reported separately; the full protocol and per-dataset counts are in [4].

Baselines. We implemented LiteDBX in Python and compared it against the following methods: (1) LOTUS [50, 51], which uses sampling and proxy models; (2) Palimpzest [38], which optimizes declarative semantic queries for cost, latency and quality; (3) ThalamusDB [27, 28], which estimates query answers under a labeling budget; (4) BigQuery [22], an industrial multimodal semantic-query system; (5) BARGAIN [66], which reduces oracle calls through a proxy-oracle cascade; (6) iPDB [32], which optimizes SQL queries containing ML and LLM predicates; and (7) LOTUS+Cache, which caches prior LOTUS results, reuses the most similar cached instances, and otherwise invokes the original pipeline. For ThalamusDB and BARGAIN, we matched their label/oracle sources and budgets, and accounted for every BARGAIN oracle call. iPDB receives no user labels; instead, we matched its on-line model and prompt, and accounted for every call on supported

predicates. Together, these baselines cover query-time evaluation, label-assisted inference, caching, and proxy-oracle cascades. We evaluated each system using its native predicate language and report unsupported workloads rather than translating them. We exclude vector stores such as LanceDB [49], which perform neither query-aware schema enrichment nor semantic-predicate rewriting.

Parameters and configuration. By default, we set $\beta_{SE} = 5$, $\beta_{lab} = 20$, $\beta_{rew} = 5$, and $\eta = 0.6$. We used Qwen3.6-Plus as the online LLM and VLM, and locally hosted Qwen3-30B-A3B-Instruct (FP8) and Qwen3-VL-30B-A3B-Instruct (FP8) as their respective proxies. Experiments were run on a server with 504 GB RAM, 104 Intel Xeon Gold 5320 CPUs @ 2.20 GHz, and four NVIDIA RTX 3090 24 GB GPUs using vLLM [33] with parallelism 20. Unless stated otherwise, we repeat each experiment three times and report the mean. For certificate calibration in Section 5.3, we fixed $|V_i| = \lceil 0.3\beta_{lab} \rceil$ and $\delta = 0.2$ before test evaluation and repeated each study over 3 independent workload-level trials covering all queries. Each dynamic trial started from its corresponding certified static configuration.

Experimental results. We next report our findings.

Exp-1: Compiling semantic interpretation. We first tested whether SEQR can compile semantic interpretation for recurring execution without sacrificing answer quality.

Accuracy under routine workloads. As shown in Table 4, LiteDBX achieves the best overall accuracy, with an average F_1 of 0.92 across the five dataset workloads. Besides, (1) across the tested datasets, LiteDBX outperforms the zero-shot LOTUS, Palimpzest, BigQuery and LOTUS+Cache by 37.3%, 16.5%, 23.9% and 34.1%, respectively. It matches the strongest baseline on Wildlife and outperforms the strongest baseline on the other four datasets, ranging from 6.5% to 21.4%. (2) LiteDBX achieves the highest F_1 of 0.68 on the most challenging dataset Medical, while achieving $F_1 \geq 0.96$ on the other datasets. (3) Although LOTUS+Cache approaches LiteDBX’s recurring execution cost, its F_1 remains 25.4% lower. That is, caching avoids repeated calls for previously matched instances but does not construct the query-aware representation and executable predicates needed for accurate semantic interpretation. These results validate the end-to-end benefit of compilation; Exp-4 further isolates the effects of supervision, enrichment, and rewriting (see below).

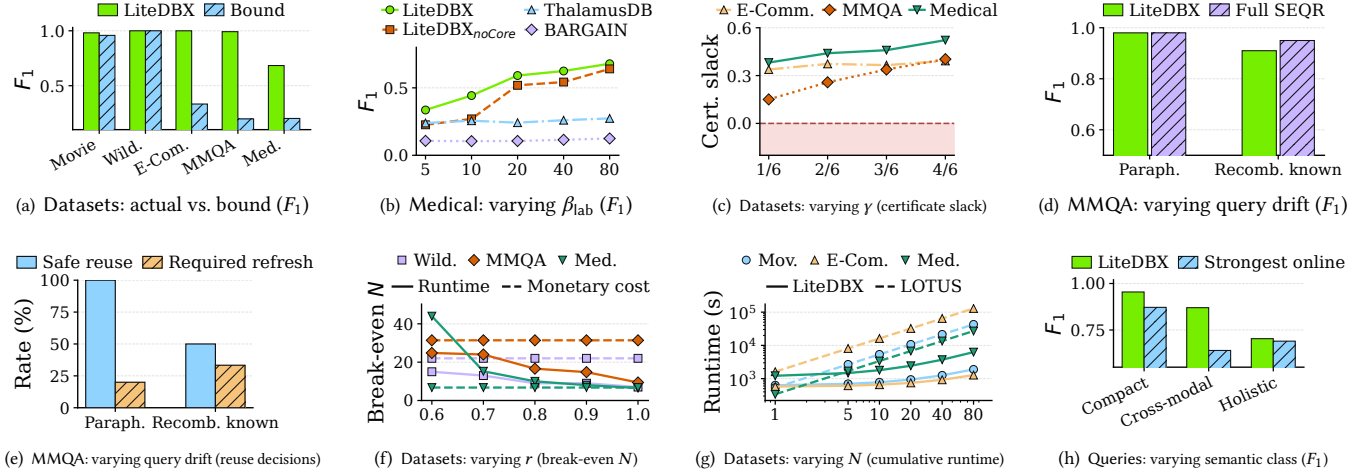


Figure 3: Performance evaluation.

Matched supervision. We tested whether LiteDBX’s accuracy gains persist when supervision is controlled (not shown; in [4]). On commonly supported workloads, we gave LiteDBX, ThalamusDB, and BARGAIN the same label/oracle source and budget $\beta_{lab} = 20$ per query, while retaining each system’s own procedure. We tested simulated user labels, revealed only when requested, and same-LLM labels under the same model and prompt, charging every call.

With simulated user labels, LiteDBX improves average F_1 over ThalamusDB and BARGAIN by 161.3% and 67.3%, respectively; with same-LLM labels, the gains are 73.4% and 46.2%. Since the source and budget are matched, these gains arise from how SEQR uses the supervision for enrichment and rewriting.

User labeling takes an average of 400 s per query and is included as a one-time cost; see Exp-3 for the break-even analysis.

Static-bound calibration. We empirically evaluated the validity and tightness of the lower bound in Theorem 3. As shown in Figure 3(a), (1) the observed macro F_1 never falls below its workload-level bound on any of the five workloads; (2) the bound closely tracks the observed F_1 on Movie and Wildlife, and remains nonzero on the other three workloads; and (3) across repeated trials, both the any-query and workload-bound violation rates are zero. These results empirically validate the guarantee of Theorem 3.

Impact of the labeling budget. We varied β_{lab} from 5 to 80 on the challenging Medical workload and compared LiteDBX with LiteDBX_{noCore}, ThalamusDB and BARGAIN, whenever applicable. Here LiteDBX_{noCore} uses only the user-labeled set O_i , and all methods use the same label source and budget. As shown in Figure 3(b), (1) increasing β_{lab} raises LiteDBX’s F_1 from 0.00 to 0.62, while ThalamusDB and BARGAIN remain within 0.09–0.15 and 0.10–0.12, respectively; this is because both baselines use additional labels to calibrate fixed predictions rather than update their semantic models. (2) The F_1 of LiteDBX_{noCore} stays at 0.00 with 5–10 labels, then rises to 0.04 and 0.51 with 20 and 80 labels, respectively. The coreset is most useful at moderate budgets, where pseudo-labels augment limited user supervision; as the budget grows, LiteDBX_{noCore} approaches LiteDBX in F_1 , narrowing their gap to 0.11 with 80 labels. We therefore set $\beta_{lab} = 20$ to balance F_1 and annotation cost.

Varying β_{SE} . We varied the schema-enrichment budget β_{SE} from 1

to 9 on MMQA (not shown; see [4]). We find that the F_1 initially increases, then stabilizes, and finally declines when β_{SE} exceeds 5. This is because (a) a larger budget admits more informative semantic attributes, improving schema expressiveness and rewrite accuracy; in contrast, (b) an excessive budget introduces noise and redundancy, increasing the risk of overfitting under limited supervision. We therefore recommend setting $\beta_{SE} = 5$.

Exp-2: Reuse and maintenance under evolution. We examined whether the enriched configuration remains reusable as data and semantic predicates evolve, and how much evolution it can accommodate before requiring selective refresh. For data evolution, denote by ΔD^E (resp. $\gamma = |\Delta D^E|/|D^E|$) an update (resp. update ratio).

Data evolution. We tested whether the current configuration remains accurate and reusable as the data grows. As shown in Table 5, LiteDBX achieves an average F_1 of 0.91 across five workloads and three growth levels; see [4] for per-query update results. Besides, (1) LiteDBX improves average F_1 over LOTUS, Palimpzest, ThalamusDB, BigQuery and LOTUS+Cache by 37.3%, 21.2%, 76.1%, 27.7% and 40.3%, respectively, up to 182.6%, 124.1%, 225.0%, 66.7% and 242.1%. (2) As data growth increases from 0 to 33% and 66%, its average F_1 changes from 0.92 to 0.91 and 0.90, respectively. Even at 66% growth, it retains $F_1 \geq 0.91$ on the first four workloads and 0.67 on the more challenging Medical workload. (3) LiteDBX triggers selective refresh for only 46.7% of the workloads. For the remaining ones, it retains the valid semantic attributes and rewrites, updating only affected tuples within the relevant scope. This validates LiteDBX’s effectiveness under routine data growth without frequent recompilation.

Data-certificate calibration. We varied γ from 0 to 4/6 to evaluate the dynamic certificate in Theorem 4. Before refresh, Figure 3(c) reports the query-level slack $B_i^{new} - \text{Err}_{new}^{(i)}$ for the unchanged pre-update rewrite, where negative values indicate certificate violations. We also tested whether the observed pre-refresh macro F_1 falls below the workload-level bound $\frac{1}{|Q^S|} \sum_i \Phi(\pi_i^{new}, B_i^{new})$ (not shown; see [4]). A query is reusable if $B_i^{new} \leq \eta$, and the configuration is reused only if all affected queries satisfy this condition.

We find that (1) the query-level slack remains nonnegative for every tested γ , yielding an any-query violation rate of zero; and

Method	Metric	Movie			Wildlife			E-Comm.			MMQA			Medical		
		$\gamma = 0$	$\gamma = 33\%$	$\gamma = 66\%$	$\gamma = 0$	$\gamma = 33\%$	$\gamma = 66\%$	$\gamma = 0$	$\gamma = 33\%$	$\gamma = 66\%$	$\gamma = 0$	$\gamma = 33\%$	$\gamma = 66\%$	$\gamma = 0$	$\gamma = 33\%$	$\gamma = 66\%$
LiteDBX	F_1	0.98	0.98	0.98	1.00	1.00	1.00	0.97	0.93	0.91	0.99	0.98	0.96	0.65	0.68	0.67
	\$	0.0053	0	0	0.0204	0	0	0.0125	0	0	0.0046	0	0	0.0123	0	0
	Time	84.99	15.73	31.67	196.18	9.61	21.93	67.84	11.63	20.51	53.26	6.16	18.01	278.09	67.00	130.94
LOTUS	F_1	0.80	0.81	0.78	1.00	0.86	0.89	0.86	0.81	0.73	0.52	0.51	0.50	0.23	0.29	0.37
	\$	0.2885	0.3720	0.4510	0.0650	0.0870	0.1100	0.6933	0.9267	1.1667	0.0508	0.0680	0.0864	0.1480	0.1958	0.2448
	Time	347.16	419.95	535.81	256.89	334.79	406.98	1061.27	1293.94	1616.04	78.35	98.70	119.13	221.38	291.31	338.30
Palimpzest	F_1	0.84	0.74	0.84	1.00	0.85	1.00	0.71	0.68	0.67	0.92	0.84	0.93	0.29	0.46	0.52
	\$	0.2635	0.3510	0.4490	0.0400	0.0500	0.0650	0.7200	0.9567	1.2033	0.0296	0.0400	0.0492	0.1430	0.1850	0.3376
	Time	309.36	424.00	514.61	91.92	120.04	149.07	366.76	478.10	672.85	37.53	51.56	66.44	143.30	192.34	313.15
ThalamusDB	F_1	0.73	0.69	0.62	1.00	0.86	0.33	0.59	0.57	0.48	0.42	0.40	0.41	0.20	0.21	0.26
	\$	0.3875	0.3840	0.3790	0.0600	0.0800	0.1000	0.8250	0.8400	0.8290	0.1034	0.1280	0.1740	0.2333	0.2800	0.3097
	Time	1438.99	1318.92	1262.82	175.25	258.06	296.38	626.99	657.34	669.85	543.74	676.22	855.05	805.16	889.83	984.95
BigQuery	F_1	0.61	0.68	0.73	1.00	0.85	1.00	0.70	0.68	0.63	0.80	0.80	0.81	0.39	0.47	0.56
	\$	0.0344	0.0447	0.0550	0.0655	0.0873	0.1093	0.6969	0.7599	0.8258	0.0072	0.0095	0.0118	0.8352	0.8489	0.8639
	Time	39.28	61.36	64.86	76.92	298.40	168.97	580.00	790.71	768.80	7.01	7.91	12.04	415.74	464.66	436.22
BARGAIN	F_1	0.66	0.64	0.66	X	X	X	0.67	1.00	0.88	0.29	0.23	0.23	0.09	0.11	0.17
	\$	0.0087	0.0111	0.0126	X	X	X	0.1102	0.1483	0.1874	0.0023	0.0032	0.0038	0.0138	0.0183	0.0230
	Time	15.08	16.46	18.59	X	X	X	18.41	23.61	29.69	11.29	11.18	15.38	15.04	17.89	18.63
iPDB	F_1	0.71	0.70	0.71	X	X	X	0.42	0.67	0.40	0.45	0.38	0.38	0.15	0.09	0.15
	\$	0.0108	0.0139	0.0170	X	X	X	0.1057	0.1423	0.1799	0.0026	0.0034	0.0043	0.0043	0.0128	0.0175
	Time	23.54	41.67	36.57	X	X	X	50.59	67.16	95.64	5.06	6.13	7.49	9.28	28.44	40.16
LOTUS+Cache	F_1	0.77	0.76	0.77	1.00	0.89	0.83	0.70	0.77	0.76	0.57	0.53	0.53	0.19	0.31	0.37
	\$	0.2845	0.0800	0.0850	0.0650	0.0220	0.0220	0.6967	0.2200	0.3933	0.0520	0.0176	0.0164	0.1503	0.0823	0.0175
	Time	281.47	86.58	198.08	254.74	91.58	180.68	952.61	308.90	708.89	66.02	32.76	61.58	178.55	104.33	133.36

Table 5: Data evolution. Per-query results are in [4]. Best results are bolded; second-best results are underlined; and **X denotes unsupported.**

(2) the observed pre-refresh macro F_1 never falls below its bound, yielding a workload-bound violation rate of zero. These results empirically validate Theorem 4 under data evolution.

Aligned-query reuse. We evaluated whether paraphrased and recombined queries can reuse the current semantic attributes and rewrites. A predicate unit is reused only when semantic alignment identifies supporting attributes and rewrite fragments, and rewrite consistency verifies their compatibility. Figure 3(e) reports the safe-reuse and required-refresh rates, while Figure 3(d) compares F_1 with a full SEQR rerun. We find the following. (1) For paraphrased queries, the safe-reuse rate is 100%. (2) For recombined-known queries, the safe-reuse rate is 50%, with an F_1 gap of only 0.04; cross-query matches account for 66.7% of the reused units. These indicate that LiteDBX effectively reuses semantic attributes/rewrites across formulations when attribute-predicate support remains valid.

Ablation of incremental maintenance. We tested incremental maintenance using the same pre-update configurations and update traces (not shown; see [4]). For query evolution, we compared LiteDBX with the following variants: (a) LiteDBX_{reuse}, which always reuses the current configuration; (b) LiteDBX_{align}, which performs semantic alignment without rewrite-consistency checking and conservatively refreshes unresolved cases; and (c) LiteDBX_{noMain}, which reconstructs the entire configuration after each update. For data evolution, where semantic alignment does not apply, we compared LiteDBX with LiteDBX_{reuse} and LiteDBX_{noMain}.

We find that (1) for query updates, LiteDBX reduces the refresh fraction and runtime by 37.2% and 31.0%, respectively, relative to LiteDBX_{align}, while retaining comparable post-refresh F_1 . (2) For data updates, LiteDBX refreshes only 46.7% of the affected state and reduces the unsafe-reuse rate by 83.3% over LiteDBX_{reuse} and the unnecessary-refresh rate by 66.8% over LiteDBX_{noMain}. (3) LiteDBX achieves a final post-update F_1 within 0.01 of LiteDBX_{noMain} and reduces runtime and monetary cost by 46.1% and 76.9%, respectively, and beats LiteDBX_{reuse} by 7.4% in F_1 . This validates LiteDBX’s effectiveness for incremental maintenance via selective refresh.

Varying update magnitude and tolerance. We varied γ and η on Medical, the most challenging dataset (see [4] for the complete curves). We find the following. (1) As γ increases from 1/6 to 4/6,

pre-refresh F_1 changes from 0.67 to 0.63; at $\gamma = 2/6$, SEQR refresh yields 0.69. (2) At $\eta = 0.5, 0.6, 0.7$ and 0.8 , the resulting F_1 scores are 0.67, 0.66, 0.65 and 0.65, respectively. This is because larger updates lead to greater drift from the current configuration, while a larger η permits more error before triggering refresh. These results indicate that LiteDBX is robust to varying update magnitudes and effectively balances accuracy and refresh frequency through η .

We also tested the semantic-drift boundary of LiteDBX and analyzed its reuse behavior across semantic shifts; see [4] for details.

Exp-3: Efficiency, amortization, and scalability. We tested whether compilation reduces recurring execution and maintenance costs after accounting for its one-time construction cost; monetary costs below \$0.0001 are reported as zero.

Execution and maintenance costs. We compared LiteDBX’s recurring execution and incremental maintenance costs with those of the online baselines. Tables 4 and 5 report execution and maintenance costs after initial construction and updates, respectively (see [4] for a detailed component-level cost breakdown). We find the following. (1) In the static setting, LiteDBX averages 6.56s and \$0.00038 per execution, 59.9× faster and 99.9% cheaper than the online baselines. It is also 9% faster than LOTUS+Cache at a similar monetary cost, while retaining its 34.1% F_1 advantage (Exp-1). This is because routine executions run rewritten SQL over materialized attributes without online LLM calls or semantic-cache lookups. (2) Under data and query evolution, LiteDBX is 98.5% cheaper on average and 3.9× and 2.1× faster, respectively, than the strongest online baselines. Across the five workloads and the 33% and 66% data updates, incremental maintenance takes only 33.3s on average; this is because it populates attributes only for new in-scope tuples, without repeating candidate generation or joint selection. (3) Compared against LiteDBX_{noMain}, which reruns SEQR from scratch, LiteDBX reduces monetary cost by 76.9% and runtime by 46.1% through incremental maintenance, with a 0.01 change in F_1 . This validates the benefit of LiteDBX in retaining the reusable elements.

Compilation break-even. We next evaluated when repeated executions amortize LiteDBX’s one-time annotation and compilation costs. For repeated or aligned queries, we accumulate annotation, SEQR construction (including certification), materialization, re-

fresh, and execution costs, plus each baseline’s setup and recurring costs. Let r be the fraction of executions served without refresh and N be the smallest execution count at which LiteDBX’s cumulative runtime or monetary cost is no greater than a baseline’s.

Figure 3(f) reports per-dataset runtime and monetary-cost break-even counts against Palimpzest on Wildlife, MMQA, and Medical as r varies. At $r = 0.8$, Figure 3(g) reports cumulative runtime against LOTUS on Movie, E-Commerce and Medical, including setup costs in the first execution. Both figures include the 400 s annotation cost and measured one-time compilation cost.

We find that (1) at $r = 0.8$, LiteDBX reaches runtime and monetary-cost break-even against Palimpzest after an average of 11.9 and 20.1 executions, respectively; and (2) as r increases from 0.6 to 1.0, the average runtime break-even count decreases from 27.9 to 7.6 executions, while the monetary-cost break-even count remains unchanged. These results show that LiteDBX favors recurring workloads with substantial reuse, whereas online processing is preferable for one-off or rapidly evolving workloads.

We also evaluated LiteDBX’s storage overhead and SEQR’s scalability across data sizes, modalities, and workload sizes (see [4]).

Exp-4: Sources of gain and applicability. We isolated LiteDBX’s gains from supervision, schema enrichment, and rewriting, and identified when its compilation-based strategy is less suitable than online semantic processing in practice.

Supervision, enrichment and rewriting. We compared LiteDBX with three variants: LiteDBX_{noSup} replaces the user-provided labels in O_i with LLM-generated labels, leaving SEQR unchanged; LiteDBX_{noEnr} rewrites queries over the raw schema; and LiteDBX_{noRew} retains the enriched schema but evaluates semantic predicates using the learned proxy rather than the selected SQL rewrite. All variants use the same sampled tuples and, where applicable, the same budgets (see [4] for query-level results across five datasets).

We find the following. (1) LiteDBX beats LiteDBX_{noSup} by 14.1% on average in F_1 , with a maximum relative gain of 416.7% on Medical Q_1 (0.62 vs. 0.12). This gain stems from human-guided semantic alignment. (2) It beats LiteDBX_{noEnr} by 16.6% on average. Among queries with nonzero baseline F_1 , its maximum relative gain is 95.7% on MMQA Q_1 (0.92 vs. 0.47); on Medical Q_1 , it raises F_1 from 0 to 0.62. This is because schema enrichment captures query-relevant semantics as reusable structured attributes unavailable in the raw schema. (3) It beats LiteDBX_{noRew} by 9.2% on average, with a maximum relative gain of 1325% on Medical Q_4 (0.57 vs. 0.04). This is because query rewriting converts the enriched representation into an explicit, executable predicate that more accurately captures the semantics. These validate the complementary roles of the three components: supervision establishes the semantics, enrichment represents them, and rewriting makes them executable.

Ablation of SEQR’s algorithmic components. We evaluated the contributions of SEQR’s algorithmic components by comparing LiteDBX with three variants under the same β_{lab} , β_{SE} , and β_{rew} (not shown; see [4]): (1) LiteDBX_{noCore} uses only the user-labeled set O_i ; (2) LiteDBX_{seq} selects attributes based on predictive utility and then derives rewrites over the fixed attribute set; and (3) LiteDBX_{pred} retains SEQR’s candidate-configuration search but ranks configurations solely by propagation error, omitting rewrite fidelity.

On average, LiteDBX outperforms LiteDBX_{noCore}, LiteDBX_{seq}

and LiteDBX_{pred} by 5.8%, 4.1% and 10.1% in F_1 , respectively, up to 30.8%, 14.6% and 30.6%. This is because LiteDBX jointly integrates coreset construction, configuration search, and fidelity-aware selection. By contrast, LiteDBX_{noCore} relies on the sparse and potentially biased O_i ; LiteDBX_{seq} commits prematurely to predictive attributes under a limited rewrite budget; and LiteDBX_{pred} may select configurations that are difficult to rewrite accurately. This result validates the effectiveness of jointly optimizing all these components.

Compilation boundary. We tested which application scenarios suit LiteDBX’s compilation-based strategy. Complementing the break-even analysis of required workload reuse (Exp-3), this experiment examines whether the underlying semantics can be accurately and compactly represented by enriched attributes and explicit rewrites. For each semantic predicate, we sweep the tested $(\beta_{SE}, \beta_{rew})$ configurations and select the least costly one whose F_1 is within five points of the strongest online baseline. Predicates with no qualifying configuration fall outside the tested compilation region. The selected configuration characterizes each predicate’s empirical relationalizability under the tested budgets. See [4] for the full configurations.

As shown in Figure 3(h), (1) LiteDBX outperforms the strongest online baseline by 9.5% on compact, attribute-expressible scenarios, using 4.5 attributes and 1.3 rewrite predicates; (2) it beats the baseline by 35.9% on cross-modal scenarios, which require 4 attributes and 2 rewrite predicates; and (3) it achieves F_1 comparable to that of the baseline on representation-heavy or holistic scenarios, but requires more complex compilation configurations. That is, LiteDBX suits compactly representable scenarios with sufficient workload reuse; its benefit diminishes as representation complexity increases.

Summary. We find the following. (1) LiteDBX achieves an average F_1 of 0.92 across the workloads, outperforming the best-performing label-assisted (resp. zero-shot) baseline by 71.6% (resp. 16.5%). (2) Its F_1 decreases only from 0.92 to 0.90 under 66% data growth and recovers from 0.41 to 0.91 through selective refresh under the tested MMQA shift. (3) LiteDBX is 59.9× faster and 99.9% cheaper than online baselines on average, with its upfront compilation cost amortized through reuse. (4) Ablations show that human supervision, schema enrichment and query rewriting improve average F_1 by 14.1%, 16.6% and 9.2%, respectively. (5) LiteDBX suits recurring workloads with compactly relationalizable semantics; otherwise, online processing is preferable.

8 CONCLUSION

The novelty of LiteDBX lies in turning recurring semantic computation into *workload-specific relational state*. It (1) compiles semantic predicates over relational and unstructured data into reusable, standard SQL for predictable online execution; (2) jointly selects workload-relevant semantic attributes and their SQL rewrites under limited supervision, with end-to-end guarantees on accuracy; and (3) maintains the resulting enriched state through a reuse-or-refresh strategy that certifies reusable state and selectively refreshes affected attributes and rewrites as data or query intent evolves. Our experiments show that LiteDBX is promising in practice.

For future work, we will develop automatic configuration mechanisms for LiteDBX, including adaptive selection of enrichment, supervision, and rewrite budgets, and maintenance thresholds that balance accuracy, reuse, and refresh cost under evolving workloads.

REFERENCES

- [1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.
- [2] Martin Anthony and Peter L Bartlett. 2009. *Neural Network Learning: Theoretical Foundations*. Cambridge University Press.
- [3] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avnika Narayan, Andrew Hojell, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *PVLDB* (2023), 92–105.
- [4] Wenchao Bai, Wenfei Fan, Weilong Ren, Zihuan Xu, Sihan Wang, and Jiahui Jin. 2026. Code, Datasets and Full Version. <https://github.com/SICS-Fundamental-Research-Center/litedbx>.
- [5] Avrim Blum and John Langford. 2003. Pac-mdl Bounds. In *COLT*. 344–357.
- [6] Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K Warmuth. 1989. Learnability and the Vapnik-Chervonenkis Dimension. *JACM* 36, 4 (1989), 929–965.
- [7] Prabir Burman. 1989. A Comparative Study of Ordinary Cross-Validation, V-Fold Cross-Validation and the Repeated Learning-Testing Methods. *Biometrika* 76, 3 (1989), 503–514.
- [8] Chengliang Chai, Jiajun Li, Yuhao Deng, Yuanhao Zhong, Ye Yuan, Guoren Wang, and Lei Cao. 2025. Doctopus: Budget-Aware Structural Table Extraction from Unstructured Documents. *PVLDB* 18, 11 (2025), 3695–3707.
- [9] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *SIGKDD*. 785–794.
- [10] Zui Chen, Zihui Gu, Lei Cao, Ju Fan, Samuel Madden, and Nan Tang. 2023. Symphony: Towards Natural Language Query Answering over Multi-modal Data Lakes. In *CIDR*. 1–7.
- [11] Nadia Chepurko, Ryan Marcus, Emanuel Zraggen, Raul Castro Fernandez, Tim Kraska, and David Karger. 2020. ARDA: Automatic Relational Data Augmentation for Machine Learning. In *PVLDB*. 2150–8097.
- [12] Yeounoh Chung, Rushabh Desai, Jian He, Yu Xiao, Thibaud Hottelier, Yves-Laurent Kom Samo, Pushkar Khadilkar, Xianshun Chen, Sam Idicula, Fatma Özcan, Alon Halevy, and Yannis Papakonstantinou. 2026. 100x Cost & Latency Reduction: Performance Analysis of AI Query Approximation using Lightweight Proxy Models: [Experiments & Analysis]. *Proc. ACM Manag. Data* 4, 3 (2026), 1–23.
- [13] Cosmin Citu, Oana Maria Gorun, Andrei Motoc, Ioana Mihaela Citu, Florin Gorun, and Daniel Malita. 2022. Correlation of Lung Damage on CT Scan with Laboratory Inflammatory Markers in COVID-19 Patients: A Single-Center Study from Romania. *Journal of Clinical Medicine* 11, 15 (2022), 4299.
- [14] E. F. Codd. 1979. Extending the Database Relational Model to Capture More Meaning. *TODS* 4, 4 (1979), 397–434.
- [15] N. Dalvi and D. Suciu. 2007. Efficient Query Evaluation on Probabilistic Databases. *VLDBJ* (2007).
- [16] Javier de la Rúa Martínez, Fabio Buso, Antonios Kouzoupis, Alexandru A Ormisan, Salman Niazi, Davit Bzhalava, Kenneth Mak, Victor Jouffrey, Mikael Ronström, Raymond Cunningham, Ralfs Zangis, Dhananjay Mukhedkar, Ayushman Khazanchi, Vladimir Vlassov, and Jim Dowling. 2024. The Hopsworks Feature Store for Machine Learning. In *Proc. ACM Manag. Data*. 135–147.
- [17] Xiang Deng, Huan Sun, Alyssa Lees, You Wu, and Cong Yu. 2022. TURL: Table Understanding through Representation Learning. *SIGMOD Rec* 51, 1 (2022), 33–40.
- [18] Yuyang Dong, Kunihiro Takeoka, Chuan Xiao, and Masafumi Oyamada. 2021. Efficient Joinable Table Discovery in Data Lakes: A High-Dimensional Similarity-Based Approach. In *ICDE*. 456–467.
- [19] Anas Dorbani, Sunny Yasser, Jimmy Lin, and Amine Mhedhbi. 2025. Beyond Quacking: Deep Integration of Language Models and RAG into DuckDB. *PVLDB* (2025), 5415–5418.
- [20] Mahdi Esmailoghli, Jorge-Arnulfo Quiáné-Ruiz, and Ziawasch Abedjan. 2021. COCOA: COrrelation COefficient-Aware Data Augmentation. In *EDBT*. 331–336.
- [21] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée Miller. 2023. Semantics-Aware Dataset Discovery from Data Lakes with Contextualized Column-Based Representation Learning. *PVLDB* (2023), 1726–1739.
- [22] Sérgio Fernandes and Jorge Bernardino. 2015. What Is Bigquery?. In *IDEAS*. 202–203.
- [23] José Ferreira, Fernando Almeida, and José Monteiro. 2017. Building an Effective Data Warehousing for Financial Sector. *arXiv preprint arXiv:1709.05874* (2017).
- [24] Thomas Ganslandt, Sebastian Mate, Krister Helbing, Ulrich Sax, and HU Prokosch. 2011. Unlocking Data for Clinical Research—The German i2b2 Experience. *Applied clinical informatics* (2011).
- [25] Benjamin Hättasch, Jan-Micha Bodensohn, Liane Vogel, Matthias Urban, and Carsten Binnig. 2023. WannaDB: Ad-hoc SQL Queries over Text Collections. In *BTW*. 157–181.
- [26] Xuming Hu, Shen Wang, Xiao Qin, Chuan Lei, Zhengyuan Shen, Christos Faloutsos, Asterios Katsifodimos, George Karypis, Lijie Wen, and S Yu Philip. 2023. Automatic Table Union Search with Tabular Representation Learning. In *ACL*. 3786–3800.
- [27] Saehan Jo and Immanuel Trummer. 2023. Demonstration of ThalamusDB: Answering Complex SQL Queries with Natural Language Predicates on Multi-Modal Data. In *SIGMOD*. 179–182.
- [28] Saehan Jo and Immanuel Trummer. 2024. ThalamusDB: Approximate Query Processing on Multi-Modal Data. *Proc. ACM Manag. Data* 2, 3 (2024), 1–26.
- [29] Richard M Karp. 2009. Reducibility among Combinatorial Problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art*. Springer, 219–241.
- [30] Michael J Kearns and Umesh Vazirani. 1994. *An introduction to Computational Learning Theory*. MIT Press.
- [31] Aamod Khatiwada, Grace Fan, Roe Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J Miller, and Mirek Riedewald. 2023. Santos: Relationship-Based Semantic Table Union Search. *Proc. ACM Manag. Data* 1, 1 (2023), 1–25.
- [32] Udeshe Kumarasinghe, Tyler Liu, Chunwei Liu, and Walid G. Aref. 2026. iPDB: Optimizing SQL Queries with ML and LLM Predicates. *arXiv preprint arXiv:2601.16432* (2026).
- [33] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*. 611–626.
- [34] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. 2025. SemBench: A Benchmark for Semantic Query Processing Engines. *arXiv preprint arXiv:2511.01716* (2025).
- [35] Yuliang Li, Aaron Xixuan Feng, Jinfeng Li, Saran Mumick, Alon Halevy, Vivian Li, and Wang-Chiew Tan. 2019. Subjective Databases. *PVLDB* (2019), 1330–1343.
- [36] Yiming Lin, Mawil Hasan, Rohan Kosalge, Alvin Cheung, and Aditya G Parameswaran. 2025. Visual Template Inference for Data Extraction from Documents. *Proc. ACM Manag. Data* 3, 6 (2025), 1–27.
- [37] Yiming Lin, Madelon Hulsebos, Ruiying Ma, Shreya Shankar, Sepanta Zeigham, Aditya G Parameswaran, and Eugene Wu. 2024. Towards Accurate and Efficient Document Analytics with Large Language Models. *arXiv preprint arXiv:2405.04674* (2024).
- [38] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, and Gerardo Vitagliano. 2025. Palimpsest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *CIDR*. 2.
- [39] Jiabin Liu, Chengliang Chai, Yuyu Luo, Yin Lou, Jianhua Feng, and Nan Tang. 2022. Feature Augmentation with Reinforcement Learning. In *ICDE*. 3360–3372.
- [40] Rui Liu, Kwanghyun Park, Fotis Psallidas, Xiaoyong Zhu, Jinghui Mo, Rathijit Sen, Matteo Interlandi, Konstantinos Karanasos, Yuanquan Tian, and Jesús Camacho-Rodríguez. 2023. Optimizing Data Pipelines for Machine Learning in Feature Stores. *PVLDB* 16, 13 (2023), 4230–4239.
- [41] Shicheng Liu, Jialiang Xu, Wesley Tjanganaka, Sina J Semnani, Chen Jie Yu, and Monica S Lam. 2024. SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models. *ACL* (2024), 4535–4555.
- [42] Huanyuan Luo, Yuancheng Wang, Songqiao Liu, Ruoling Chen, Tao Chen, Yi Yang, Duolao Wang, and Shenghong Ju. 2022. Associations Between CT Pulmonary Opacity Score on Admission and Clinical Characteristics and Outcomes in Patients with COVID-19. *Internal and emergency medicine* 17, 1 (2022), 153–163.
- [43] Ankur Mallick, Kevin Hsieh, Behnaz Arzani, and Gauri Joshi. 2022. Matchmaker: Data drift mitigation in machine learning for large-scale systems. *Proceedings of Machine Learning and Systems* 4 (2022), 77–94.
- [44] Christopher D Manning. 2009. *An Introduction to Information Retrieval*. Synpress Publishing.
- [45] Himel Mondal and Muhammad Zubair. 2024. Hematocrit. *StatPearls* (2024).
- [46] Emily Mu, Sarah Jabbour, Adrian V Dalca, John Gutttag, Jenna Wiens, and Michael W Sjoding. 2022. Augmenting Existing Deterioration Indices with Chest Radiographs to Predict Clinical Deterioration. *PLoS One* (2022).
- [47] Fatemeh Nargesian, Erkang Zhu, Ken Q Pu, and Renée J Miller. 2018. Table Union Search on Open Data. *PVLDB* 11, 7 (2018), 813–825.
- [48] Laurel Orr, Atindriyo Sanyal, Xiao Ling, Karan Goel, and Megan Leszczynski. 2021. Managing ML Pipelines: Feature Stores and The Coming Wave of Embedding Ecosystems. *arXiv preprint arXiv:2108.05053* (2021).
- [49] Weston Pace, Chang She, Lei Xu, Will Jones, Albert Lockett, Jun Wang, and Raunak Shah. 2025. Lance: Efficient random access in columnar storage through adaptive structural encodings. *arXiv preprint arXiv:2504.15247* (2025).
- [50] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. Lotus: Enabling Semantic Queries with LLMs over Tables of Unstructured and Structured Data. *arXiv preprint arXiv:2407.11418* (2024).
- [51] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *PVLDB* (2025).
- [52] Jana Sedlakova, Paola Daniore, Andrea Horn Wintsch, Markus Wolf, Mina Stanikic, Christina Haag, Chloé Sieber, Gerold Schneider, Kaspar Staub, Dominik

- Alois Ettlin, Oliver Grubner, Fabio Rinaldi, and Viktor von Wyl. 2023. Challenges and Best Practices for Digital Unstructured Data Enrichment in Health Research: A Systematic Narrative Review. *PLOS digital health* (2023).
- [53] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G. Parameswaran, and Eugene Wu. 2025. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *PVLDB* 18, 9 (2025), 3035–3048.
- [54] Mervyn Stone. 1977. An Asymptotic Equivalence of Choice of Model by Cross-Validation and Akaike's Criterion. *J. R. Stat. Soc. Ser. B Methodol.* 39, 1 (1977), 44–47.
- [55] James Thorne, Majid Yazdani, Marzieh Saeidi, Fabrizio Silvestri, Sebastian Riedel, and Alon Halevy. 2021. Database Reasoning over Text. *ACL* (2021), 3091–3104.
- [56] James Thorne, Majid Yazdani, Marzieh Saeidi, Fabrizio Silvestri, Sebastian Riedel, and Alon Halevy. 2021. From Natural Language Processing to Neural Databases. In *PVLDB*, Vol. 14. 1033–1039.
- [57] Matthias Urban and Carsten Binnig. 2024. CAESURA: Language Models as Multi-Modal Query Planners. In *CIDR*.
- [58] Matthias Urban and Carsten Binnig. 2024. ELEET: Efficient Learned Query Execution over Text and Tables. *PVLDB* 17, 13 (2024), 4867–4880.
- [59] Leslie G Valiant. 1984. A Theory of The Learnable. *CACM* 27, 11 (1984), 1134–1142.
- [60] Vladimir N Vapnik. 1999. An Overview of Statistical Learning Theory. *TNNLS* 10, 5 (1999), 988–999.
- [61] Moshe Y Vardi. 2016. A Theory of Regular Queries. In *PODS*. 1–9.
- [62] Jiayi Wang, Chengliang Chai, Nan Tang, Jiabin Liu, and Guoliang Li. 2022. Core-sets over Multiple Tables for Feature-rich and Data-efficient Machine Learning. *PVLDB* 16, 1 (2022), 64–76.
- [63] Weizhen Wang. 2015. Exact Optimal Confidence Intervals for Hypergeometric Parameters. *J. Amer. Statist. Assoc.* 110, 512 (2015), 1491–1499. <https://doi.org/10.1080/01621459.2014.966191>
- [64] Sarah Wooders, Xiangxi Mo, Amit Narang, Kevin Lin, Ion Stoica, Joseph M Hellerstein, Natacha Crooks, and Joseph E Gonzalez. 2023. RALF: Accuracy-Aware Scheduling for Feature Store Maintenance. *PVLDB* 17, 3 (2023), 563–576.
- [65] Joy Tzung-yu Wu, Miguel Ángel Armengol de La Hoz, Po-Chih Kuo, Joseph Alexander Paguio, Jasper Seth Yao, Edward Christopher Dee, Wesley Yeung, Jerry Jurado, Achintya Moulick, Carmelo Milazzo, Paloma Peinado, Paula Villares, Antonio Cubillo, José Felipe Varona, Hyung-Chul Lee, Alberto Estirado, José Maria Castellano, and Leo Anthony Celi. 2022. Developing and Validating Multi-Modal Models for Mortality Prediction in COVID-19 Patients: A Multi-Center Retrospective Study. *Journal of Digital Imaging* (2022).
- [66] Sepanta Zeighami, Shreya Shankar, and Aditya Parameswaran. 2025. Cut Costs, Not Accuracy: LLM-Powered Data Processing with Guarantees. *Proc. ACM Manag. Data* 3, 6 (2025), 1–26.
- [67] Jianing Zhang, Zhaojing Luo, Quanqing Xu, and Meihui Zhang. 2023. PA-FEAT: Fast Feature Selection for Structured Data Via Progress-Aware Multi-Task Deep Reinforcement Learning. In *ICDE*. 394–407.
- [68] Jiani Zhang, Hengrui Zhang, Rishav Chakravarti, Yiqun Hu, Patrick Ng, Asterios Katsifodimos, Huzefa Rangwala, George Karypis, and Alon Halevy. 2025. CodLLM: Empowering Large Language Models for Data Analytics. *arXiv preprint arXiv:2502.00329* (2025).
- [69] Zixuan Zhao and Raul Castro Fernandez. 2022. Leva: Boosting Machine Learning Performance with Relational Embedding Data Augmentation. In *SIGMOD*. 1504–1517.
- [70] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J Miller. 2019. JOSIE: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. In *SIGMOD*. 847–864.

A PROOF OF THEOREM 1

Theorem 1: *The schema enrichment problem is NP-hard.* $\square$

Proof: We show that schema enrichment is NP-hard by reduction from the NP-complete set cover problem [29]. Consider a set cover instance with an element universe $W = \{w_1, \dots, w_n\}$, a collection of subsets $\mathcal{S} = \{S_1, \dots, S_\ell\}$ where $S_j \subseteq W$, and a budget k . The task is to decide whether there exist at most k sets whose union covers W . We reduce set cover to a relaxed oracle version of the decision schema enrichment problem, in which the ground-truth answer sets G_i are available for query-accuracy evaluation and observed-set labeling. This oracle relaxation only exposes the ground truth used by the decision predicate; it does not add candidate attributes or rewrite forms. The oracle cannot make the decision problem harder, so NP-hardness in this setting suffices.

(1) Schema $\mathcal{R}$ and instance $\mathcal{D}$: Let $\mathcal{R} = \{R(\text{id})\}$ and $\mathcal{D} = \{D\}$, where id is not available to predicates in P_1^T , and D contains $n + 1$ tuples: one positive tuple t_p and n negative tuples $t_1, \dots, t_n$, one for each element $w_i \in W$.

(2) Unstructured data $\mathcal{U}$: Adopting the standard bag-of-words representation [44], we construct $\mathcal{U}$ as a finite set of documents $\{d_p, d_1, \dots, d_n\}$ over vocabulary $\mathcal{V} = \{\tau_1, \dots, \tau_\ell\}$, where d_i (resp. d_p) corresponds to the tuple t_i (resp. t_p) in $\mathcal{D}$, and each token τ_j corresponds to a subset $S_j \in \mathcal{S}$ in the set cover instance:

$$d_p = \mathcal{V}, \quad d_i = \mathcal{V} \setminus \{\tau_j \mid w_i \in S_j\},$$

where d_p contains all tokens in $\mathcal{V}$ and each d_i excludes the tokens whose corresponding sets cover w_i . For example, as in Figure 4, the constructed document d_1 corresponding to the element w_1 does not contain tokens τ_1 and τ_ℓ as both subsets S_1 and S_ℓ can cover w_1 .

(3) Extractable attributes from $\mathcal{U}$: Each token τ_j defines a candidate attribute $B_j(t) = \mathbb{1}[\tau_j \text{ occurs in the document associated with } t]$. Hence, $B_j(t_p) = 1$, while $B_j(t_i) = 0$ iff $w_i \in S_j$.

(4) Semantic query Q^S , observed set O_1 , and ground truth G_1 : Let $Q^S = \{q_1^S\}$ contain one semantic query with the ground truth $G_1 = \{t_p\}$. Its relational scope is $\Sigma_1(\mathcal{D}) = D$; under $\beta_{\text{lab}} = 1$, the observed set is $O_1 = \{t_p\}$ with the positive label induced by G_1 .

(5) Budgets $\beta_{\text{lab}} = 1$, $\beta_{\text{SE}} = k$, $\beta_{\text{rew}} = 1$ and accuracy threshold $\omega = 1$.

The reduction can be constructed in PTIME. Since Q^S contains a single query, $\text{acc}(Q^S, \mathcal{E})$ coincides with its F_1 -score. With $\omega = 1$, we have $\text{precision}_1 = \text{recall}_1 = 1$, and thus $E_1 = G_1 = \{t_p\}$.

We show that there exist extensions $\mathcal{R}^E$ of $\mathcal{R}$ and $\mathcal{D}^E$ of $\mathcal{D}$ and a rewrite q_1^T using at most β_{SE} new attributes and one conjunctive clause, such that $\text{acc}(Q^S, \mathcal{E}) = 1$, iff at most k sets cover W ; the observed-set constraint holds since $|O_1| = 1 \leq \beta_{\text{lab}}$.

$\Rightarrow$ Assume that a feasible enriched configuration achieves $\text{acc}(Q^S, \mathcal{E}) = 1$. Then its single-clause rewrite q_1^T returns t_p and excludes every t_i . Because the clause accepts t_p , every comparison in it is true on t_p . For each excluded t_i , at least one comparison is false on t_i and true on t_p . Since id is unavailable and the candidate pool is exactly $\{B_1, \dots, B_\ell\}$, such a comparison must use an enriched attribute B_j with $B_j(t_i) \neq B_j(t_p)$, hence $B_j(t_i) = 0$ and $w_i \in S_j$. Therefore, the sets corresponding to the enriched attributes used by q_1^T cover every element of W . At most k attributes are available, so these sets form a cover of size at most k .

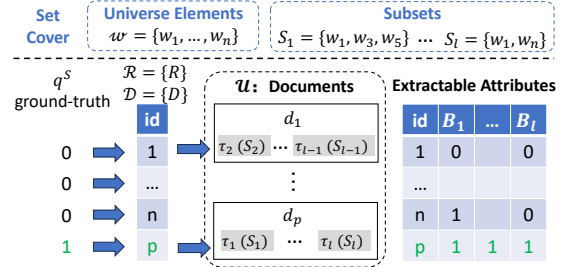


Figure 4: Reduction example

$\Leftarrow$ Conversely, let $\{S_j : j \in J\}$ be a cover with $|J| \leq k$. Add the attributes $\{B_j : j \in J\}$ and define the single-clause rewrite q_1^T by $\bigwedge_{j \in J} (B_j = 1)$. It returns t_p because $B_j(t_p) = 1$ for every j . For each t_i , some selected set S_j contains w_i , so $B_j(t_i) = 0$ and the conjunction excludes t_i . Hence q_1^T returns exactly $\{t_p\}$ and achieves $F_1 = 1$ using at most k attributes and one conjunctive clause. $\square$

B DETAILS FOR SEQR

This section gives details for the SEQR process in Section 5.

Query-aware preparation. It processes in the following steps.

Step 1: Scoped tuple retrieval and initial extraction. For each query $q_i^S = (\Pi, \mathcal{F}, \Sigma_i, P_i^S)$, it first applies the scope predicate Σ_i on $\mathcal{D}$ to obtain $\Sigma_i(\mathcal{D})$, and then samples $O_i \subseteq \Sigma_i(\mathcal{D})$ with $|O_i| \leq \beta_{\text{lab}}$. It next prompts an online LLM with (a) the semantic condition P_i^S in q_i^S and (b) the tuples in O_i together with their associated unstructured contents in $\mathcal{U}$, to obtain an initial candidate attribute pool, attribute values on tuples in O_i , and pseudo labels for these tuples, as illustrated in Prompt-1.

LLM Prompt-1: Query-Aware Attribute Extraction.

= TASK OVERVIEW =

You are given a semantic query, a sampled set of tuples from its scoped data subset, and the unstructured contents associated with these tuples. Your task is to analyze the semantic condition in the query, inspect the sampled tuples together with their linked unstructured contents, and produce an *initial* query-aware candidate attribute pool for schema enrichment.

= OBJECTIVE =

Infer a small set of *explicit, stable, and human-interpretable* attributes that can be extracted from the unstructured source alone and are potentially useful for evaluating the semantic condition. For each sampled tuple, also return: (1) tentative values of the extracted attributes, and (2) a pseudo label indicating whether the tuple appears to satisfy the semantic condition.

= INPUT SPECIFICATION =

The input consists of the following parts:

- (1) *semantic_condition*: the semantic condition in the query.
- (2) *sampled_tuples*: a sampled subset of tuples from the scoped data relevant to the query.
- (3) *unstructured_contents*: the unstructured contents associated with the sampled tuples.
- (4) *feature_budget*: the maximum number of candidate attributes to propose.

= INPUT EXAMPLE =

semantic_condition:

"The patient appears critically ill based on clinical indicators and CXR."

sampled_tuples:

[{"tuple_id": "t1", "time": "2020-12", "age": 76, "SpO2": 89, "CRP": 140}, {"tuple_id": "t2", "time": "2020-12", "age": 82, "SpO2": 88, "CRP": 160}, {"tuple_id": "t3", "time": "2020-12", "age": 69, "SpO2": 95, "CRP": 18}]

unstructured_contents:

{"t1": "CXR shows diffuse bilateral opacity with pleural effusion.", "t2": "CXR shows extensive bilateral infiltrates and severe opacity.", "t3": "CXR shows mild focal change without bilateral involvement."}

feature_budget: 5

= INSTRUCTIONS (STEP-BY-STEP) =

- Read the semantic condition and determine what evidence in the unstructured content would be relevant to deciding whether a tuple satisfies this condition.
- Examine the sampled tuples together with their associated unstructured contents, and infer pseudo labels indicating whether each tuple appears to satisfy the semantic condition.
- Based on the semantic condition and the sampled tuples, identify a small set of candidate attributes that are: (a) directly extractable from the unstructured source alone, (b) stable

across tuples, (c) useful for distinguishing tuples that likely satisfy the semantic condition from those that do not, and (d) suitable for downstream filtering, ranking, or query rewriting.

- For each candidate attribute, provide: (a) a concise snake_case attribute name, (b) its type, restricted to bool, int, or float, and (c) a precise extraction instruction that can later be used to populate this attribute for a single tuple.
- For each sampled tuple, return the tentative value of every proposed attribute, together with the pseudo label inferred for this tuple.
- Propose no more than the specified feature budget, and avoid free-text, categorical, or high-cardinality string attributes.

= **OUTPUT SPECIFICATION** =
Return a valid JSON object with the following fields:

```
{
  "candidate_features": [
    {
      "target_col": "feature_name",
      "feature_type": "bool | int | float",
      "extraction_prompt": "instruction for extracting this feature from one tuple"
    }
  ],
  "tuple_annotations": [
    {
      "tuple_id": "identifier of a sampled tuple",
      "pseudo_label": <boolean>,
      "feature_values": { "feature_name_1": <value> }
    }
  ]
}
```

= **OUTPUT EXAMPLE** =

```
{
  "candidate_features": [
    {
      "target_col": "opacity",
      "feature_type": "float",
      "extraction_prompt": "Estimate the extent of lung opacity from the CXR as a number between 0 and 1."
    }
  ],
  "tuple_annotations": [
    {
      "tuple_id": "t1",
      "pseudo_label": true,
      "feature_values": { "opacity": 0.78 }
    }
  ]
}
```

Step 2: Refinement with annotated feedback. After the initial extraction, users annotate the tuples in O_i to provide ground-truth labels. It then feeds these corrected labels, together with the candidate attributes and values returned in Step 1, back to the LLM, such that it can refine the candidate attribute pool $\mathcal{R}_u$ by removing or updating unsuitable attributes and refreshing their materialized values on O_i , following Prompt-2. The corrected labels are also used to estimate the selectivity $\hat{\pi}_i = \frac{1}{|O_i|} \sum_{t \in O_i} Y_i(t)$.

LLM Prompt-2: Attribute Refinement with Annotated Feedback.

= **TASK OVERVIEW** =
You are given a semantic query, a sampled set of tuples from its scoped data subset, the unstructured contents associated with these tuples, and a previously proposed candidate attribute pool with tentative attribute values and pseudo labels indicating whether the tuple appears to satisfy the semantic condition. Users have now provided corrected labels for the sampled tuples. Your task is to refine the previously proposed attributes using this annotated feedback, so that the retained attributes better align with the semantic condition and the labeled evidence.

= **OBJECTIVE** =
Update the previously proposed candidate attribute pool by removing, revising, or retaining attributes as needed. Use the corrected labels as feedback on the initial extraction results. For each retained attribute, also return refreshed attribute values on the sampled tuples that are consistent with the refined attribute definitions.

= **INPUT SPECIFICATION** =
The input consists of the following parts:
(1) semantic_condition: the semantic condition in the query.
(2) sampled_tuples: the sampled subset of tuples used in the initial extraction step.
(3) unstructured_contents: the unstructured contents associated with the sampled tuples.
(4) initial_candidate_features: the previously proposed candidate attributes.
(5) initial_tuple_annotations: the tentative attribute values and pseudo labels previously returned for the sampled tuples.
(6) corrected_labels: the ground-truth labels provided by users for the sampled tuples.

= **INPUT EXAMPLE** =
semantic_condition:
"The patient appears critically ill based on clinical indicators and CXR."
sampled_tuples:
[{"tuple_id": "t2", "time": "2020-12", "age": 82, "SpO2": 88, "CRP": 160}]
unstructured_contents:
{ "t2": "CXR shows extensive bilateral infiltrates and severe opacity." }
initial_candidate_features:
[{"target_col": "opacity", "feature_type": "float",
 "extraction_prompt": "Estimate the extent of lung opacity from the CXR as a number between 0 and 1."}]

initial_tuple_annotations:
[{"tuple_id": "t2", "pseudo_label": false,
 "feature_values": { "opacity": 0.82 } }]
corrected_labels:
{ "t2": true }
= **INSTRUCTIONS (STEP-BY-STEP)** =

- Read the semantic condition, the previously proposed candidate attributes, the initial tuple annotations, and the corrected labels, and identify which parts of the initial extraction appear consistent or inconsistent with the annotated evidence.
- Compare the initial pseudo labels with the corrected labels, and use the disagreements as feedback for refining the previously proposed attributes.
- Remove attributes that appear irrelevant, unstable, redundant, or poorly aligned with the corrected labels.
- Revise retained attributes when needed, including their names, definitions, or extraction instructions, so that they better capture evidence relevant to the semantic condition.
- Refresh the attribute values of the retained attributes on the sampled tuples using the refined definitions.
- Keep the refined attribute set compact, interpretable, and directly extractable from the unstructured source alone. Avoid free-text, categorical, or high-cardinality string attributes.

= **OUTPUT SPECIFICATION** =
Return a valid JSON object with the following fields:

```
{
  "refined_candidate_features": [
    {
      "target_col": "feature_name",
      "feature_type": "bool | int | float",
      "extraction_prompt": "instruction for extracting this feature from one tuple"
    }
  ],
  "refined_tuple_annotations": [
    {
      "tuple_id": "identifier of a sampled tuple",
      "feature_values": { "feature_name_1": <value> }
    }
  ]
}
```

= **OUTPUT EXAMPLE** =

```
{
  "refined_candidate_features": [
    {
      "target_col": "opacity",
      "feature_type": "float",
      "extraction_prompt": "Estimate the extent of lung opacity from the CXR as a number between 0 and 1."
    }
  ],
  "refined_tuple_annotations": [
    {
      "tuple_id": "t2",
      "feature_values": { "opacity": 0.82 }
    }
  ]
}
```

Coreset construction for supervision. It proceeds as follows.

Step 1: Semantic-attribute population. To materialize candidate attributes in $\mathcal{R}_u$ for tuples in $\Sigma_i(\mathcal{D}) \setminus O_i$, SEQR prompts low-cost locally deployed models (e.g., Llama3-8B) with tuples in O_i as in-context examples, as illustrated in Prompt-3. When multiple local models are used, SEQR accepts the majority output when they reach consensus; otherwise, it falls back to a stronger cloud-hosted LLM.

LLM Prompt-3: Attribute Population with In-Context Examples.

= **TASK OVERVIEW** =
You are given a set of attributes with extraction instructions, a set of example tuples with associated unstructured contents and known attribute values, and a set of new tuples whose attribute values need to be populated. Your task is to infer the values of all target attributes for each new tuple by following the extraction instructions and using the provided examples as guidance.

= **OBJECTIVE** =
Populate the values of a set of candidate attributes for a set of target tuples. The returned values must follow the required attribute types and be jointly consistent with the extraction instructions, the in-context examples, and the unstructured contents of the target tuples.

= **INPUT SPECIFICATION** =
The input consists of the following parts:
(1) target_attributes: the set of attributes to be populated, where each attribute includes its name, type, and extraction instruction.
(2) in_context_examples: a small set of example tuples, each with its unstructured content and the known values of the target attributes.
(3) target_tuples: the tuples whose attribute values need to be populated.
(4) target_contents: the unstructured contents associated with the target tuples.

= **INPUT EXAMPLE** =
target_attributes:
[{"target_col": "opacity", "feature_type": "float",
 "extraction_prompt": "Estimate the extent of lung opacity from the CXR as a number between 0 and 1."},
 {"target_col": "bilateral", "feature_type": "bool",
 "extraction_prompt": "Does the CXR show bilateral pulmonary involvement?"}]

```

Respond with True or False.",
{"target_col": "effusion", "feature_type": "bool",
 "extraction_prompt": "Does the CXR show pleural effusion? Respond with True
or False."}]
in_context_examples:
[{"tuple_id": "t1",
 "content": "CXR shows diffuse bilateral opacity with pleural effusion.",
 "feature_values": {"opacity": 0.78, "bilateral": true, "effusion": true}},
 {"tuple_id": "t3",
 "content": "CXR shows mild focal change without bilateral involvement.",
 "feature_values": {"opacity": 0.12, "bilateral": false, "effusion": false}}]
target_tuples:
[{"tuple_id": "t4", "time": "2021-01", "age": 70, "SpO2": 90, "CRP": 120}]
target_contents:
{"t4": "CXR shows marked bilateral opacity with pleural effusion."}
= INSTRUCTIONS (STEP-BY-STEP) =
o Read all target attributes, their types, and their extraction instructions carefully.
o Examine the in-context examples and use them to understand how the target attributes should
be inferred from unstructured content.
o For each target tuple, inspect its associated unstructured content and infer the values of all
target attributes by applying the same extraction logic jointly.
o Ensure that each returned value matches the required attribute type: return true or false
for bool, an integer for int, and a numeric score for float.
o Return only the populated values for the requested attributes of each target tuple. Do not
include explanations, comments, or additional text.

= OUTPUT SPECIFICATION =
Return a valid JSON object with the following field:
{
  "tuple_annotations": [
    {"tuple_id": "identifier of a target tuple",
     "feature_values": {"target_col_1": <value>, "target_col_2": <value>}
    }
  ]
}

= OUTPUT EXAMPLE =
{
  "tuple_annotations": [
    {"tuple_id": "t4",
     "feature_values": {"opacity": 0.70, "bilateral": true, "effusion": true}
    }
  ]
}

```

Step 2: Prediction confidence. Using the populated attributes in $\mathcal{R} \cup \mathcal{R}_U$, SEQR trains a tree-based classifier on O_i . For each unlabeled tuple $t \in \Sigma_i(\mathcal{D}) \setminus O_i$, the classifier outputs (a) a prediction confidence $f_p(t) \in [0, 1]$ and (b) feature-importance scores over attributes in $\mathcal{R}_U$. Here $f_p(t)$ measures how strongly the classifier predicts t to satisfy the query semantics under the current supervision.

Step 3: Structural confidence. For each unlabeled tuple $t \in \Sigma_i(\mathcal{D}) \setminus O_i$, SEQR computes a structural confidence $f_s(t) \in [0, 1]$ by comparing t with labeled tuples in O_i . It first computes the feature-importance-weighted L2 distance from t to each labeled positive and negative tuple in O_i , and then averages these distances within each class to obtain $d^+(t)$ and $d^-(t)$, respectively.

The structural confidence is defined as

$$f_s(t) = \frac{d^-(t)}{d^+(t) + d^-(t)}.$$

Thus, $f_s(t)$ is larger (resp. smaller) when t is structurally closer to labeled positives (resp. negatives).

Step 4: Selectivity-aware coreset construction. Given the prediction confidence $f_p(t)$ and structural confidence $f_s(t)$, SEQR combines them into a selectivity-aware score

$$s(t) = \hat{\pi}_i f_p(t) + (1 - \hat{\pi}_i) f_s(t),$$

where $\hat{\pi}_i$ is the estimated selectivity from O_i . It then ranks tuples in $\Sigma_i(\mathcal{D}) \setminus O_i$ by $s(t)$ in descending order and constructs C_i as follows: it (a) fixes the coreset size under a budget chosen with respect to the data scale and labeling budget; (b) selects pseudo positives with the highest scores so that the positive fraction in C_i matches $\hat{\pi}_i$; and (c) selects pseudo negatives with the lowest scores to fill C_i .

Joint evaluation in schema enrichment and rewriting. It consists of the following steps.

Step 1: Candidate attribute ranking and construction. After constructing C_i , SEQR refits the tree-based model on C_i and computes the feature-importance score $w_i(A)$ of each attribute $A \in \mathcal{R}_U$. It aggregates these query-specific scores into a global score

$$w(A) = \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} w_i(A),$$

which ranks attributes in $\mathcal{R}_U$ by $w(A)$, and constructs candidate attribute sets $\mathcal{R}_k$ from the top- k ranked attributes for $k \in [1, \beta_{SE}]$.

Step 2: Label propagation. For each candidate attribute set $\mathcal{R}_k$, it constructs the enriched schema $\mathcal{R}^E = \mathcal{R} \cup \mathcal{R}_k$ and materializes the selected attributes in $\mathcal{R}_k$ for scope-filtered tuples in $\Sigma_i(\mathcal{D})$, following the same semantic-attribute population procedure as in Step 1 of coreset construction, to obtain the corresponding enriched data $\mathcal{D}^E$. For each query q_i^S , it then trains a tree-based model on the coreset C_i under $\mathcal{R}^E$, and applies the trained model to tuples in the scoped enriched set $\Sigma_i(\mathcal{D}^E) \setminus C_i$. The predicted labels are used as propagated labels for these tuples, while the labels of tuples in C_i remain fixed. Thus, each tuple in $\Sigma_i(\mathcal{D}^E)$ has a user, pseudo, or propagated label; only the propagated labels outside C_i may change across candidate schemas.

Step 3: Subjective score computation. For each query q_i^S , SEQR evaluates label-propagation reliability under the candidate schema by computing a leave-one-out (LOO) error on O_i and adding a stability-aware finite-sample penalty.

LOO error. For each tuple $t \in O_i$, performing label propagation with $O_i \setminus \{t\}$ as the seed set yields the LOO prediction $\hat{Y}_i^{\text{LOO}}(t)$. Given the ground-truth label $Y_i(t)$ of t , the LOO error is:

$$\hat{\mathcal{L}}_{\text{LOO}}^{(i)} = \frac{1}{|O_i|} \sum_{t \in O_i} \ell_{\text{prop}}(Y_i(t), \hat{Y}_i^{\text{LOO}}(t)).$$

Here $\ell_{\text{prop}}(\cdot, \cdot)$ is a class-imbalance-aware 0-1 error, defined as:

$$\ell_{\text{prop}}(Y_i(t), \hat{Y}_i^{\text{LOO}}(t)) = \begin{cases} \frac{\max\{\hat{\pi}_i, 1 - \hat{\pi}_i\}}{\hat{\pi}_i}, & Y_i(t) = 1, \hat{Y}_i^{\text{LOO}}(t) = 0, \\ \frac{\max\{\hat{\pi}_i, 1 - \hat{\pi}_i\}}{1 - \hat{\pi}_i}, & Y_i(t) = 0, \hat{Y}_i^{\text{LOO}}(t) = 1, \\ 0, & Y_i(t) = \hat{Y}_i^{\text{LOO}}(t), \end{cases}$$

where $\hat{\pi}_i \in (0, 1)$ is the estimated semantic selectivity from O_i . The error weights mistakes on the minority class more heavily, such that propagation quality is assessed under class imbalance.

Propagation penalty. With $n_i = |O_i|$ and $\hat{\pi}_i \in (0, 1)$, the propagation loss is at most $n_i - 1$. For any candidate index, the expected difference between its population propagation error and LOO score is therefore at most $n_i - 1$, and replacing one tuple changes this difference by at most $2(n_i - 1)$. Under the *i.i.d.* sampling condition of Lemma 2, McDiarmid's inequality and a union bound over queries and candidate indices give

$$\rho_{\text{prop}}^{(i)} = (n_i - 1) + 2(n_i - 1) \sqrt{\frac{n_i}{2} \ln(\beta_{SE} |Q^S| / \delta)}.$$

With probability at least $1 - \delta$, every candidate's population error is at most its LOO score plus this penalty. The penalty is shared across candidates and hence does not change their ordering.

Thus, the subjective score is

$$\hat{\mathcal{L}}_{\text{subj}}^{(i)} = \hat{\mathcal{L}}_{\text{LOO}}^{(i)} + \rho_{\text{prop}}^{(i)}.$$

Step 4: Rewrite construction. Given a trained tree-based model under candidate schema $\mathcal{R} \cup \mathcal{R}_k$, SEQR extracts positive decision paths, converts each path into a conjunctive predicate over attributes in $\mathcal{R} \cup \mathcal{R}_k$, and ranks these predicates by support. It forms the rewrite as a disjunction of at most β_{rew} highest-ranked predicates. Here, the support of a positive path is the fraction of predicted positive tuples in $\Sigma_i(\mathcal{D}^E)$ that pass this path. Each selected path, therefore, contributes one conjunctive component, and the final rewrite is a union of the selected conjunctive predicates.

Step 5: Objective score computation. For each query q_i^S , SEQR evaluates the rewritten query against the user, pseudo, and propagated labels in $\Sigma_i(\mathcal{D}^E)$. It computes the empirical rewriting error and adds a complexity-aware penalty.

Empirical rewriting error. Using a cost-sensitive rewriting error $\ell_{\text{rew}}(\cdot)$, the empirical rewriting error is defined as:

$$\widehat{\mathcal{L}}_{\text{rew}}^{(i)} = \frac{1}{|\Sigma_i(\mathcal{D}^E)|} \sum_{t \in \Sigma_i(\mathcal{D}^E)} \ell_{\text{rew}}(\widehat{Y}_i(t), q_i^T(t)),$$

where $\Sigma_i(\mathcal{D}^E)$ is the tuple subset satisfying the scope predicates of q_i^S , $\widehat{Y}_i(t) \in \{0, 1\}$ is the label associated with t after label propagation, and $q_i^T(t) \in \{0, 1\}$ is the label returned by executing q_i^T on $\mathcal{D}^E$.

Cost-sensitive rewriting error. Define the positive-label fraction in $\Sigma_i(\mathcal{D}^E)$ after label propagation as:

$$\widehat{\pi}_i^E = \frac{1}{|\Sigma_i(\mathcal{D}^E)|} \sum_{t \in \Sigma_i(\mathcal{D}^E)} \widehat{Y}_i(t)$$

We define the cost-sensitive rewriting error $\ell_{\text{rew}}(\cdot)$ as:

$$\ell_{\text{rew}}(\widehat{Y}_i(t), q_i^T(t)) = \begin{cases} \frac{\max\{\widehat{\pi}_i^E, 1 - \widehat{\pi}_i^E\}}{\widehat{\pi}_i^E}, & \widehat{Y}_i(t) = 1, q_i^T(t) = 0, \\ \frac{\max\{\widehat{\pi}_i^E, 1 - \widehat{\pi}_i^E\}}{1 - \widehat{\pi}_i^E}, & \widehat{Y}_i(t) = 0, q_i^T(t) = 1, \\ 0, & \widehat{Y}_i(t) = q_i^T(t). \end{cases}$$

When $\widehat{\pi}_i^E$ is small or close to 1, the error amplifies mistakes on the rare class, so that rewrite fidelity is assessed under imbalance.

Rewrite penalty. We use a complexity-aware regularizer to discourage a rewrite that fits the propagated labels only through a high-complexity query class:

$$\rho_{\text{rew}}^{(i)} = \Gamma_{\text{rew}}^{(i)} \sqrt{\frac{d_{\text{VC}}^{(i)} \ln(|\Sigma_i(\mathcal{D}^E)|) + \ln(2|Q^S|/\delta)}{|\Sigma_i(\mathcal{D}^E)|}},$$

where $\delta \in (0, 1)$ controls the regularization strength, $\Gamma_{\text{rew}}^{(i)} = \frac{\max\{\widehat{\pi}_i^E, 1 - \widehat{\pi}_i^E\}}{\min\{\widehat{\pi}_i^E, 1 - \widehat{\pi}_i^E\}}$, and $d_{\text{VC}}^{(i)}$ is a complexity bound for the rewrite class. Since each q_i^T is a union of at most β_{rew} conjunctive queries over $\mathcal{R}^E$, standard bounds [2, 6, 30] give $d_{\text{VC}}^{(i)} = O(\beta_{\text{rew}} \cdot \text{arity}(\mathcal{R}^E) \cdot \log(\beta_{\text{rew}} + 1))$; we retain $d_{\text{VC}}^{(i)}$ as the corresponding class-complexity term in $\rho_{\text{rew}}^{(i)}$. The term affects candidate selection only; the final rewrite is certified independently in Theorem 3.

Thus, the objective score is

$$\widehat{\mathcal{L}}_{\text{obj}}^{(i)} = \widehat{\mathcal{L}}_{\text{rew}}^{(i)} + \rho_{\text{rew}}^{(i)}.$$

Step 6: Candidate selection. For each candidate attribute set $\mathcal{R}_k$, SEQR combines its subjective and objective scores as

$$\mathcal{L}_{\text{static}}^{(i)} = \widehat{\mathcal{L}}_{\text{sub}}^{(i)} + \widehat{\mathcal{L}}_{\text{obj}}^{(i)},$$

and aggregates them across queries by

$$\frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} \mathcal{L}_{\text{static}}^{(i)}.$$

It repeats Steps 2–5 for all candidate sets $\mathcal{R}_k$ with $k \in [1, \beta_{\text{SE}}]$, and returns the candidate set whose aggregated score is minimum as the final enriched schema configuration.

C PROOF OF LEMMA 2

Lemma 2: Let $n_i = |O_i| \geq 2$, where O_i is drawn i.i.d. from $\Sigma_i(\mathcal{D})$, and let $\mathcal{L}_{\text{prop}, n_i-1}^{(i)}$ denote the expected propagation error of a deterministic propagation algorithm trained on $n_i - 1$ i.i.d. labeled tuples and evaluated on an independent tuple under a fixed bounded loss. Then,

$$\mathbb{E}[\widehat{\mathcal{L}}_{\text{LOO}}^{(i)}] = \mathcal{L}_{\text{prop}, n_i-1}^{(i)},$$

where the expectation $\mathbb{E}$ is over O_i and the held-out tuple. $\square$

Proof: Index the tuples in O_i as $t_1, \dots, t_{n_i}$, where each $t_j \in \Sigma_i(\mathcal{D})$ has ground-truth label $Y_i(t_j)$. For each j , removing t_j from O_i gives the labeled sample $O_i^{-j} = O_i \setminus \{t_j\}$. Training the deterministic propagation algorithm on O_i^{-j} and evaluating it at t_j yields the LOO prediction label $\widehat{Y}_i^{\text{LOO}}(t_j)$. The LOO error is

$$\widehat{\mathcal{L}}_{\text{LOO}}^{(i)} = \frac{1}{n_i} \sum_{j=1}^{n_i} \ell_{\text{prop}}(Y_i(t_j), \widehat{Y}_i^{\text{LOO}}(t_j)).$$

For every j , O_i^{-j} consists of $n_i - 1$ i.i.d. labeled tuples and is independent of the held-out labeled tuple $(t_j, Y_i(t_j))$. Hence, $\widehat{Y}_i^{\text{LOO}}(t_j)$ has the same distribution as the prediction obtained by training the algorithm on $n_i - 1$ independent draws and evaluating it on an independent tuple. Under the fixed loss, exchangeability shows that

$$\mathbb{E}[\ell_{\text{prop}}(Y_i(t_j), \widehat{Y}_i^{\text{LOO}}(t_j))] = \mathcal{L}_{\text{prop}, n_i-1}^{(i)}.$$

Linearity of expectation proves the claimed equality:

$$\mathbb{E}[\widehat{\mathcal{L}}_{\text{LOO}}^{(i)}] = \frac{1}{n_i} \sum_{j=1}^{n_i} \mathcal{L}_{\text{prop}, n_i-1}^{(i)} = \mathcal{L}_{\text{prop}, n_i-1}^{(i)}, \quad \square$$

D PROOF OF THEOREM 3

Theorem 3: Given a semantic query set Q^S , enriched data $\mathcal{D}^E$ under schema $\mathcal{R}^E$, the rewritten SQL workload Q^T produced by SEQR, and a confidence parameter $\delta \in (0, 1)$, the certification procedure provides, for each q_i^S , an auxiliary rewrite $\widehat{q}_i^T$, a full-scope 0–1 error bound U_i for $\widehat{q}_i^T$ with confidence $1 - \delta/|Q^S|$, and the exact fraction d_i of tuples in $\Sigma_i(\mathcal{D})$ on which $\widehat{q}_i^T$ and the final rewrite q_i^T differ. Define $B_i = \min\{1, U_i + d_i\}$. The results $\mathcal{E}$ of executing Q^T over $\mathcal{D}^E$ satisfy, with probability at least $1 - \delta$,

$$\text{acc}(Q^S, \mathcal{E}) \geq \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} \Phi(\pi_i, B_i),$$

$$\Phi(\pi_i, B_i) = \begin{cases} \frac{2(\pi_i - B_i)}{2\pi_i - B_i}, & 0 \leq B_i < \pi_i, \\ 0, & B_i \geq \pi_i, \end{cases}$$

where $\pi_i = \frac{|G_i|}{|\Sigma_i(\mathcal{D})|}$ is the true selectivity of q_i^S , and $\Phi(\pi_i, B_i)$ lower-bounds the per-query F_1 of q_i^T . $\square$

Proof: We first certify the full-scope 0–1 error of each rewrite and then derive the workload-level F_1 guarantee.

(S1) Certifying the auxiliary rewrite. For each query $q_i^S \in Q^S$ with $|\Sigma_i(\mathcal{D})| > 0$, the certification process obtains the initial labeled

set T_i and the auxiliary rewrite $\tilde{q}_i^T$ compiled from it. The certification set V_i is sampled only after $\tilde{q}_i^T$ is fixed. Let $M_i = |\Sigma_i(\mathcal{D}) \setminus T_i|$ and $h_i = |V_i|$, where $1 \leq h_i \leq M_i$. Let e_i^{init} , K_i , and X_i denote the numbers of errors of $\tilde{q}_i^T$ on T_i , $\Sigma_i(\mathcal{D}) \setminus T_i$, and V_i , respectively:

$$\begin{aligned} e_i^{\text{init}} &= \sum_{t \in T_i} \mathbb{1}[\tilde{q}_i^T(t) \neq Y_i(t)], \\ K_i &= \sum_{t \in \Sigma_i(\mathcal{D}) \setminus T_i} \mathbb{1}[\tilde{q}_i^T(t) \neq Y_i(t)], \\ X_i &= \sum_{t \in V_i} \mathbb{1}[\tilde{q}_i^T(t) \neq Y_i(t)]. \end{aligned}$$

Since V_i is drawn uniformly without replacement from the M_i remaining tuples, $X_i \mid K_i \sim \text{Hypergeom}(M_i, K_i, h_i)$. Let x_i be the observed value of X_i , and set $\alpha_i = \delta/|Q^S|$. For each candidate error count $k \in \{0, \dots, M_i\}$, denote by

$$F_{i,k}(x) = \Pr_{X \sim \text{Hypergeom}(M_i, k, h_i)}[X \leq x].$$

Using the exact construction in [63], the $(1 - \alpha_i)$ upper confidence limit for K_i is given by

$$K_i^+ = \max\{k \in \{0, \dots, M_i\} : F_{i,k}(x_i) > \alpha_i\}.$$

The set is nonempty because $F_{i,0}(x_i) = 1$. Also, $F_{i,k}(x_i)$ is non-increasing in k , since changing a non-error into an error cannot reduce a sample's error count. Thus, $K_i > K_i^+$ implies $F_{i,K_i}(x_i) \leq \alpha_i$. Under the true K_i , the lower tail $\{x : F_{i,K_i}(x) \leq \alpha_i\}$ has probability at most α_i . Hence,

$$\Pr[K_i \leq K_i^+ \mid T_i, \tilde{q}_i^T] \geq 1 - \alpha_i.$$

The full-scope error count of $\tilde{q}_i^T$ is $e_i^{\text{init}} + K_i$. Therefore, letting $U_i = (e_i^{\text{init}} + K_i^+)/|\Sigma_i(\mathcal{D})|$ yields

$$\Pr\left[\frac{1}{|\Sigma_i(\mathcal{D})|} \sum_{t \in \Sigma_i(\mathcal{D})} \mathbb{1}[\tilde{q}_i^T(t) \neq Y_i(t)] \leq U_i\right] \geq 1 - \alpha_i.$$

Since the argument holds for every fixed T_i and $\tilde{q}_i^T$, the same bound holds unconditionally.

(S2) Transferring the certificate to the final rewrite. By definition, d_i is the full-scope disagreement rate between $\tilde{q}_i^T$ and q_i^T :

$$d_i = \frac{1}{|\Sigma_i(\mathcal{D})|} \sum_{t \in \Sigma_i(\mathcal{D})} \mathbb{1}[\tilde{q}_i^T(t) \neq q_i^T(t)].$$

LiteDBX computes d_i exactly by executing both rewrites over $\Sigma_i(\mathcal{D})$; if their enriched schemas differ, it aligns their outputs by original tuple identifier. For each $t \in \Sigma_i(\mathcal{D})$, an error by q_i^T implies that either $\tilde{q}_i^T$ is wrong or the two rewrites disagree. Hence,

$$\mathbb{1}[q_i^T(t) \neq Y_i(t)] \leq \mathbb{1}[\tilde{q}_i^T(t) \neq Y_i(t)] + \mathbb{1}[\tilde{q}_i^T(t) \neq q_i^T(t)].$$

Averaging over $\Sigma_i(\mathcal{D})$ bounds the error of q_i^T by the error of $\tilde{q}_i^T$ plus d_i . From (S1), with probability at least $1 - \alpha_i$,

$$\text{Err}^{(i)} = \frac{1}{|\Sigma_i(\mathcal{D})|} \sum_{t \in \Sigma_i(\mathcal{D})} \mathbb{1}[q_i^T(t) \neq Y_i(t)] \leq U_i + d_i.$$

Since a 0–1 error rate cannot exceed one, this establishes $\text{Err}^{(i)} \leq B_i = \min\{1, U_i + d_i\}$ on the same event.

(S3) Converting error to F_1 . If $\pi_i = 0$, the lower bound in the theorem is zero. Suppose that $\pi_i > 0$. Denote by TP_i , FP_i , and FN_i the true-positive, false-positive, and false-negative rates of q_i^T over $\Sigma_i(\mathcal{D})$. By the definitions of selectivity and 0–1 error,

$$\pi_i = \text{TP}_i + \text{FN}_i, \quad \text{Err}^{(i)} = \text{FP}_i + \text{FN}_i.$$

Thus, $\text{TP}_i = \pi_i - \text{FN}_i$, $\text{FP}_i = \text{Err}^{(i)} - \text{FN}_i$, and $0 \leq \text{FN}_i \leq \min\{\text{Err}^{(i)}, \pi_i\}$. Substituting these equalities into the F_1 score gives

$$F_1(E_i, G_i) = \frac{2(\pi_i - \text{FN}_i)}{2(\pi_i - \text{FN}_i) + \text{Err}^{(i)}}.$$

For fixed π_i and $\text{Err}^{(i)}$, the derivative with respect to FN_i is

$$\frac{\partial F_1(E_i, G_i)}{\partial \text{FN}_i} = -\frac{2\text{Err}^{(i)}}{(2(\pi_i - \text{FN}_i) + \text{Err}^{(i)})^2} \leq 0.$$

Hence, the smallest F_1 occurs at $\text{FN}_i = \min\{\text{Err}^{(i)}, \pi_i\}$. If $\text{Err}^{(i)} \geq \pi_i$, this choice gives $\text{TP}_i = 0$ and the lower bound is zero. If $\text{Err}^{(i)} < \pi_i$, the same choice gives

$$F_1(E_i, G_i) \geq \frac{2(\pi_i - \text{Err}^{(i)})}{2\pi_i - \text{Err}^{(i)}}.$$

Combining the two cases yields $F_1(E_i, G_i) \geq \Phi(\pi_i, \text{Err}^{(i)})$. Moreover, $\Phi(\pi_i, e)$ is nonincreasing in e : its derivative is $-2\pi_i/(2\pi_i - e)^2$ for $0 \leq e < \pi_i$, while $\Phi(\pi_i, e) = 0$ for $e \geq \pi_i$. From (S2), $\text{Err}^{(i)} \leq B_i$ with probability at least $1 - \alpha_i$. Hence, with the same probability,

$$F_1(E_i, G_i) \geq \Phi(\pi_i, \text{Err}^{(i)}) \geq \Phi(\pi_i, B_i).$$

(S4) Extending to the workload. By definition, the macro-averaged F_1 satisfies

$$\text{acc}(Q^S, \mathcal{E}) = \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} F_1(E_i, G_i).$$

For each query, the event in (S1) fails with probability at most $\alpha_i = \delta/|Q^S|$, while (S2) and (S3) follow deterministically from this event. Applying a union bound over all queries yields, with probability at least $1 - \delta$,

$$\text{acc}(Q^S, \mathcal{E}) \geq \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} \Phi(\pi_i, B_i).$$

This proves the theorem. $\square$

E DETAILS FOR CERTIFIED REUSE

This section gives additional details for certified reuse in Section 6.

Step 1: Updating enriched data. Given the enriched schema $\mathcal{R}^E$ and data $\mathcal{D}^E$ produced by SEQR, it first applies incoming updates to the underlying data, including tuple insertions, deletions, and value updates (treated as delete–insert operations). It then applies the query-scope predicates Σ_i of each query $q_i^S \in Q^S$ to the updated data, materializes the required attributes in $\mathcal{R}^E$ for the resulting newly introduced tuples using the same semantic-attribute population procedure as in Section B, and obtains the updated enriched data $\mathcal{D}_{\text{new}}^E$.

Step 2: Updating coresets. For each query q_i^S , LiteDBX first removes from C_i tuples that are absent from $\Sigma_i(\mathcal{D}_{\text{new}}^E)$. For each newly introduced tuple, it computes $s(t)$ as in Section B. It adds t as a pseudo positive if its score exceeds the lowest selected pseudo-positive score, and as a pseudo negative if its score is below the highest selected pseudo-negative score recorded when C_i was built. Under the original coreset-size budget, the resulting C_i^{new} retains the most informative surviving and new tuples selected by these thresholds.

Step 3: Re-propagating labels. Using C_i^{new} , LiteDBX repeats the label-propagation procedure in Section B under the fixed schema $\mathcal{R}^E$. It trains a tree-based model on the updated coreset and applies it to $\Sigma_i(\mathcal{D}_{\text{new}}^E) \setminus C_i^{\text{new}}$. The resulting supervision state is retained for selective refresh if the reuse test fails.

Step 4: Computing incremental update term. Given $q_i^T(t; \mathcal{D}^E)$ and $q_i^T(t; \mathcal{D}_{\text{new}}^E)$ of the current rewrite on the two data versions, let $\Omega_i = \Sigma_i(\mathcal{D}^E)$ and $\Omega_i^{\text{new}} = \Sigma_i(\mathcal{D}_{\text{new}}^E)$. LiteDBX computes

$$\Delta^{(i)} = \frac{|\Omega_i^{\text{new}} \setminus \Omega_i| + \sum_{t \in \Omega_i \cap \Omega_i^{\text{new}}} \mathbb{1}[q_i^T(t; \mathcal{D}^E) \neq q_i^T(t; \mathcal{D}_{\text{new}}^E)]}{|\Omega_i^{\text{new}}|}.$$

The numerator assigns unit error to newly introduced tuples and counts the shared tuples whose rewrite output changes; the denominator normalizes these contributions by the updated query scope.

Step 5: Reuse test. The user-specified tolerance on the updated 0–1 error is $\eta \in [0, 1]$. Under the conditions of Theorem 4, LiteDBX reuses the current configuration if $B_i^{\text{new}} \leq \eta$ for every affected query; otherwise, it triggers selective refresh.

Workload-evolution reuse and refresh. For workload evolution, LiteDBX tests whether the current enriched configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ can answer a new/revised semantic predicate. Predicate units define the checking granularity and localize selective refresh.

Step 1: Predicate decomposition. For a newly arrived query q_i^S , a local LLM decomposes its semantic predicate into predicate units. For semantic drift, the revision signal comes from updated criteria, user feedback, or validation evidence for an existing predicate; the revised predicate is decomposed in the same way.

Step 2: Semantic alignment. Given predicate units, the local LLM aligns each unit with supporting attributes in $\mathcal{R}^E$ and rewrite fragments in Q^T . Matched units are marked covered, retaining links to their attributes and SQL predicates; unmatched units are uncovered.

Step 3: Rewrite consistency checking. For each covered unit, LiteDBX traces it to the attributes and SQL conditions used by the current rewrite. If the new or revised predicate requires a missing, changed, or conflicting condition, the unit is marked inconsistent; otherwise, it is marked consistent.

Step 4: Reuse decision. For each query, reuse requires every predicate unit to be covered and consistent. If the test fails, LiteDBX forwards the uncovered or inconsistent units to selective refresh.

Step 5: Selective refresh. LiteDBX constructs the affected query set from newly arrived queries and existing queries that fail the reuse test. It uses the failed predicate units to derive or revise candidate attributes, re-evaluates feature importance under β_{SE} , materializes required attributes over affected scopes, and refreshes only the corresponding rewrites. A full SEQR rerun is used only when bounded refresh cannot preserve the required accuracy.

F PROOF OF THEOREM 4

Theorem 4: Given the current configuration $(\mathcal{R}^E, \mathcal{D}^E, Q^T)$ and updated data $\mathcal{D}_{\text{new}}^E$, denote the new certificate for each $q_i^S \in Q^S$ by

$$B_i^{\text{new}} = \min \left\{ 1, \frac{|\Sigma_i(\mathcal{D}^E)|}{|\Sigma_i(\mathcal{D}_{\text{new}}^E)|} B_i + \Delta^{(i)} \right\},$$

where B_i is the pre-update full-scope 0–1 error certificate for q_i^T from Theorem 3. For any $\delta \in (0, 1)$, the results $\mathcal{E}_{\text{new}}$ of executing Q^T on $\mathcal{D}_{\text{new}}^E$ satisfy, with probability at least $1 - \delta$,

$$\text{acc}(Q^S, \mathcal{E}_{\text{new}}) \geq \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} \Phi(\pi_i^{\text{new}}, B_i^{\text{new}}).$$

Here Φ is the F_1 lower-bound function from Theorem 3, and π_i^{new} is the true selectivity of q_i^S on $\Sigma_i(\mathcal{D}_{\text{new}}^E)$. $\square$

Proof: We first transfer the pre-update 0–1 error certificate to each updated query scope and then convert the resulting bounds into the claimed workload F_1 guarantee.

(S1) Conditioning on the pre-update certificate. For each $q_j^S \in Q^S$, let $\text{Err}^{(j)}$ be the pre-update full-scope 0–1 error of q_j^T . By Theorem 3, the simultaneous event

$$\mathcal{A} = \{\text{Err}^{(j)} \leq B_j \text{ for every } q_j^S \in Q^S\}$$

has probability at least $1 - \delta$. We therefore condition on $\mathcal{A}$.

(S2) Bounding the updated 0–1 error. For each q_i^S , let $\Omega_i = \Sigma_i(\mathcal{D}^E)$ and $\Omega_i^{\text{new}} = \Sigma_i(\mathcal{D}_{\text{new}}^E)$. Denote the updated ground-truth label by $Y_i^{\text{new}}(t)$. The pre- and post-update errors are

$$\text{Err}^{(i)} = \frac{1}{|\Omega_i|} \sum_{t \in \Omega_i} \mathbb{1}[q_i^T(t; \mathcal{D}^E) \neq Y_i(t)],$$

$$\text{Err}_{\text{new}}^{(i)} = \frac{1}{|\Omega_i^{\text{new}}|} \sum_{t \in \Omega_i^{\text{new}}} \mathbb{1}[q_i^T(t; \mathcal{D}_{\text{new}}^E) \neq Y_i^{\text{new}}(t)].$$

Enrichment preserves tuple identifiers, so Ω_i contains the same tuples as $\Sigma_i(\mathcal{D})$; hence, (S1) certifies the first quantity above.

As the workload is fixed and value updates are treated as delete–insert operations, a tuple shared by the two scopes retains its ground-truth label. For every $t \in \Omega_i \cap \Omega_i^{\text{new}}$, $Y_i^{\text{new}}(t) = Y_i(t)$, and the pointwise triangle inequality gives

$$\begin{aligned} \mathbb{1}[q_i^T(t; \mathcal{D}_{\text{new}}^E) \neq Y_i^{\text{new}}(t)] &\leq \mathbb{1}[q_i^T(t; \mathcal{D}^E) \neq Y_i(t)] \\ &\quad + \mathbb{1}[q_i^T(t; \mathcal{D}_{\text{new}}^E) \neq q_i^T(t; \mathcal{D}^E)]. \end{aligned}$$

Every tuple in $\Omega_i^{\text{new}} \setminus \Omega_i$ contributes at most one error. Decomposing the updated scope into new and shared tuples, and then summing the pointwise bound over the shared tuples, gives

$$\begin{aligned} |\Omega_i^{\text{new}}| \text{Err}_{\text{new}}^{(i)} &\leq |\Omega_i^{\text{new}} \setminus \Omega_i| \\ &\quad + \sum_{t \in \Omega_i \cap \Omega_i^{\text{new}}} \mathbb{1}[q_i^T(t; \mathcal{D}^E) \neq Y_i(t)] \\ &\quad + \sum_{t \in \Omega_i \cap \Omega_i^{\text{new}}} \mathbb{1}[q_i^T(t; \mathcal{D}_{\text{new}}^E) \neq q_i^T(t; \mathcal{D}^E)] \\ &\leq |\Omega_i| \text{Err}^{(i)} + |\Omega_i^{\text{new}}| \Delta^{(i)}. \end{aligned}$$

The last inequality extends the old-error sum from the shared tuples to all of Ω_i . The other two terms are exactly $|\Omega_i^{\text{new}}| \Delta^{(i)}$ by the definition of $\Delta^{(i)}$. Dividing by $|\Omega_i^{\text{new}}|$ therefore yields

$$\text{Err}_{\text{new}}^{(i)} \leq \frac{|\Omega_i|}{|\Omega_i^{\text{new}}|} \text{Err}^{(i)} + \Delta^{(i)}.$$

On $\mathcal{A}$, $\text{Err}^{(i)} \leq B_i$. Hence,

$$\text{Err}_{\text{new}}^{(i)} \leq \frac{|\Omega_i|}{|\Omega_i^{\text{new}}|} B_i + \Delta^{(i)}.$$

Since a 0–1 error rate cannot exceed one, the definition of B_i^{new} gives $\text{Err}_{\text{new}}^{(i)} \leq B_i^{\text{new}}$.

(S3) Extending to the workload. Let $F_{1, \text{new}}^{(i)}$ be the updated per-query F_1 . The error-to- F_1 conversion in step (S3) of Theorem 3 is deterministic. Applying it to the updated scope gives, on $\mathcal{A}$,

$$F_{1, \text{new}}^{(i)} \geq \Phi(\pi_i^{\text{new}}, B_i^{\text{new}}).$$

By the definition of macro-averaged F_1 ,

Datasets	$ Q^S $	Human Time (s)	Human \$	LLM Calls	LLM Time/\$
Movie	2	800	2.67	6	81.23/0.0114
Wildlife	1	400	1.33	3	45.03/0.0057
E-Commerce	3	1200	4.00	9	125.71/0.0206
MMQA	5	2000	6.67	15	170.00/0.0221
Medical	4	1600	5.34	12	574.32/0.0803

Table 6: Per-dataset supervision accounting.

$$\begin{aligned}
\text{acc}(Q^S, \mathcal{E}_{\text{new}}) &= \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} F_{1,\text{new}}^{(i)} \\
&\geq \frac{1}{|Q^S|} \sum_{q_i^S \in Q^S} \Phi(\pi_i^{\text{new}}, B_i^{\text{new}}).
\end{aligned}$$

This inequality holds on $\mathcal{A}$, whose probability is at least $1 - \delta$. This proves the theorem. $\square$

G MORE EXPERIMENTAL RESULTS

We give the complete experimental protocols and additional results for evaluating LiteDBX. We first describe the supervision and workload construction, and then report the results for compilation accuracy, maintenance under evolution, efficiency, and applicability.

Additional settings. We present the workload-construction and supervision protocols used in the experiments below.

Semantic workloads. For data evolution, we keep the queries fixed and expand the initial 60% release with two disjoint 20% batches, yielding 60% $\rightarrow$ 80% $\rightarrow$ 100% for each dataset.

For query evolution, we transform each original query through paraphrasing, new combinations of known units, unseen units, or revised semantic criteria, preserving its relational skeleton unless explicitly changed. For each trace, we use 3 instances and 3 seeds to compare direct reuse, selective refresh, a full SEQR rerun, and online execution, assessing both routine reuse and when evolution invalidates the current configuration.

Metrics and supervision. We define the evaluation metrics and supervision protocols for compilation, certification and comparisons.

Evaluation metrics. Following SemBench, we report query-macro F_1 , runtime and LLM monetary cost. For evolution, we additionally report pre- and post-refresh F_1 , certificate slack and violations, and safe-reuse, required-refresh, unsafe-reuse and unnecessary-refresh rates at tolerance $\eta \in [0, 1]$.

Annotation and certification protocol. For each semantic query q_i^S , LiteDBX partitions its labeled tuple set O_i into an initial set T_i and a disjoint certification set V_i , where $|V_i| = \lceil 0.3\beta_{\text{lab}} \rceil$ and $|O_i| \leq \beta_{\text{lab}}$. It runs SEQR on T_i to obtain the auxiliary rewrite $\tilde{q}_i^T$ and uses the held-out labels in V_i to certify the final rewrite $\tilde{q}_i^T$, as described in Section 5.3. All labels outside O_i remain hidden until evaluation. We separately record requested and completed labels, annotation time, LLM calls, and monetary cost; pseudo- and propagated labels are outputs of SEQR.

Matched supervision. On commonly supported workloads, LiteDBX, ThalamusDB and BARGAIN receive the same label source and budget β_{lab} while retaining their native sample-selection procedures. Labels are revealed only for selected tuples, and every source request, including each adaptive BARGAIN oracle call, counts against the budget; unused requests are not reassigned. For same-LLM

labeling, all methods use the same model and prompt, and we measure calls, latency, and token cost. iPDB receives no labels and is evaluated separately using the same online model and prompt.

We report human annotation time and cost together with LLM calls, latency, and monetary cost. For human labeling, we estimate time and cost using $t_{\text{lab}} = 20$ s and $c_{\text{lab}} = \$0.0667$ per label. Annotation is treated as a one-time compilation cost and excluded from recurring execution costs. Table 6 reports the supervision accounting.

Exp-1: Compiling semantic interpretation. We first tested whether SEQR can compile semantic interpretation for recurring execution without sacrificing answer quality, and then calibrated its static guarantee and schema-enrichment budget.

Accuracy under routine workloads. We tested whether compiled execution preserves answer quality under routine workloads. Table 7 reports the per-query F_1 , monetary cost, and runtime, including unsupported workloads and one-hour timeouts. As shown there, LiteDBX achieves the best overall accuracy, with an average F_1 of 0.92 across the five dataset workloads. Besides, (1) across the tested datasets, LiteDBX outperforms the zero-shot LOTUS, Palimpzest, BigQuery and LOTUS+Cache by 37.3%, 16.5%, 23.9% and 34.1%, respectively. It matches the strongest baseline on Wildlife and outperforms the strongest baseline on the other four datasets, ranging from 6.5% to 21.4%. These results demonstrate consistent improvements across datasets and modalities. (2) LiteDBX achieves the highest F_1 of 0.68 on the most challenging dataset Medical, while achieving $F_1 \geq 0.96$ on the other datasets. (3) Although LOTUS+Cache approaches LiteDBX’s recurring execution cost, its F_1 remains 25.4% lower. That is, caching avoids repeated calls for previously matched instances but does not construct the query-aware representation and executable predicates needed for accurate semantic interpretation. These results validate the end-to-end benefit of compilation.

Matched supervision. We tested whether LiteDBX’s accuracy gains persist when the label source and budget are controlled. On commonly supported workloads, we gave LiteDBX, ThalamusDB, and BARGAIN the same label/oracle source and budget $\beta_{\text{lab}} = 20$ per query, while retaining each system’s own procedure. We tested simulated user labels, revealed only when requested, and same-LLM labels under the same model and prompt, charging every call.

As shown in Table 8, (1) With simulated user labels, LiteDBX improves average F_1 over ThalamusDB and BARGAIN by 161.3% and 67.3%, respectively; with same-LLM labels, the gains are 73.4% and 46.2%. Since the source and budget are matched, these gains arise from how SEQR uses the supervision for enrichment and rewriting. These show the advantage of LiteDBX *w.r.t.* baselines under matched supervision.

Static-bound. We empirically evaluated the coverage and tightness of the lower bound in Theorem 3. We use $|V_i| = \lceil 0.3\beta_{\text{lab}} \rceil$, $\beta_{\text{lab}} = 45$, $\delta = 0.2$, and 3 independent workload-level trials without query-specific recalibration. Each trial reruns annotation and

Method	Metric	Movie		Wildlife	E-Comm.			MMQA					Medical			
		Q ₁	Q ₂	Q ₁	Q ₁	Q ₂	Q ₃	Q ₁	Q ₂	Q ₃	Q ₄	Q ₅	Q ₁	Q ₂	Q ₃	Q ₄
LiteDBX	F_1	0.96	1.00	1.00	1.00	1.00	0.87	0.92	1.00	1.00	1.00	1.00	0.62	0.89	0.65	0.57
	\$	0.0003	0.0002	0.0005	0.0002	0.0003	0.0006	0.0004	0.0004	0.0003	0.0004	0.0002	0.0005	0.0004	0.0003	0.0004
	Time	6.51	3.59	9.20	4.59	8.60	6.40	7.44	7.90	0.68	7.76	4.96	8.31	7.39	4.95	4.50
LOTUS	F_1	0.88	0.80	0.89	1.00	0.67	0.64	0.76	0.75	0.28	0.20	0.39	0.33	0.79	0.36	0
	\$	0.8520	0.0490	0.1100	0.9800	1.1500	1.2900	0.0750	0.0800	0.0771	0.0900	0.1000	0.5800	0.1670	0.0520	0.1800
	Time	992.96	67.00	406.98	881.00	1996.78	1978.40	111.61	77.07	138.41	104.59	128.46	683.00	307.41	77.50	291.63
Palimpzest	F_1	0.88	0.80	1.00	1.00	0.50	0.51	0.76	0.85	1.00	1.00	1.00	0.26	0.88	0.61	0.31
	\$	0.8600	0.0380	0.0660	0.8300	1.2700	1.5200	0.0426	0.0430	0.0322	0.0320	0.0330	0.7000	0.2680	0.1023	0.2800
	Time	939.53	89.68	146.10	711.23	418.23	890.00	51.00	33.22	54.14	25.11	52.36	689.12	271.33	115.85	176.28
ThalamusDB	F_1	0.46	0.78	0.33	0.73	0.22	X	0.80	1.00	0.09	0.08	0.07	0.32	0.19	0.28	TO
	\$	0.6800	0.0780	0.1000	1.3700	0.2880	X	0.1000	0.1000	0.1500	0.2400	0.2400	0.8200	0.0730	0.0360	TO
	Time	2262.36	263.27	251.90	1115.02	224.67	X	503.26	370.85	1332.12	1237.96	1062.16	2350.88	292.17	311.81	TO
BigQuery	F_1	0.85	0.60	1.00	1.00	0.30	0.58	0.80	0.60	1.00	1.00	0.63	0.19	0.85	0.63	0.56
	\$	0.1037	0.0062	0.1093	0.2142	1.5122	0.7551	0.0151	0.0152	0.0096	0.0095	0.0097	0.0465	2.7591	0.3598	0.2900
	Time	121.24	8.48	168.97	221.53	1808.68	276.19	2.46	4.14	18.90	17.30	17.40	59.01	1103.59	484.13	98.14
BARGAIN	F_1	0.88	0.64	X	0.88	X	X	0.76	0.40	0.04	0.05	0.21	0.17	X	X	X
	\$	1.1400	0.0750	X	0.1874	X	X	0.0026	0.0046	0.0015	0.0015	0.0015	0.0230	X	X	X
	Time	1209.25	114.49	X	29.69	X	X	18.26	13.89	5.03	10.75	8.43	18.60	X	X	X
iPDB	F_1	0.88	0.63	X	0.39	X	X	0.76	0.57	0.14	0.10	0.21	0.18	X	X	X
	\$	1.3750	0.0800	X	0.1800	X	X	0.0073	0.0073	0.0023	0.0023	0.0024	0.0023	X	X	X
	Time	810.39	55.73	X	84.75	X	X	10.30	8.80	9.80	9.30	9.40	11.59	X	X	X
LOTUS+Cache	F_1	0.86	0.67	1.00	0.92	0.73	0.64	0.85	0.86	0.22	0.29	0.43	0.32	0.79	0.35	0
	\$	0.0003	0.0002	0.0005	0.0002	0.0003	0.0006	0.0004	0.0004	0.0003	0.0004	0.0002	0.0005	0.0004	0.0003	0.0004
	Time	6.78	3.88	9.22	5.70	8.85	7.51	7.46	7.92	6.09	7.78	4.99	9.31	8.41	5.19	4.79

Table 7: Static results across queries. Bold and underlined values denote the best and second-best results; **X** and TO denote unsupported queries and the one-hour timeout, respectively.

Method	Label Source	Movie	Wildlife	E-Comm.	MMQA	Medical
LiteDBX	Benchmark GT (human-sim.)	0.98	1.00	0.94	0.98	0.68
	LLM	0.86	1.00	0.72	0.97	0.52
ThalamusDB	Benchmark GT (human-sim.)	0.77	0.33	0.18	0.16	0.26
	LLM	0.62	0.33	0.48	0.41	0.26
BARGAIN	Benchmark GT (human-sim.)	0.69	X	1.00	0.28	0.17
	LLM	0.76	X	0.88	0.29	0.17

Table 8: Matched supervision under equal budgets. **X** denotes unsupported workloads.

certification for each query using independent seeds while fixing the data, workload and budgets. For each query, we compute U_i , d_i , B_i and $\Phi(\pi_i, B_i)$ and aggregate the query-level bounds over the workload as in Theorem 3. A query-level violation occurs when $\text{Err}^{(i)} > B_i$; a simultaneous violation occurs when any query in a workload trial violates its certificate; and a workload-level violation occurs when the observed macro F_1 gets below the aggregate lower bound. Ground truth outside O_i is used only to compute these evaluation statistics and π_i .

Table 9 evaluates the certificate in Theorem 3. The theorem bounds the final-rewrite error by $B_i = \min\{1, U_i + d_i\}$, where U_i is the full-scope 0–1 error bound for the auxiliary rewrite and d_i is its exact disagreement rate with the final rewrite, and converts B_i into the per-query F_1 lower bound $\Phi(\pi_i, B_i)$, whose average lower-bounds the workload-level macro- F_1 .

We find the following. (1) None of the 45 query-trial cases violates its query certificate, and both the any-query and workload-bound violation rates are zero, empirically validating the guarantee of Theorem 3. (2) Across the 45 query-trial cases, the separately computed medians of U_i , d_i and B_i are 0.035, 0.001 and 0.040, respectively; for each metric, its value is at or below the reported median in at least half of the cases. Specifically, $U_i = 0.035$ certifies that the full-scope 0–1 error of the auxiliary rewrite is at most 3.5%. A disagreement rate $d_i = 0.001$ means that the auxiliary and final rewrites disagree on no more than 0.1% of the tuples in $\Sigma_i(\mathcal{D})$. The final-rewrite bound $B_i = 0.040$ certifies that its full-scope 0–1 error is at most 4.0%. These low medians indicate small certified errors and little

disagreement introduced by the final rewrite. (3) LiteDBX produces nonzero query-level F_1 bounds in 48.9% of the cases, where $B_i < \pi_i$ yields a nontrivial per-query guarantee. For the remaining cases, $B_i \geq \pi_i$ makes $\Phi(\pi_i, B_i) = 0$; the error certificate remains valid, while the resulting slack equals the observed F_1 and reflects conservative error-to- F_1 conversion rather than a certificate violation.

Varying β_{SE} . We varied the schema-enrichment budget β_{SE} from 1 to 9 on MMQA in Figure 5. We find that the F_1 initially increases, then stabilizes, and finally declines when β_{SE} exceeds 5. This is because (a) a larger budget admits more informative semantic attributes, improving schema expressiveness and rewrite accuracy; in contrast, (b) an excessive budget introduces noise and redundancy, increasing the risk of overfitting under limited supervision. We therefore recommend setting $\beta_{SE} = 5$.

Exp-2: Reuse and maintenance under evolution. We examined whether the enriched configuration remains reusable as data and semantic predicates evolve, and how much evolution it can accommodate before requiring selective refresh. For data evolution, denote by $\Delta\mathcal{D}^E$ (resp. $\gamma = |\Delta\mathcal{D}^E|/|\mathcal{D}^E|$) an update (resp. update ratio).

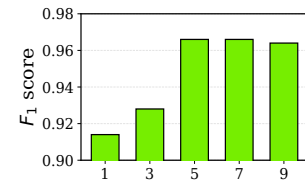


Figure 5: MMQA: varying β_{SE} (F_1).

Dataset	Query	Median U_i	Median d_i	Median B_i	Median Bound	Median F_1	Nonzero Bound (%)	Cert. Viol. (%)	Slack Med./P90/Max
Movie	Q_1	0.139	0.001	0.140	0.917	0.963	100	0	0.046/0.047/0.047
	Q_2	0.000	0.000	0.000	1.000	1.000	100	0	0.000/0.000/0.000
Wildlife	Q_1	0.000	0.000	0.000	1.000	1.000	100	0	0.000/0.000/0.000
	Q_2	0.000	0.000	0.000	1.000	1.000	100	0	0.000/0.000/0.000
E-Commerce	Q_1	0.125	0.073	0.200	0.000	1.000	0	0	1.000/1.000/1.000
	Q_2	0.242	0.151	0.394	0.000	1.000	33	0	1.000/1.000/1.000
	Q_3	0.025	0.000	0.030	0.700	0.960	100	0	0.260/0.280/0.285
MMQA	Q_1	0.035	0.005	0.040	0.000	1.000	0	0	1.000/1.000/1.000
	Q_2	0.015	0.000	0.015	0.000	1.000	0	0	1.000/1.000/1.000
	Q_3	0.015	0.000	0.015	0.000	1.000	0	0	1.000/1.000/1.000
	Q_4	0.026	0.000	0.026	0.289	1.000	67	0	0.711/0.942/1.000
	Q_5	0.110	0.003	0.113	0.000	0.640	33	0	0.451/0.602/0.640
Medical	Q_1	0.168	0.077	0.256	0.808	0.897	100	0	0.089/0.142/0.156
	Q_2	0.431	0.146	0.550	0.000	0.667	0	0	0.667/0.668/0.668
	Q_3	0.443	0.545	0.953	0.000	0.531	0	0	0.531/0.570/0.580
	Q_4								

Table 9: Per-query static-certificate calibration and decomposition.

Data evolution. We evaluated whether the current configuration remains accurate and reusable as the data grows. Table 10 reports per-query F_1 /cost/runtime. As shown there, LiteDBX achieves an average F_1 of 0.91 across five workloads and three growth levels. Besides, we find the following. (1) LiteDBX improves average F_1 over LOTUS, Palimpzest, ThalamusDB, BigQuery and LOTUS+Cache by 37.3%, 21.2%, 76.1%, 27.7% and 40.3%, respectively, up to 182.6%, 124.1%, 225.0%, 66.7% and 242.1%. (2) As data growth increases from 0 to 33% and 66%, its average F_1 changes from 0.92 to 0.91 and 0.90, respectively. Even at 66% growth, it retains $F_1 \geq 0.91$ on the first four workloads and 0.67 on the more challenging Medical workload. (3) LiteDBX triggers selective refresh for only 46.7% of the workloads. For the remaining workloads, it retains the valid semantic attributes and rewrites, updating only affected tuples within the relevant scope. This result validates LiteDBX’s effectiveness under routine data growth without frequent recompilation.

Data-certificate calibration. We varied γ from 0 to 4/6 to evaluate the dynamic certificate in Theorem 4. Before refresh, Table 11 reports the query-level slack $B_i^{\text{new}} - \text{Err}_{\text{new}}^{(i)}$ of the unchanged pre-update rewrite, where a negative value indicates a query-certificate violation. An any-query violation occurs when at least one affected query violates its certificate, while a workload-level violation occurs when the observed pre-refresh macro F_1 falls below $\frac{1}{|Q^S|} \sum_i \Phi(\pi_i^{\text{new}}, B_i^{\text{new}})$. A query is reusable when $B_i^{\text{new}} \leq \eta$, and the configuration is reused only if all affected queries pass.

Table 11 evaluates the certificate in Theorem 4. The theorem transfers the pre-update error certificate to B_i^{new} , which, with probability at least $1 - \delta$, bounds the updated query error and yields a workload-level F_1 lower bound.

We find the following. (1) Across the calibration trials, all query-level slacks are nonnegative, and both the any-query and workload-bound violation rates are zero, empirically validating the guarantee of Theorem 4. (2) Across the calibration trials, the mean query-level slack is 0.14. Specifically, B_i^{new} exceeds the observed pre-refresh error of the unchanged rewrite on the updated data by 0.14 on average, indicating a valid but conservative query-level certificate. (3) LiteDBX produces nonzero query-level F_1 bounds in 46.7% of the cases, where $B_i^{\text{new}} < \pi_i^{\text{new}}$ yields a nontrivial per-query guarantee. For the remaining cases, $B_i^{\text{new}} \geq \pi_i^{\text{new}}$ makes $\Phi(\pi_i^{\text{new}}, B_i^{\text{new}}) = 0$, so the conversion gives no nontrivial F_1 guarantee; the error certificate remains valid and can still guide reuse through $B_i^{\text{new}} \leq \eta$. (4) In the separate end-to-end maintenance evaluation in Table 12, the unsafe-reuse and unnecessary-refresh rates are 6.7% (2/30) and 13.3% (4/30), respectively. Unsafe reuse occurs when $B_i^{\text{new}} \leq \eta$ but the observed

error exceeds η , whereas unnecessary refresh occurs when $B_i^{\text{new}} > \eta$ but the observed error is at most η . Under the theorem’s high-probability guarantee, the certificate avoids unsafe reuse in 93.3% of the decisions and limits unnecessary refresh to 13.3%.

Ablation of incremental maintenance. We tested incremental maintenance using the same pre-update configurations and update traces. For query evolution, we compared LiteDBX with the following variants: (a) LiteDBX_{reuse}, which always reuses the current configuration; (b) LiteDBX_{align}, which performs semantic alignment without rewrite-consistency checking; and (c) LiteDBX_{noMain}, which reconstructs the entire configuration after each update. For data evolution, where semantic alignment does not apply, we compared LiteDBX with LiteDBX_{reuse} and LiteDBX_{noMain}. We report pre- and post-refresh F_1 , unsafe-reuse and unnecessary-refresh rates, refresh fraction, runtime, and monetary cost.

As shown in Table 12, (1) for query updates, LiteDBX reduces the refresh fraction from 75.0% to 47.1% and runtime from 24.5 s to 16.9 s relative to LiteDBX_{align}, with only 0.02 lower post-refresh F_1 (0.91 vs. 0.93). (2) For data updates, LiteDBX refreshes only 46.7% of the affected state while reducing the unsafe-reuse rate by 83.3% over LiteDBX_{reuse} and the unnecessary-refresh rate by 66.8% over LiteDBX_{noMain}. (3) LiteDBX achieves a final post-update F_1 within 0.01 of LiteDBX_{noMain} and reduces runtime and monetary cost by 46.1% and 76.9%, respectively, and outperforms LiteDBX_{reuse} by 7.4%. This validates LiteDBX’s effectiveness for incremental maintenance via selective refresh.

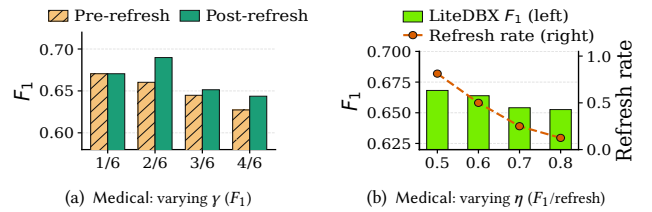


Figure 6: Varying update magnitude and tolerance.

Varying update magnitude and tolerance. We varied γ and η on Medical, the most challenging dataset, in Figures 6(a) and 6(b), respectively. As shown there, (1) as γ increases from 1/6 to 4/6, pre-refresh F_1 changes from 0.67 to 0.63; at $\gamma = 2/6$, SEQR refresh yields 0.69. (2) At $\eta = 0.5, 0.6, 0.7$, and 0.8 , the resulting F_1 scores are 0.67, 0.66, 0.65 and 0.65, respectively. This is because larger updates cause greater drift from the current configuration, while a larger η permits more error before triggering refresh. These results indicate that LiteDBX is robust to varying update magnitudes and effectively balances accuracy and refresh frequency through η .

Dataset	Query	γ	LiteDBX	LOTUS	Palimpzest	ThalamusDB	BigQuery	BARGAIN	iPDB	LOTUS+Cache
Movie	Q_1	0	0.96/0.0053/142.57	0.87/0.5270/629.46	0.88/0.5200/575.59	0.67/0.7000/2590.47	0.62/0.0626/71.82	0.90/0.0154/22.55	0.91/0.0197/41.76	0.86/0.5200/501.68
		33%	0.96/0/26.80	0.87/0.6940/769.09	0.88/0.6900/790.16	0.54/0.6900/2345.98	0.75/0.0832/115.38	0.91/0.0201/22.97	0.91/0.0259/76.19	0.85/0.1600/173.14
		66%	0.96/0/57.28	0.88/0.8520/992.96	0.88/0.8600/939.53	0.46/0.6800/2262.36	0.85/0.1037/121.24	0.91/0.0231/26.84	0.92/0.0321/67.69	0.86/0.1700/396.11
	Q_2	0	1.00/0.0053/27.41	0.72/0.0500/64.85	0.80/0.0070/43.12	0.78/0.0750/287.51	0.60/0.0061/6.74	0.41/0.0020/7.61	0.50/0.0019/5.31	0.67/0.0490/61.25
		33%	1.00/0/4.66	0.75/0.0500/70.81	0.60/0.0120/57.84	0.83/0.0780/291.85	0.60/0.0061/7.33	0.37/0.0020/9.95	0.48/0.0019/7.15	0.67/0/0.02
		66%	1.00/0/6.06	0.67/0.0500/78.65	0.80/0.0380/89.68	0.78/0.0780/263.27	0.60/0.0062/8.48	0.41/0.0020/10.34	0.50/0.0019/5.45	0.67/0/0.04
Wildlife	Q_1	0	1.00/0.0204/196.18	1.00/0.0650/256.89	1.00/0.0400/91.92	1.00/0.0600/175.25	1.00/0.0655/76.92	X/X/X	X/X/X	1.00/0.0650/254.74
		33%	1.00/0/9.61	0.86/0.0870/334.79	0.85/0.0500/120.04	0.86/0.0800/258.06	0.85/0.0873/298.40	X/X/X	X/X/X	0.89/0.0220/91.58
		66%	1.00/0/21.93	0.89/0.1100/406.98	1.00/0.0650/149.07	0.33/0.1000/296.38	1.00/0.1093/168.97	X/X/X	X/X/X	0.83/0.0220/180.68
E-Commerce	Q_1	0	1.00/0.0050/25.10	1.00/0.5700/545.60	1.00/0.4900/455.26	0.67/1.3600/1037.57	1.00/0.1283/132.63	0.67/0.1102/18.41	0.42/0.1057/50.59	0.86/0.5800/523.91
		33%	1.00/0/1.44	0.93/0.7700/743.57	1.00/0.6600/531.53	0.73/1.3900/1078.10	1.00/0.1709/87.43	1.00/0.1483/23.61	0.67/0.1423/67.16	0.92/0.1800/175.87
		66%	1.00/0/2.68	1.00/0.9800/923.66	1.00/0.8300/711.23	0.73/1.3700/1115.02	1.00/0.2100/221.53	0.88/0.1874/29.69	0.40/0.1799/95.64	0.92/0.3900/545.64
	Q_2	0	1.00/0.0177/60.92	0.89/0.6800/1168.74	0.56/0.7700/255.38	0.50/0.2900/216.41	0.42/1.5134/1360.36	X/X/X	X/X/X	0.57/0.6900/1167.87
		33%	1.00/0/13.44	0.73/0.9100/1551.15	0.48/1.0200/338.88	0.40/0.2900/236.57	0.45/1.5136/2155.81	X/X/X	X/X/X	0.73/0.2200/374.68
		66%	1.00/0/17.74	0.50/1.1500/1946.06	0.50/1.2700/418.23	0.22/0.2880/224.67	0.30/1.5122/1808.68	X/X/X	X/X/X	0.73/0.2400/793.09
	Q_3	0	0.92/0.0147/117.51	0.70/0.8300/1469.46	0.58/0.9000/389.65	X/X/X	0.67/0.4489/247.02	X/X/X	X/X/X	0.67/0.8200/1166.05
		33%	0.80/0/20.00	0.76/1.1000/1587.11	0.56/1.1900/563.89	X/X/X	0.60/0.5953/128.90	X/X/X	X/X/X	0.67/0.2600/376.14
		66%	0.74/0/41.10	0.70/1.3700/1978.40	0.51/1.5100/889.10	X/X/X	0.58/0.7551/276.19	X/X/X	X/X/X	0.64/0.5500/787.93
MMQA	Q_1	0	0.94/0.0047/61.20	0.85/0.0460/62.24	0.86/0.0250/34.11	0.93/0.0550/229.36	0.83/0.0093/1.96	0.76/0.0026/19.13	0.92/0.0044/5.13	0.85/0.0460/61.27
		33%	0.91/0/5.25	0.85/0.0620/90.06	0.86/0.0340/34.12	0.90/0.0760/347.80	0.87/0.0123/3.15	0.58/0.0048/15.55	0.88/0.0059/6.42	0.85/0.0160/31.23
		66%	0.81/0/36.80	0.73/0.0770/95.37	0.80/0.0400/43.70	0.80/0.1000/503.26	0.80/0.0151/2.46	0.62/0.0052/22.71	0.79/0.0073/8.27	0.85/0.0150/54.23
	Q_2	0	1.00/0.0043/57.80	0.86/0.0490/61.81	0.75/0.0470/54.90	0.85/0.0550/233.12	0.60/0.0092/1.58	0.40/0.0045/15.55	0.67/0.0045/5.24	0.86/0.0490/55.91
		33%	1.00/0/3.18	1.00/0.0630/80.31	0.86/0.0630/77.26	0.86/0.0740/297.81	0.55/0.0122/2.35	0.33/0.0053/16.44	0.57/0.0059/5.72	0.86/0.0150/20.21
		66%	1.00/0/15.87	0.86/0.0800/104.04	0.86/0.0780/94.00	1.00/0.1000/370.85	0.60/0.0152/4.14	0.30/0.0063/17.25	0.67/0.0073/7.59	0.86/0.0160/46.37
	Q_3	0	1.00/0.0044/35.54	0.20/0.0500/69.90	1.00/0.0190/28.61	0.0/0.1300/801.23	1.00/0.0057/11.50	0.04/0.0015/7.07	0.18/0.0014/4.88	0.29/0.0500/61.49
		33%	1.00/0/11.49	0.13/0.0720/95.80	1.00/0.0260/35.21	0.0/0.1500/950.88	1.00/0.0077/11.70	0.03/0.0020/7.55	0.13/0.0018/5.85	0.22/0.0170/27.70
		66%	1.00/0/15.61	0.25/0.0870/114.67	1.00/0.0320/54.14	0.08/0.1900/1101.00	1.00/0.0096/18.90	0.02/0.0025/10.43	0.11/0.0023/7.43	0.22/0.0160/59.05
	Q_4	0	1.00/0.0053/46.78	0.29/0.0480/76.39	1.00/0.0370/40.47	0.13/0.1270/585.81	1.00/0.0058/10.08	0.05/0.0015/7.04	0.15/0.0014/5.05	0.33/0.0550/55.94
		33%	1.00/0/2.05	0.15/0.0660/93.97	0.50/0.0500/73.23	0.08/0.1500/874.61	1.00/0.0077/12.31	0.04/0.0020/7.56	0.13/0.0018/6.15	0.29/0.0210/48.68
		66%	1.00/0/7.17	0.22/0.0880/130.14	1.00/0.0630/87.99	0.08/0.2400/1237.96	1.00/0.0095/17.30	0.04/0.0024/12.44	0.10/0.0023/7.02	0.29/0.0160/74.91
	Q_5	0	1.00/0.0044/64.98	0.42/0.0610/121.39	1.00/0.0200/29.57	0.17/0.1500/869.18	0.56/0.0058/9.95	0.20/0.0015/7.67	0.31/0.0014/5.01	0.53/0.0600/95.49
		33%	1.00/0/8.82	0.42/0.0770/133.36	1.00/0.0270/37.97	0.18/0.1900/910.01	0.59/0.0077/10.06	0.17/0.0020/8.78	0.19/0.0018/6.49	0.45/0.0190/35.97
		66%	1.00/0/14.58	0.42/0.1000/151.44	1.00/0.0330/52.36	0.07/0.2400/1062.16	0.63/0.0097/17.40	0.17/0.0024/14.05	0.23/0.0023/7.15	0.43/0.0190/73.36
Medical	Q_1	0	0.67/0.0042/114.85	0.20/0.3400/457.13	0.13/0.1800/222.71	0.17/0.5900/1784.00	0.10/0.0468/15.69	0.09/0.0138/15.04	0.15/0.0043/9.28	0.10/0.3600/377.44
		33%	0.61/0/18.02	0.20/0.4600/614.50	0.24/0.2300/292.89	0.22/0.7300/2084.25	0.12/0.0465/15.64	0.11/0.0183/17.89	0.09/0.0128/28.44	0.25/0.1900/213.35
		66%	0.61/0/42.44	0.33/0.5800/683.00	0.26/0.7000/689.12	0.32/0.8200/2350.88	0.19/0.0465/59.01	0.17/0.0230/18.63	0.15/0.0175/40.16	0.32/0.0410/266.04
	Q_2	0	0.87/0.0129/440.15	0.48/0.1000/195.78	0.52/0.1590/172.26	0.17/0.0740/298.26	0.64/2.7591/1117.91	X/X/X	X/X/X	0.42/0.1000/116.63
		33%	0.88/0/133.57	0.63/0.1300/253.71	0.70/0.2100/239.02	0.16/0.0740/284.61	0.75/2.7591/1340.52	X/X/X	X/X/X	0.68/0.0600/71.30
		66%	0.86/0/249.76	0.79/0.1670/307.41	0.88/0.2680/271.33	0.19/0.0730/292.17	0.85/2.7591/1103.59	X/X/X	X/X/X	0.79/0.0100/93.06
	Q_3	0	0.52/0.0137/153.05	0.22/0.0320/52.59	0.32/0.0630/79.86	0.25/0.0360/333.21	0.43/0.3598/463.88	X/X/X	X/X/X	0.24/0.0310/46.60
		33%	0.69/0/36.27	0.30/0.0430/67.08	0.62/0.0800/104.55	0.25/0.0360/300.63	0.54/0.3598/459.30	X/X/X	X/X/X	0.32/0.0180/35.06
		66%	0.63/0/71.39	0.36/0.0520/77.50	0.61/0.1023/115.85	0.28/0.0360/311.81	0.63/0.3598/484.13	X/X/X	X/X/X	0.35/0.0040/45.63
	Q_4	0	0.55/0.0185/404.30	0.0/0.1200/180.00	0.20/0.1700/98.36	TO/TO/TO	0.39/0.1749/65.48	X/X/X	X/X/X	0.0/0.1100/173.51
		33%	0.54/0/80.12	0.01/0.1500/229.94	0.26/0.2200/132.88	TO/TO/TO	0.48/0.2303/43.16	X/X/X	X/X/X	0.0/0.0610/97.59
		66%	0.56/0/160.15	0.0/0.1800/285.29	0.31/0.2800/176.28	TO/TO/TO	0.56/0.2900/98.14	X/X/X	X/X/X	0.0/0.1500/128.72

Table 10: Data evolution ($\gamma \in \{0, 33\%, 66\%\}$). Each entry is F_1 /cost/runtime; **X** and TO denote unsupported queries and the one-hour timeout, respectively. Best results are bolded; second-best are underlined.

Dataset	Median B_l^{new}	Mean Slack	Query Viol.	F_1 Viol.	Nonzero Bound / Query Counts
Movie	0.130	0.046	0	0	2 / 2
Wildlife	0.250	0	0	0	1 / 1
E-Commerce	0.0193	0.314	0	0	1 / 3
MMQA	0.033	0.030	0	0	2 / 5
Medical	0.562	0.295	0	0	1 / 4

Table 11: Dynamic-certificate calibration.

Variant	Pre- F_1	Post- F_1	Unsafe Reuse (%)	Unnecessary Refresh (%)	Refresh (%)	Time (s)	\$
LiteDBX	0.99	0.91	11.1	12.5	47.1	16.9	0.0023
LiteDBX _{reuse} (Always reuse)	0.99	0.62	52.9	0	0	0	0.0001
LiteDBX _{align} (Align only)	0.99	0.93	0	25.0	75.0	24.5	0.0034
LiteDBX _{noMain} (Full SEQR)	0.99	0.98	0	41.2	100	66.6	0.0047

(a) Query evolution.

Variant	Pre- F_1	Post- F_1	Unsafe Reuse (%)	Unnecessary Refresh (%)	Refresh (%)	Time (s)	\$
LiteDBX	0.88	0.87	6.7	13.3	46.7	110.1	0.0021
LiteDBX _{reuse} (Always reuse)	0.88	0.81	40.0	0	0	25.7	0
LiteDBX _{noMain} (Full SEQR)	0.88	0.88	0	40.0	100	204.3	0.0091

(b) Data evolution.

Table 12: Ablation of incremental maintenance.

Predicate Stratum	Always reuse F_1	Selective refresh F_1	Full-SEQR F_1	Strongest online F_1	Safe Reuse (%)	Required Refresh (%)	Runtime Saving (%)	Cost Saving (%)
Aligned/paraphrased	0.98	0.98	0.98	0.92	100.0	20.0	69.4	79.9
Recombined known	0.41	0.91	0.95	0.88	50.0	33.3	74.9	61.4
Unseen	0.34	1.00	1.00	0.93	—	100.0	57.0	—11.9
Revised criterion	0.56	0.84	1.00	1.00	66.7	50.0	86.3	53.7

Table 13: Complete query-evolution sweep and reuse breakdown.

Semantic-drift boundary and selective refresh. We evaluated when query evolution requires refresh across aligned or paraphrased, recombined-known, unseen, and revised predicate units. For each category, we compared direct reuse, selective refresh, a full SEQR

rerun, and online semantic processing. Safe reuse measures the fraction of reusable units accepted by semantic alignment and rewrite-consistency checking, while required refresh measures the fraction of units with invalidated support that are refreshed.

Component	Initial Compilation			Selective Refresh		
	Time (s)	LLM \$	Human Labels/Time (s)	Time (s)	LLM \$	Affected Tuples
Attribute generation	26.8	0.0089	—	15.6	0.0021	—
Annotation	19.6	0	20 / 400	10.5	0	502.6
Attribute population	169.1	0	—	76.7	0	502.6
Coreset/pseudo labels	0.1	0	—	0	0	41.1
Query rewriting	53.8	0	—	7.3	0	—
Total (per query)	269.4	0.0089	20 / 400	110.1	0.0021	1046.3

Table 14: Execution and maintenance costs.

Setting	Label Source	t_{lab}	c_{lab}	Runtime Break-even N	Monetary Break-even N
Human time only	Human	20 s	0	1–47	1
Measured human	Human	20 s	\$ 0.0667	1–47	1–42
Low effort	Human	10 s	\$ 0.0333	1–30	1–22
High effort	Human	30 s	\$ 0.1000	1–63	2–63
Same LLM	LLM	Measured	Measured	1–14	1

Table 15: Annotation-effort sensitivity.

Table 13 reports accuracy, reuse decisions, and runtime and monetary-cost savings over a full rerun. (1) For aligned or paraphrased and recombined-known units, direct reuse remains within 0.00 and 0.54 of a full rerun in F_1 , with safe-reuse rates of 100% and 50%, respectively; e.g., on MMQA, when the update at $|Q^S| = 3$ fails the coverage-and-consistency test, direct reuse reduces F_1 from 0.95 to 0.41, whereas selective refresh restores it to 0.91. (2) For unseen and revised units, direct reuse is 0.66 and 0.44 lower in F_1 than a full rerun, whereas selective refresh recovers 0.66 and 0.28 by refreshing 100% and 50% of the units. (3) Across all four categories, selective refresh remains within 0.16 of a full rerun in F_1 and reduces runtime and monetary cost by 71.9% and 45.8%, respectively. These show that reuse requires semantically complete attribute-predicate lineage; otherwise, selective refresh is needed to preserve accuracy.

Exp-3: Efficiency, amortization, and scalability. We supplemented the main results with a component-level cost breakdown, sensitivity to annotation effort, and analyses of storage overhead and scalability. Monetary costs below \$0.0001 are reported as zero. *Compilation and refresh cost breakdown.* Table 14 breaks down the execution and maintenance costs reported in Tables 4 and 5. Certification is included in the SEQR stages: initial compilation includes the auxiliary SEQR run on T_i and final construction from O_i , while query rewriting includes the full-scope disagreement scan. Human annotation is reported by label count and time, whereas LLM labeling and other paid calls are reported by monetary cost.

As Table 14 shows, (1) initial compilation takes 269.4 s and costs \$0.0089, and attribute generation and population account for 9.9% and 62.8% of the runtime, respectively. (2) Selective refresh processes 1046.3 tuples and takes 110.1 s, reducing runtime and monetary cost by 46.1% and 76.9%, respectively, over full reconstruction. (3) Annotation accounts for 7.3% of the initial construction time, while coreset construction and query rewriting together account for 20.0%. These validate the effectiveness of selective refresh in reducing maintenance costs by avoiding unaffected compilation.

Annotation-effort sensitivity. We further analyzed the compilation break-even results in Figures 3(f) and 3(g) under different labeling sources and effort assumptions. We considered human-time-only, measured-human, low-effort and high-effort settings, as well as LLM labeling. Human-time-only excludes monetary annotation costs, while LLM labeling uses measured latency and token costs.

As Table 15 shows, the runtime break-even point ranges from 1 to 63 executions across the human settings and from 1 to 14 under LLM

labeling; the corresponding monetary break-even ranges are 1-63 and 1-1, respectively. Thus, although the exact break-even point depends on annotation, compilation remains beneficial for sufficiently repeated workloads. This validates LiteDBX’s benefit for repeated routine queries across different annotation-effort assumptions.

Storage overhead and scalability. We measured the number of added attributes, the tuples populated after scope filtering, and the serialized size of enriched values, coresets, proxy state, and rewrite templates. We then varied (1) $|D|$ from 1,000 to 16,000 on Movie; (2) $\mathcal{U}$ from image-only input to images and text on E-Commerce; and (3) $|Q^S|$ from one to three on MMQA. For each setting, we report runtime and monetary cost; for workload scaling, we also record cross-query attribute reuse. Table 16 reports the complete results. We find the following. (1) Across datasets, LiteDBX adds 4–49 attributes and uses 1.2 MB of persistent state, or 10.4% of the original data size. (2) On Movie, as $|D|$ grows from 1,000 to 4,000, runtime increases from 173.8s to 411.2s, while monetary cost remains \$0.0055. At 16,000 tuples, SEQR takes 1,356.6s and remains at least 4.9 $\times$ faster than LOTUS. This is because runtime grows with the scope-filtered tuples requiring proxy population, while online calls remain budgeted. (3) On E-Commerce, adding text increases runtime from 142.9s to 178.9s and monetary cost from \$0.0224 to \$0.0236, showing modest overhead when incorporating an additional modality. (4) As $|Q^S|$ grows from one to three, monetary cost increases from \$0.0043 to \$0.0132, while runtime rises only from 100.1s to 153.9s. The number of materialized attributes grows more slowly than the workload because related queries reuse shared enriched attributes. These results show that SEQR scales across data sizes, modalities, and workload sizes, while cross-query reuse limits the growth of workload-specific relational state.

Exp-4: Sources of gain and applicability. We isolated LiteDBX’s gains from supervision, schema enrichment and rewriting, and reported the configurations underlying the compilation boundary.

Supervision, enrichment and rewriting. We compared LiteDBX with three variants: LiteDBX_{noSup} replaces the user-provided labels in O_i with LLM-generated labels, leaving SEQR unchanged; LiteDBX_{noEnr} rewrites queries over the raw schema; and LiteDBX_{noRew} retains the enriched schema but evaluates semantic predicates using the learned proxy rather than the selected SQL rewrite. All variants use the same sampled tuples and, where applicable, the same budgets.

As Table 17 shows, (1) LiteDBX beats LiteDBX_{noSup} by 14.1% on average w.r.t. F_1 , with a maximum relative gain of 416.7% on

Dataset	# of Attributes	# of Populated Tuples	Enriched Values (B)	Coresets (B)	Proxy State (B)	Rewrites (B)	Total / Original (%)
Movie	26	2577	51391	19370	4294	411	105.30
Wildlife	4	200	136	42290	1147	198	100.04
E-Commerce	49	6000	3204	895184	4805	513	100.13
MMQA	8	985	24599	28266	4371	613	146.66
Medical	11	5313	145459	16802	7140	636	100.05

(a) Per-dataset storage overhead.

Axis	Setting	Runtime (s)	LLM \$	Reused Attributes
$ \mathcal{D} $	1,000	173.77	0.0055	—
	2,000	234.15	0.0055	—
	4,000	411.24	0.0055	—
	8,000	726.08	0.0055	—
	16,000	1,356.61	0.0055	—
Modality	Image	142.9	0.0224	—
	Image+text	178.9	0.0236	—
$ \mathcal{Q}^S $	1	100.1	0.0043	0
	2	123.1	0.0087	1
	3	153.9	0.0132	2

(b) Detailed scalability results.

Table 16: Storage overhead and scalability.

Dataset	Query	LiteDBX	LiteDBX _{noSup}	LiteDBX _{noEnr}	LiteDBX _{noRew}
Movie	Q_1	0.96	0.84	0.90	0.95
	Q_2	1.00	0.88	1.00	1.00
Wildlife	Q_1	1.00	1.00	1.00	1.00
	Q_2	1.00	0.45	1.00	1.00
E-Commerce	Q_3	0.87	0.70	0.67	0.76
	Q_1	0.92	0.85	0.47	0.85
	Q_2	1.00	1.00	1.00	1.00
MMQA	Q_3	1.00	1.00	1.00	1.00
	Q_4	1.00	1.00	1.00	1.00
	Q_5	1.00	1.00	0.89	1.00
Medical	Q_1	0.62	0.12	0.00	0.24
	Q_2	0.89	0.84	0.73	0.88
	Q_3	0.65	0.61	0.54	0.62
	Q_4	0.57	0.52	0.36	0.04
Average		0.90	0.79	0.77	0.82

Table 17: Supervision, enrichment and rewriting.

Dataset	F_1				Number of predicates in the rewrite template			
	LiteDBX	LiteDBX _{noCore}	LiteDBX _{seq}	LiteDBX _{pred}	LiteDBX	LiteDBX _{noCore}	LiteDBX _{seq}	LiteDBX _{pred}
Movie	0.98	0.98	0.98	0.98	1.00	1.00	1.00	1.00
Wildlife	1.00	1.00	1.00	1.00	2.00	2.00	2.00	2.00
E-Commerce	0.94	0.83	0.82	0.72	1.67	1.00	1.00	1.00
MMQA	0.96	0.98	0.96	0.86	0.40	0.40	0.4	0.8
Medical	0.68	0.52	0.62	0.58	1.50	0.50	1.25	2.25

Table 18: Ablation of SEQR’s algorithmic components.

Medical Q_1 (0.62 vs. 0.12). This is because limited human supervision establishes the intended semantics and improves accuracy. (2) It outperforms LiteDBX_{noEnr} by 16.6% on average. Among queries with nonzero baseline F_1 , its maximum relative gain is 95.7% on MMQA Q_1 (0.92 vs. 0.47); on Medical Q_1 , it raises F_1 from 0 to 0.62. This is because schema enrichment captures query-relevant semantics as reusable structured attributes unavailable in the raw schema. (3) It beats LiteDBX_{noRew} by 9.2% on average, with a maximum relative gain of 1325% on Medical Q_4 (0.57 vs. 0.04). This is because query rewriting converts the enriched representation into an explicit, executable predicate that more accurately captures the intended semantics. These validate the complementary roles of the three components: supervision establishes the semantics, enrichment represents them, and rewriting makes them executable.

Ablation of SEQR’s algorithmic components. We evaluated the contributions of SEQR’s algorithmic components by comparing LiteDBX with three variants under the same β_{lab} , β_{SE} , and β_{rew} : (1) LiteDBX_{noCore} uses only the user-labeled set O_i ; (2) LiteDBX_{seq} selects attributes based on predictive utility and then derives rewrites over the fixed attribute set; and (3) LiteDBX_{pred} retains SEQR’s candidate-configuration search but ranks configurations solely by propagation error, omitting rewrite fidelity.

As shown in Table 18, LiteDBX beats LiteDBX_{noCore}, LiteDBX_{seq}

and LiteDBX_{pred} by 5.8%, 4.1% and 10.1% on average, respectively, up to 30.8%, 14.6% and 30.6%. This is because LiteDBX jointly integrates coreset construction, configuration search, and fidelity-aware selection. By contrast, LiteDBX_{noCore} relies on the sparse and potentially biased O_i ; LiteDBX_{seq} commits prematurely to predictive attributes under a limited rewrite budget; and LiteDBX_{pred} may select configurations that are difficult to rewrite accurately. This result validates the effectiveness of jointly optimizing all these components.

Compilation-boundary configurations. We report the configurations underlying the compilation boundary in Figure 3(h). For each semantic predicate, we evaluated all tested $(\beta_{SE}, \beta_{rew})$ pairs and selected the least costly combination of enriched attributes and a bounded UCQ rewrite whose F_1 is within five points of the strongest applicable online baseline; predicates with no qualifying configuration fall outside the boundary. For compact, attribute-expressible predicates, LiteDBX extracts 4.5 attributes from each record and expresses the semantic predicate using 1.3 UCQ conditions over these attributes. Cross-modal predicates require 4 modality-specific attributes and 2 conditions that combine information across modalities. Representation-heavy or holistic predicates remain within LiteDBX’s tested accuracy boundary but offer a smaller advantage over online semantic processing.